psoc_e84-edgi-talk SDK

Documentation

The Edgi-Talk board is based on the PSoC™ E84 MCU and provides a flexible and comprehensive development platform.It integrates a variety of peripheral interfaces and sample modules, helping developers quickly implement AI+multi-sensor, display, and communication applications.

Version 1.0.0 | English | October 17, 2026

www.rt-thread.org

Copyright (c) 2006-2025, RT-Thread Development Team

Contents

- 1. Basics
  - 1.1. Edgi_Talk_M33_Template Example Project
- 2. Drivers
  - 2.1. Edgi-Talk_M33_Driver_All Multi-Demo Project
  - 2.2. Edgi-Talk_M55_Driver_All Example Project
  - 2.3. Edgi_Talk_M55_WIFI Example Project
- 3. Example
  - 3.1. Edgi-Talk_M55_LVGL Example Project
  - 3.2. XiaoZhi Sample Project
- 4. USB
  - 4.1. Edgi_Talk_Extend_Screen CherryUSB USB Extend Screen Example
  - 4.2. Edgi_Talk_M33_USB_RoleSwitch CherryUSB Example Project
  - 4.3. Edgi_Talk_M55_USB_RoleSwitch CherryUSB Example Project
  - 4.4. Edgi_Talk_M55_USB_UVC CherryUSB Example Project
  - 4.5. Edgi_Talk_USB_Deepcraft Example Project
- 5. IPC
  - 5.1. Edgi_Talk_M33_IPC Dual-Core Communication Example Project
  - 5.2. Edgi_Talk_M55_IPC Dual-Core Communication Example Project
- 6. AI
  - 6.1. Edgi_Talk_M55_DEEPCRAFT_Deploy_Vision
- 7. FAQ
  - 7.1. 常见问题解答（FAQ）
  - 7.2. PWM Device Configurator 配置指南

1. Basics

1.1. Edgi_Talk_M33_Template Example Project

中文 | **English**

Introduction

This template project is based on the **Edgi-Talk platform** and runs a minimal **RT-Thread** application on the Cortex-M33 core.
It is intended to initialize the board, boot the Cortex-M55 core, and then stay idle with all optional peripheral demos disabled.

The shared Secure M33 firmware package lives under `libraries/components/infineon-pse84-secure-firmware-latest`, while keeping the Template `.config` minimal.

Software Description

- The project is developed based on the **Edgi-Talk** platform.
- `SOC_Enable_CM55` is enabled so board initialization starts the M55 core.
- AHT20, LSM6DS3, audio, ADC, RTC, SD card, filesystem, LCD, Wi-Fi, and other optional peripheral demos are disabled.
- External Wi-Fi/audio power control pins are driven low during board initialization.

Usage Instructions

Compilation and Download

1. Open the project and complete the compilation.
2. Connect the board's USB port to the PC using the **onboard debugger (DAP)**.
3. Use the programming tool to flash the generated firmware to the development board.

Runtime Behavior

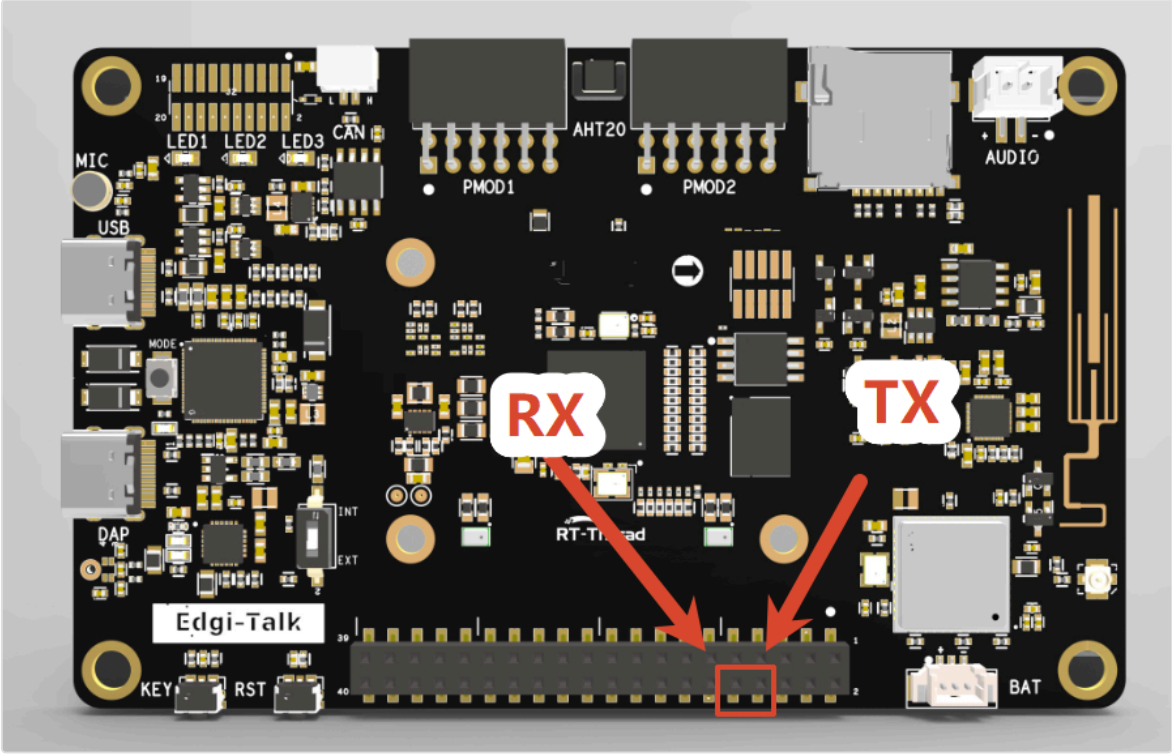
- After flashing, power on the board to run the example project.
- M33 initializes RT-Thread, starts M55, prints a short boot message, and then stays idle.

- The template does not blink LEDs or start peripheral demos automatically.

Notes

> ⚠️ ***Note:*** *This project requires* ***RT-Thread Studio 2.2.9*** *or higher.*

- The M33 project serial log is not output through the onboard DAP virtual COM port directly. To view `msh />` and demo logs, connect an external USB-to-UART adapter, such as CH340. The connection position is shown below. Connect the board RX to the UART adapter TX, connect the board TX to the UART adapter RX, and set the host serial baud rate to 115200:



- To modify the **graphical configuration** of the project, open the configuration file using the following tool:

```
tools/device-configurator/device-configurator.exe
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

- After modification, save the configuration and regenerate the code.
- To run an M55 project, it is recommended to flash **Edgi_Talk_M33_Template** first. It is a clean M33 project and is suitable as the base firmware before starting M55.

Boot Sequence

The system boot sequence is as follows:

```
+------------------+
|   Secure M33     |
|   (Secure Core)  |
+------------------+
          |
          v
+------------------+
|       M33        |
| (Non-Secure Core)|
+------------------+
          |
          v
+-------------------+
|       M55         |
| (Application Core)|
+-------------------+
```

⚠️ Please strictly follow the boot sequence above when flashing firmware; otherwise, the system may not run properly.

- If the example project does not run correctly, compile and flash the **Edgi_Talk_M33_Template** project first to ensure proper initialization and core startup before running this example.
- To enable the M55 core, configure the **M33 project** as follows:

```
RT-Thread Settings --> Hardware --> select SOC Multi Core Mode --> Enable CM5
```

2. Drivers

2.1. Edgi-Talk_M33_Driver_All Multi-Demo Project

Chinese | **English**

Introduction

This project is based on the **Edgi-Talk platform** and runs on **RT-Thread on the Cortex-M33 core**. It integrates multiple common peripheral demos into one firmware, making it convenient to validate sensors, ADC, HyperRAM, key interrupts, RTC, SD card filesystem, external Flash filesystem, and WAV audio playback in a single image.

The project provides the following MSH commands:

Command	Description
`demo_aht20`	Read AHT10/AHT20 temperature and humidity
`demo_lsm6ds3 [count] [delay_ms]`	Read LSM6DS3 6-axis sensor data
`demo_audio start/stop/status [speaker_volume] [mic_gain]`	Real-time audio microphone-to-speaker loopback
`wavplay -s <file>`	Play a WAV audio file
`demo_adc`	Read ADC1 Channel 1 voltage
`demo_hyperram`	Run a basic HyperRAM read/write check
`demo_hyperram_speed [size_kb] [loops]`	Run a HyperRAM bandwidth test
`demo_key`	Register the P8.3 key interrupt and toggle the blue LED when pressed
`demo_rtc [YYYY MM DD HH MM SS]`	Read or set RTC time
`demo_sdcard`	Run an SD card file write/read test
`demo_sdcard_speed [total_kb] [block_kb]`	Run an SD card sequential read/write speed test
`demo_flash_speed [total_kb] [block_kb]`	Run a `/flash` filesystem sequential read/write speed test

Command	Description
`core_mark`	Run the CoreMark CPU benchmark

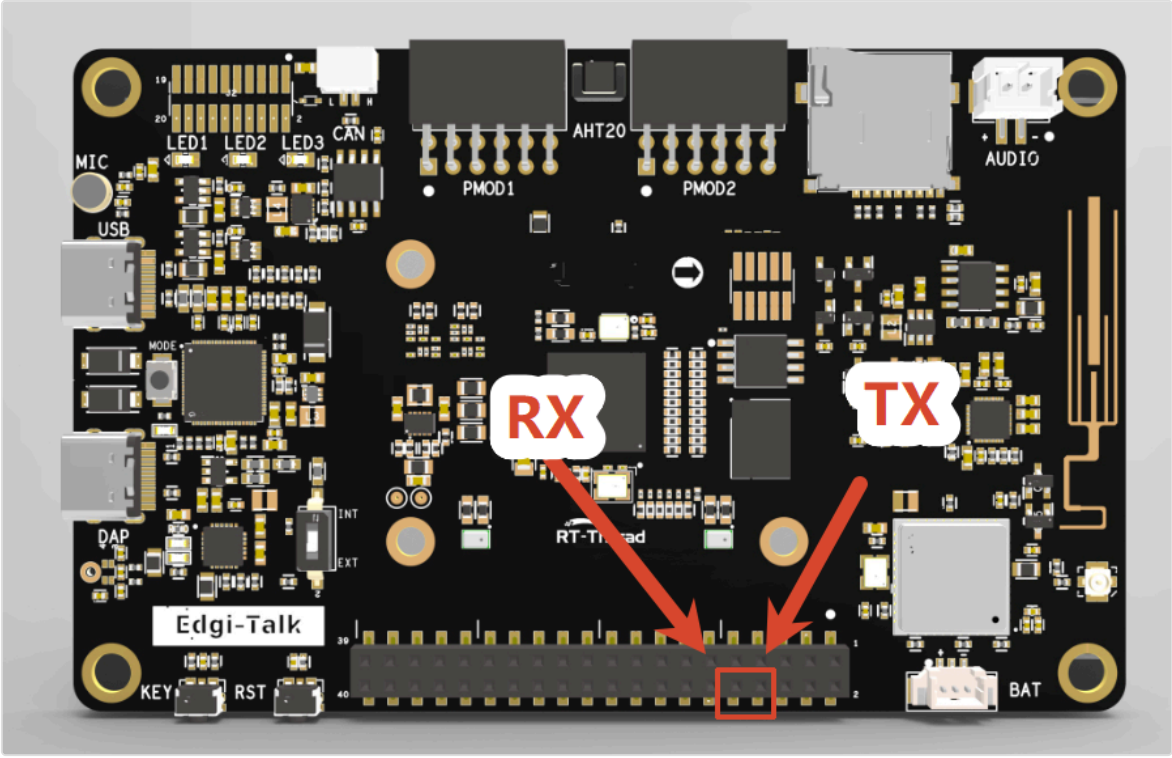
Software Description

- The project is developed for the **Edgi-Talk** platform.
- **RT-Thread** is used as the operating system kernel.
- Each demo is triggered manually through an MSH command to avoid running multiple peripheral tests automatically after boot.
- The Audio demo uses the RT-Thread Audio framework, PDM microphone, and ES8388 codec. It occupies `mic0` and `sound0` after startup.
- The WavPlayer demo uses the RT-Thread `wavplayer` package to play WAV files. It occupies `sound0` during playback and cannot run together with the `demo_audio` loopback.
- CoreMark uses the RT-Thread `CoreMark` package. The default iteration count is `36000`, and it is started manually through an MSH command.
- SD card mounting is handled by the common filesystem initialization logic. The card is mounted automatically after insertion and unmounted after removal.
- External Flash is registered as `norflash0` through FAL. The `filesystem` partition is mounted to `/flash` as littlefs.
- Normal SDIO `CMD5 arg=0` timeout probing logs are silenced to avoid repeated console output during normal SD card detection.

Notes

> ***Note:*** *This project requires* ***RT-Thread Studio 2.2.9*** *or higher.*

- The M33 project serial log is not output through the onboard DAP virtual COM port directly. To view `msh />` and demo logs, connect an external USB-to-UART adapter, such as CH340. The connection position is shown below. Connect the board RX to the UART adapter TX, connect the board TX to the UART adapter RX, and set the host serial baud rate to 115200:



- After editing, save the configuration and regenerate the code.
- Format the SD card as FAT16/FAT32.
- If the SD card speed result is low, check the card speed grade, filesystem fragmentation, SDIO clock, bus width, and read/write block size first.
- `/flash` uses the external Flash `filesystem` partition. If the first mount fails, it is automatically formatted. If you need to preserve data on this partition, do not run a full-chip erase casually.
- If this example project does not run correctly, compile and flash **Edgi_Talk_M33_Template** first to ensure the initialization and core startup flow are normal before running this project.
- To run an M55 project, it is recommended to flash **Edgi_Talk_M33_Template** first. It is a clean M33 project and is suitable as the base firmware before starting M55.

AHT20 Temperature and Humidity Demo

Function

`demo_aht20` uses the AHT10/AHT20 package to read temperature and humidity once through `i2c1`, then prints the result through the serial console. The device handle is cached, so repeated command execution does not repeatedly initialize the sensor.

Usage

```
demo_aht20
```

Example output:

```
AHT10: temp 26.3 C, humidity 48.7 %
```

LSM6DS3 6-Axis Sensor Demo

Function

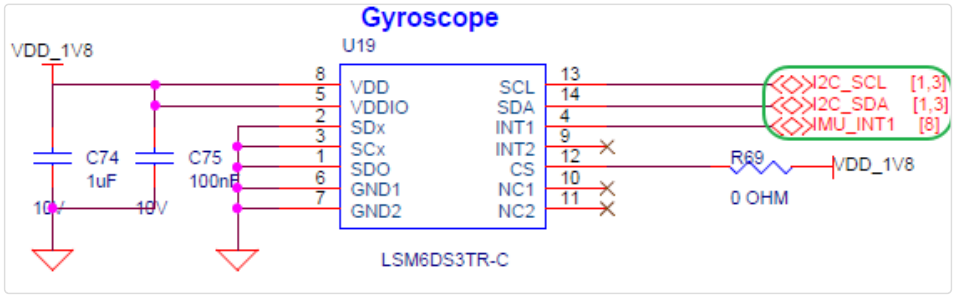
This demo is ported from the `Edgi_Talk_M33_LSM6DS3` example. It demonstrates how to use the **LSM6DS3TR-C 6-axis inertial measurement unit (IMU)**. The LSM6DS3TR-C integrates a 3-axis accelerometer, 3-axis gyroscope, and temperature sensor. It supports I2C/SPI interfaces and is commonly used for posture detection, motion recognition, gesture recognition, and wearable devices.

The command performs the following operations:

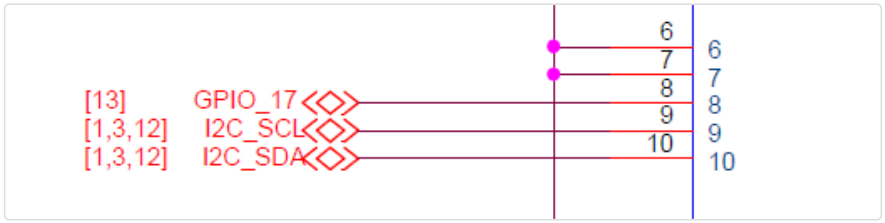
- Detects the LSM6DS3 device ID through `i2c0`.
- Resets the sensor and configures output data rate, accelerometer range, and gyroscope range.
- Polls 3-axis acceleration, 3-axis angular rate, and temperature.
- Prints data through the serial console. It does not print continuously after power-on.

Hardware

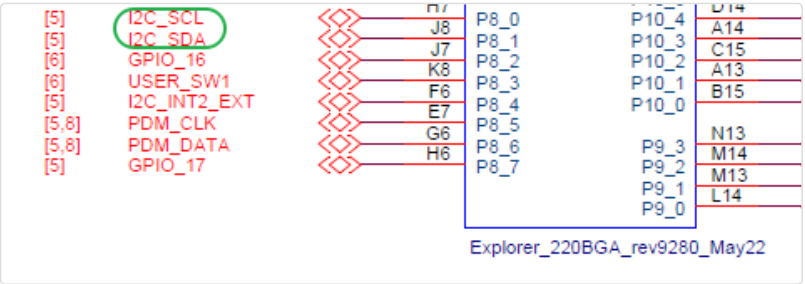
LSM6DS3TR-C Interface



BTB Connector



MCU Pins



Usage

```
demo_lsm6ds3
demo_lsm6ds3 10 200
```

Parameters:

- `count`: Number of samples. Default is `5`; set to `0` for continuous sampling.
- `delay_ms`: Sampling interval. Default is `500 ms`.

Example output:

```
LSM6DS3: found on i2c0 addr 0x6A
LSM6DS3 sample: count=5, delay=500 ms
Acceleration [mg]: 15.23  -3.12  1000.45
Angular rate [mdps]: 2.50  -1.25  0.75
Temperature [degC]: 26.54
```

Audio Loopback Demo

Function

This demo is ported from the `Edgi_Talk_M33_Audio` example. It demonstrates real-time audio loopback using **PDM microphone capture + ES8388 codec playback**. The audio device consists of the audio bus interface, control bus interface, codec, speaker, and microphone. The RT-Thread Audio framework handles device registration, open/close, read/write, volume control, and stream control.

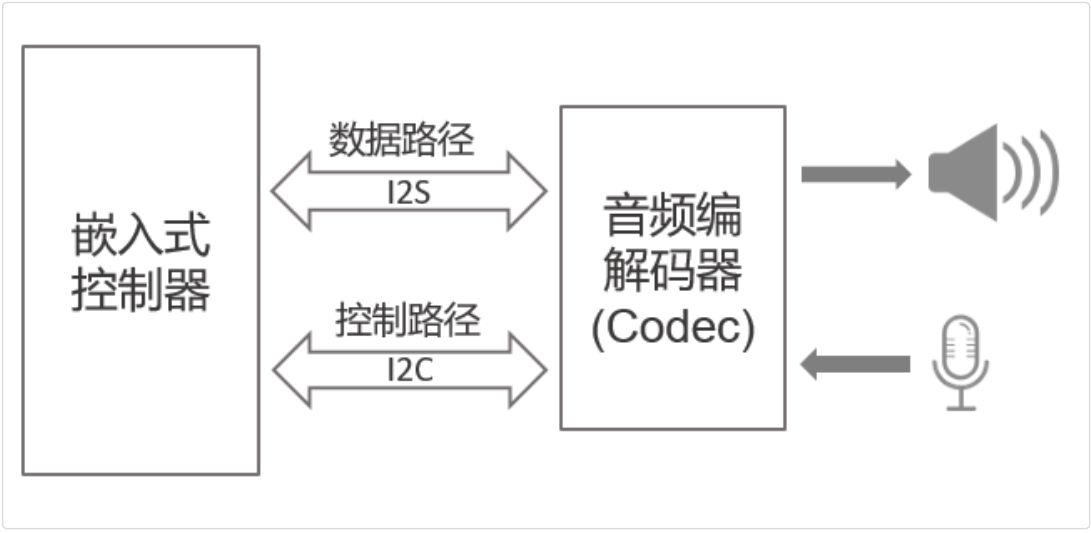
The command performs the following operations:

- Opens the `mic0` recording device and the `sound0` playback device.
- Explicitly configures audio as 16 kHz, mono, 16-bit, matching the PDM microphone output.
- Sets speaker volume and microphone gain.
- Creates a background thread that continuously reads microphone data and writes it to the speaker.
- Starts and stops manually through MSH commands. It does not enter loopback automatically after power-on.

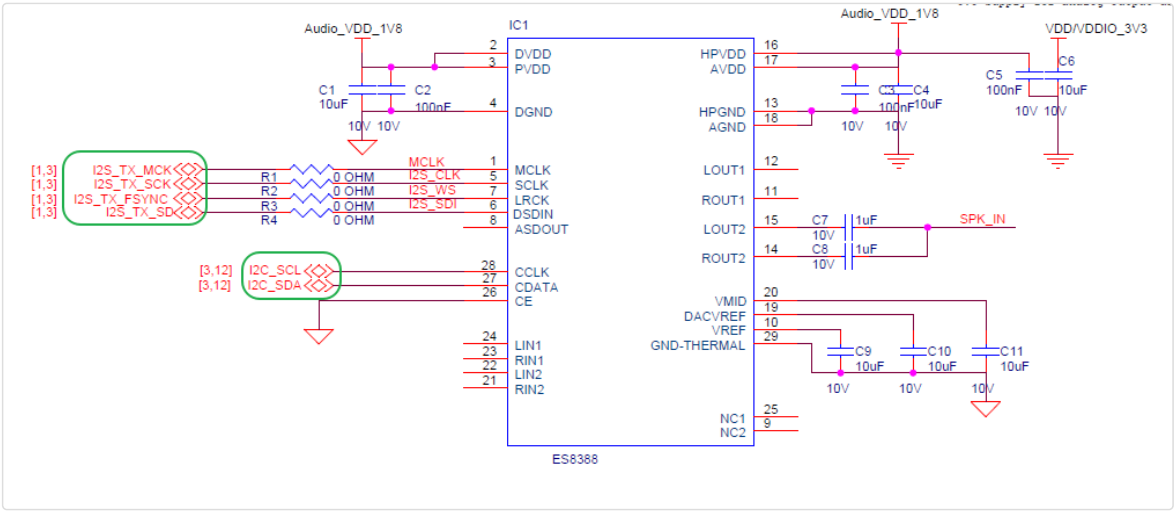
> ***Note:*** *The original Audio project used the P8.3 key to toggle playback state. In this multi-demo project, P8.3 is used by* `demo_key`*, so the Audio demo is controlled by MSH commands to avoid two demos registering the same key interrupt.*

Hardware

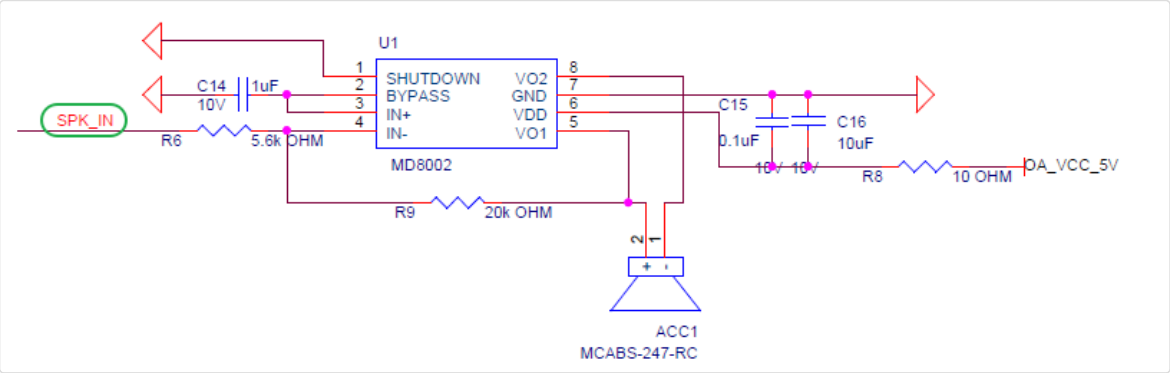
Embedded Audio System



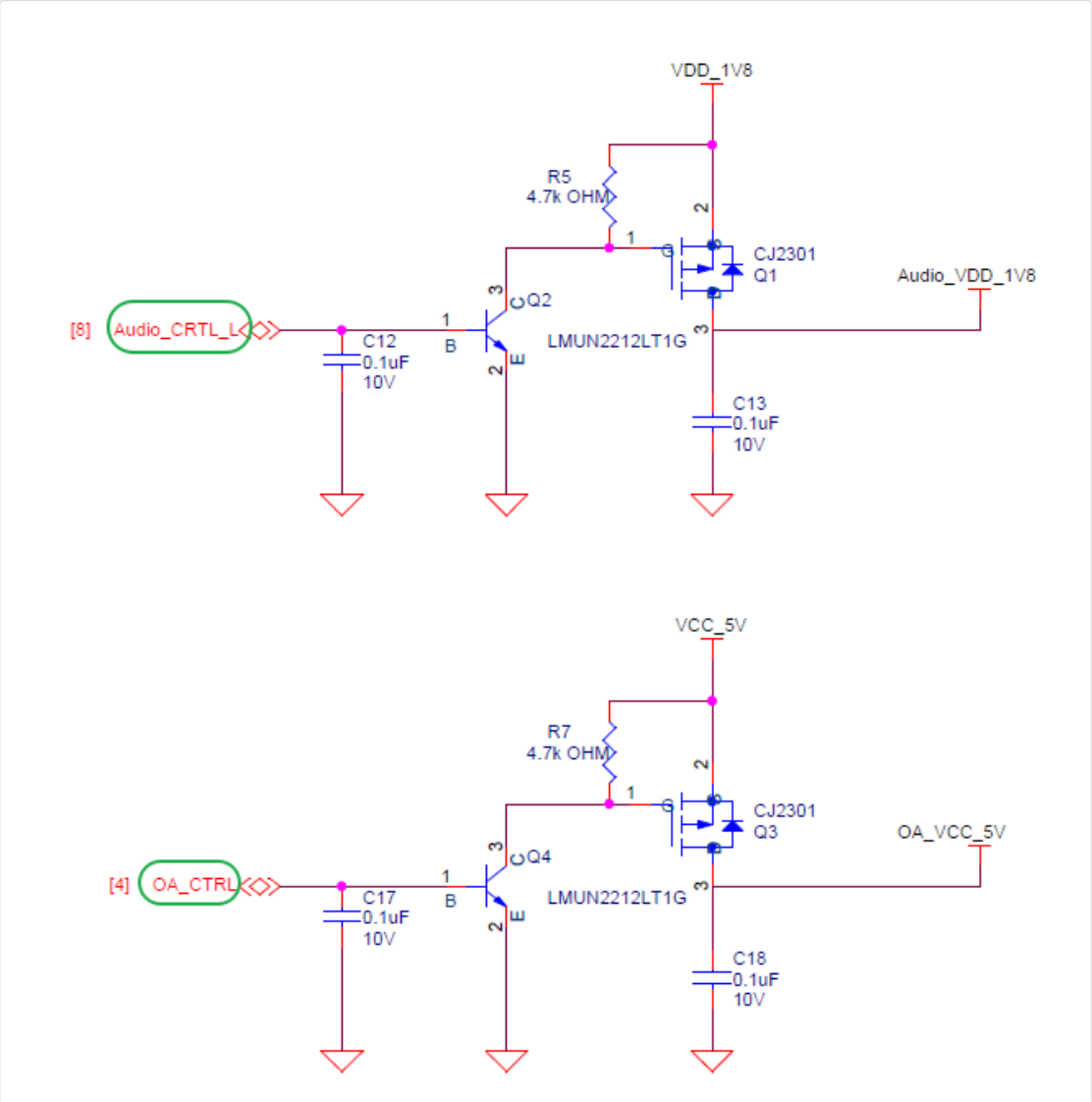
ES8388 Interface



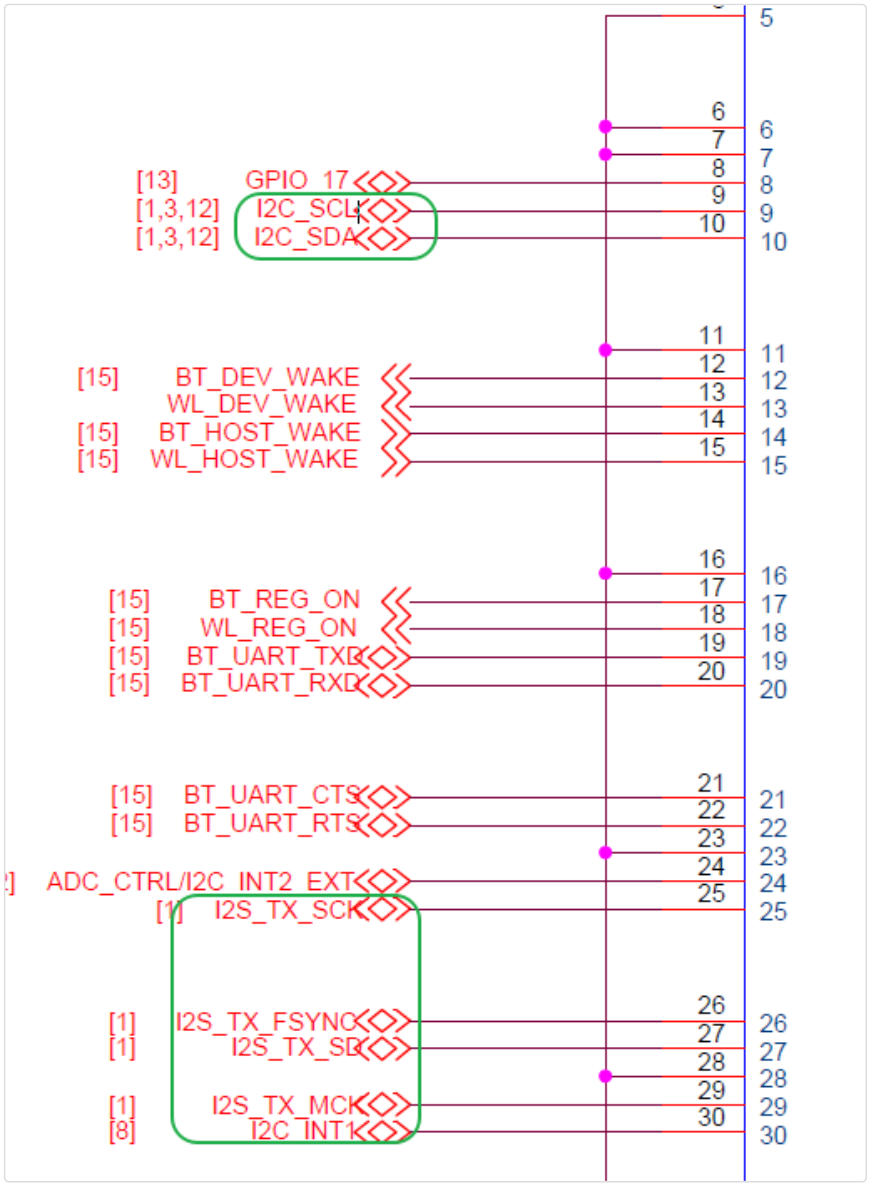
Speaker Interface



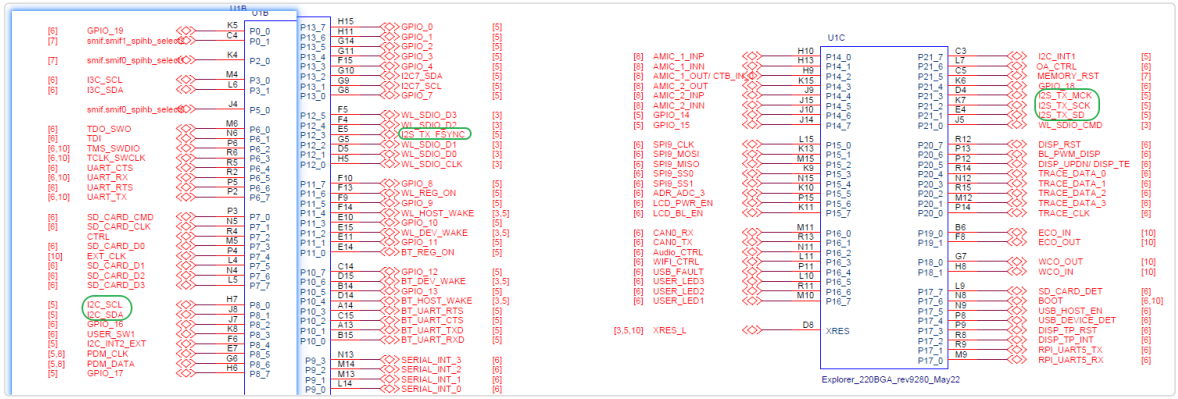
Control Pins



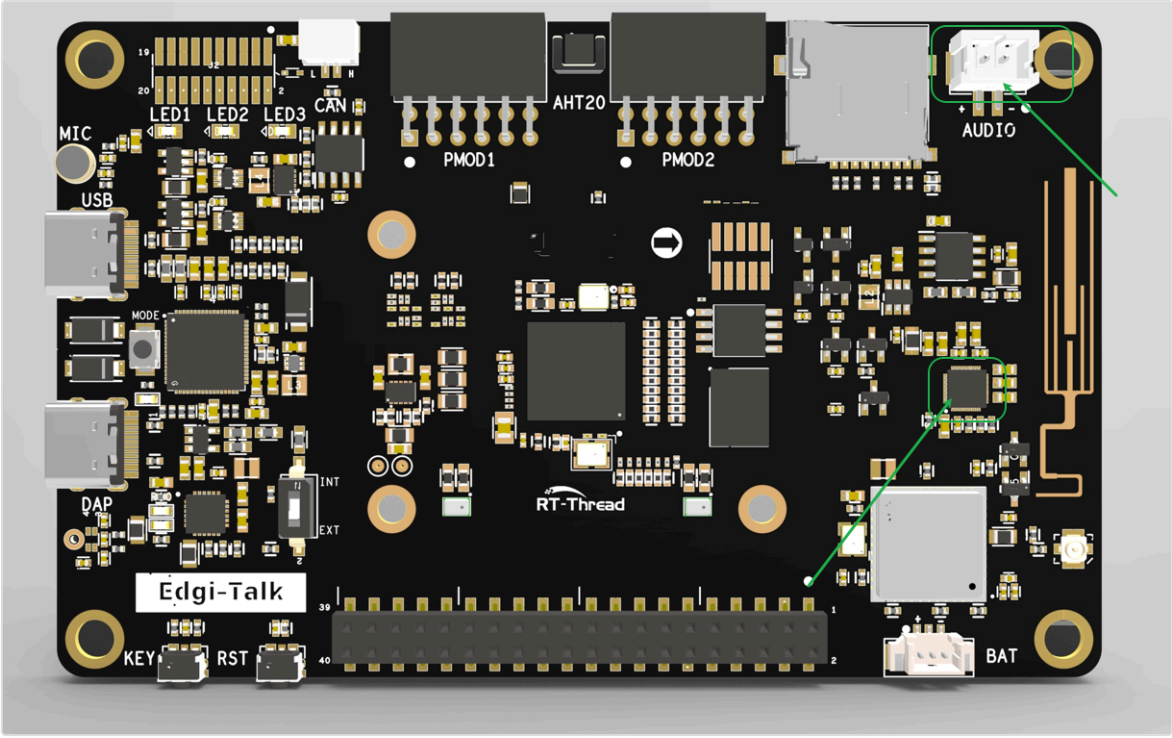
BTB Connector



MCU Pins



Board Location



Usage

```
demo_audio start
demo_audio start 60 30
demo_audio status
demo_audio stop
```

Parameters:

- `speaker_volume`: Speaker volume. Default is `60`.
- `mic_gain`: Microphone gain. Default is `30`.

Example output:

```
Audio loopback started: speaker=60, mic_gain=30
Audio loopback: running, speaker=60, mic_gain=30
Stopping audio loopback...
Audio loopback stopped
```

WavPlayer Audio Playback Demo

Function

This demo is ported from the `Edgi_Talk_M33_WavPlayer` example. It demonstrates WAV file playback using the **RT-Thread wavplayer package + Audio device driver + filesystem**. `wavplayer` reads the WAV header, configures `sound0` according to the sample rate, channel count, and 16-bit width in the file, then outputs audio through the ES8388 codec and speaker.

The current project enables:

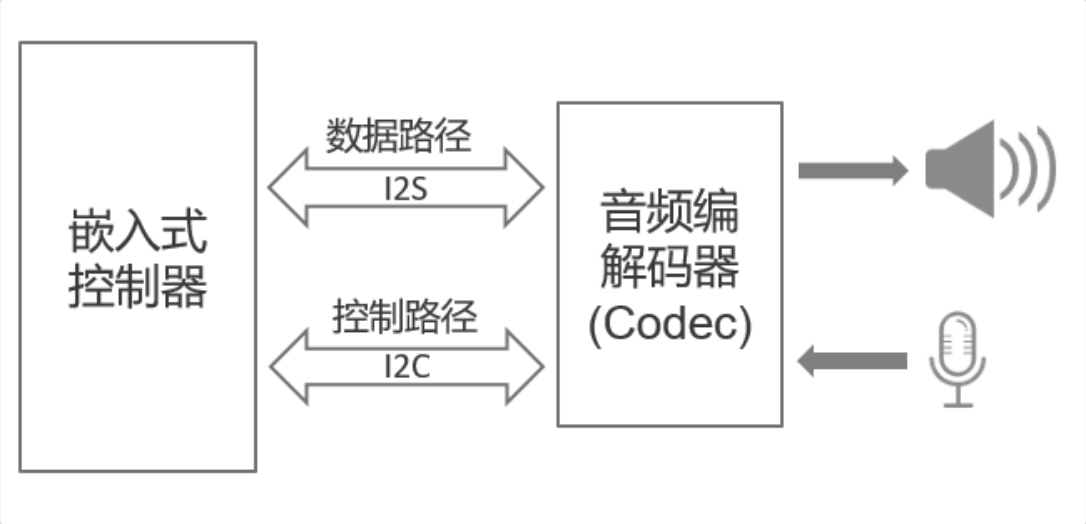
- The `wavplay` playback command.
- The `optparse` command-line parsing package.
- The `sound0` playback device.

The `wavrecord` recording command is not enabled in this project. Use `demo_audio` for recording/loopback validation to avoid resource-management conflicts in the multi-demo project.

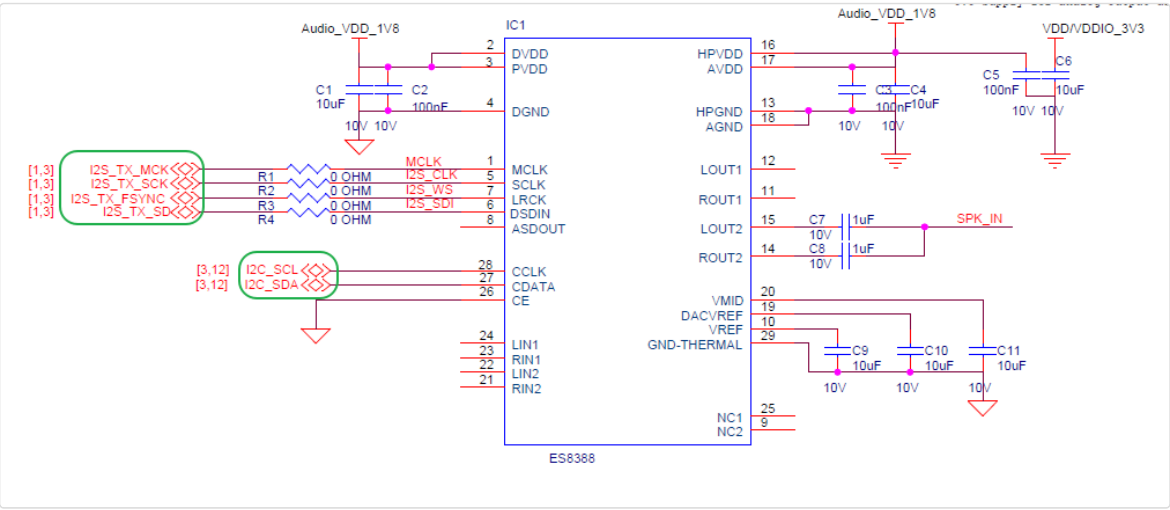
> ***Note:*** *`wavplay` and `demo_audio` both occupy `sound0`. Before playing a WAV file, run `demo_audio stop` and confirm that the loopback demo has stopped.*

Hardware

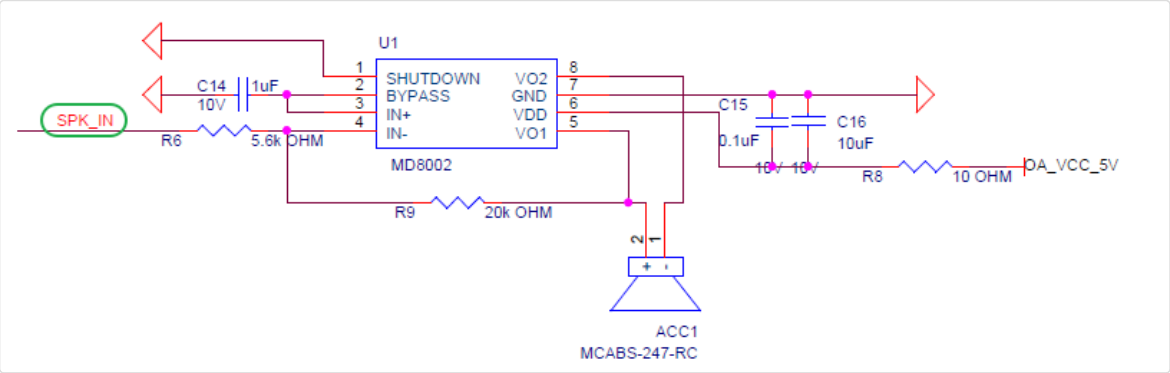
Audio System



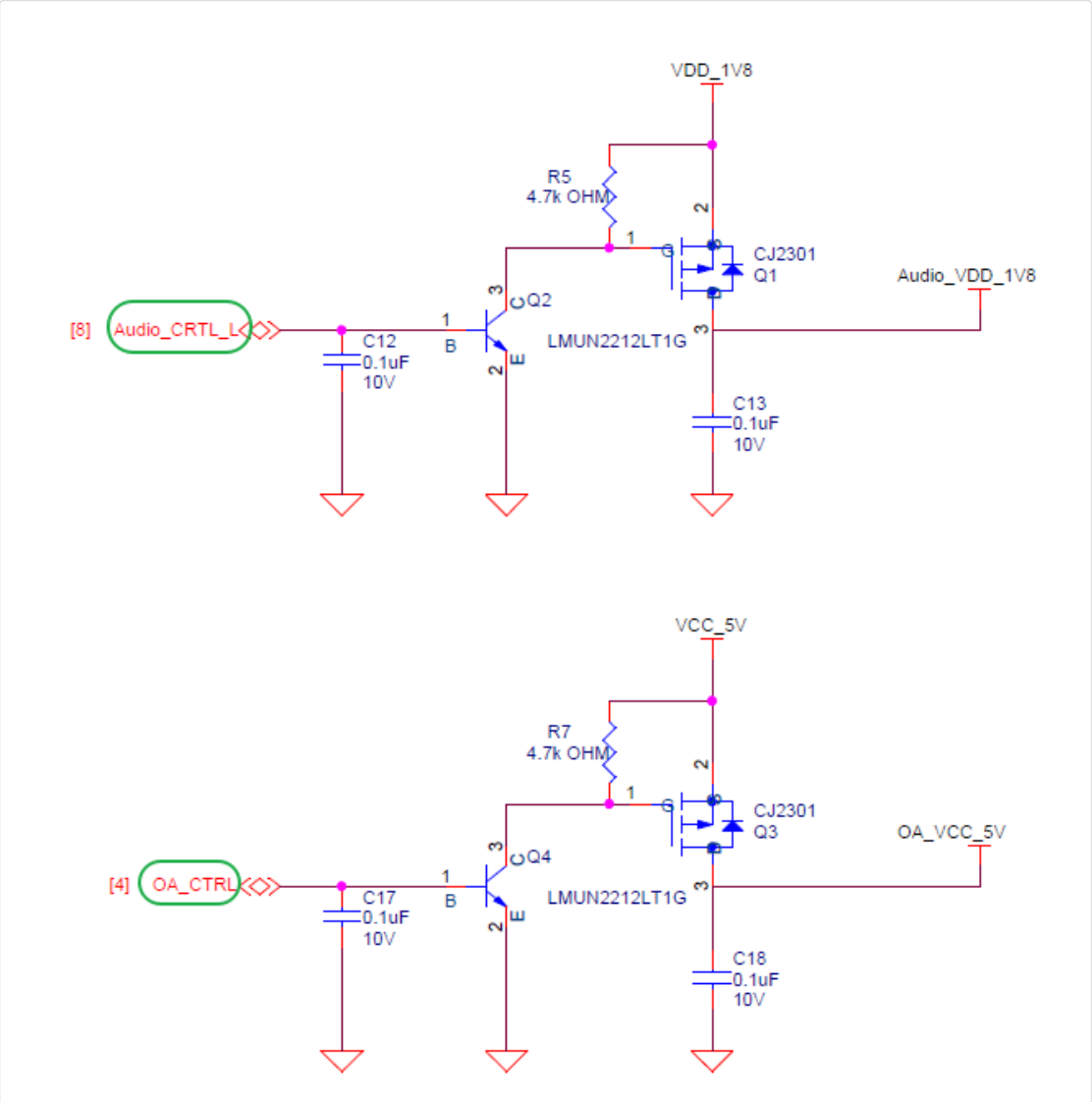
ES8388 Interface



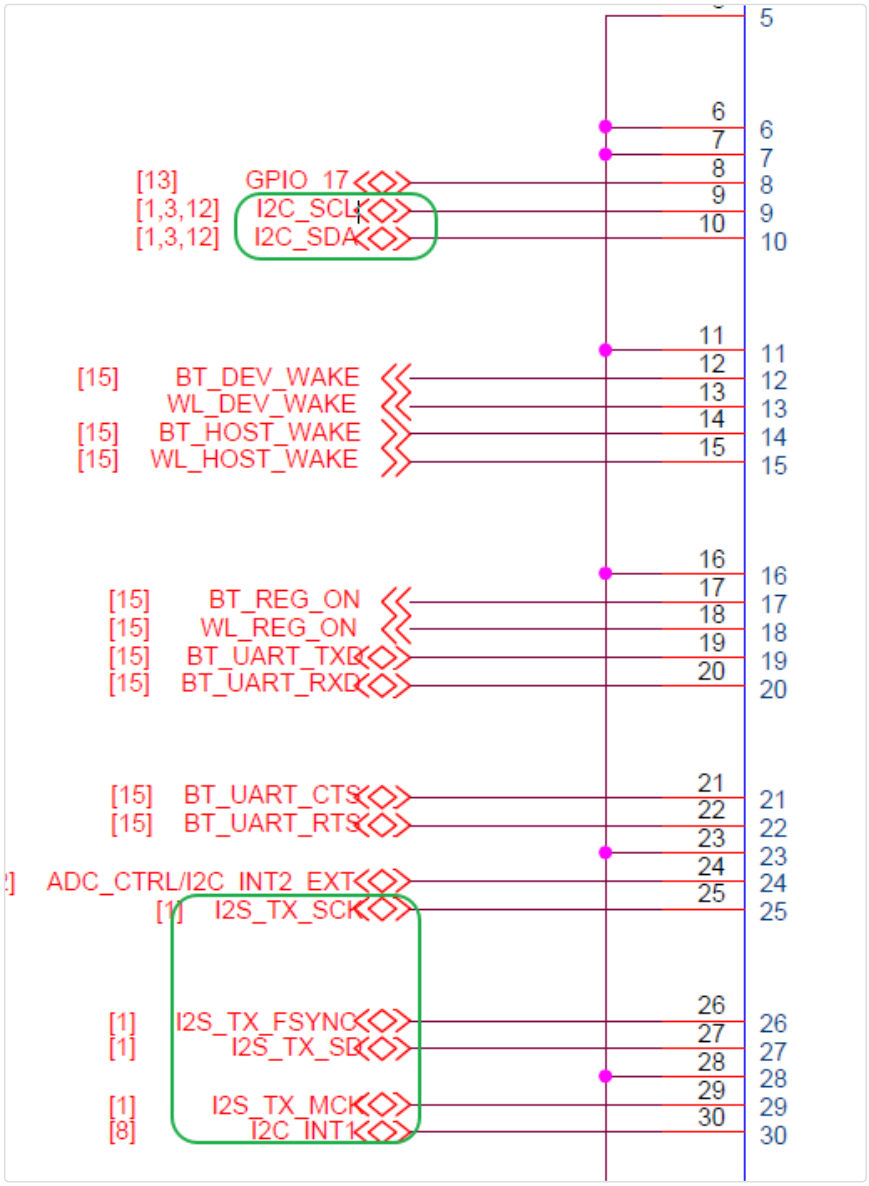
Speaker Interface



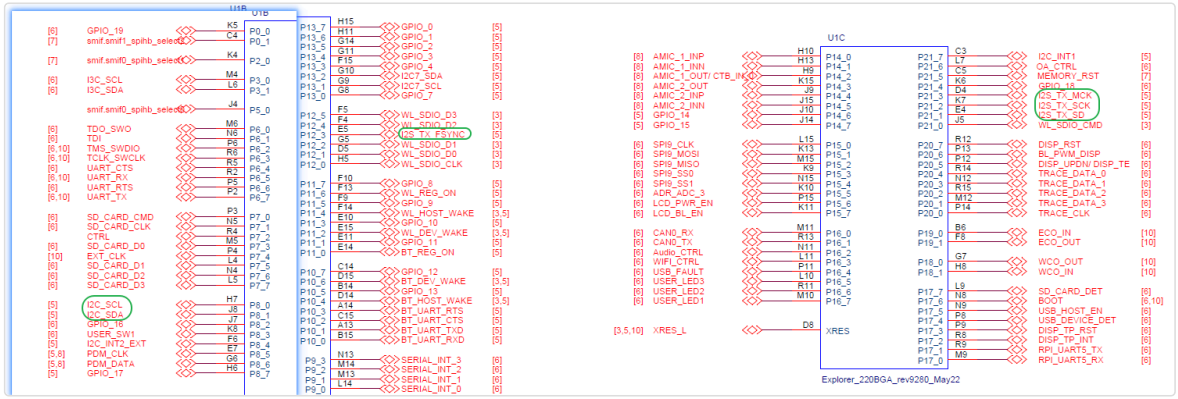
Control Pins



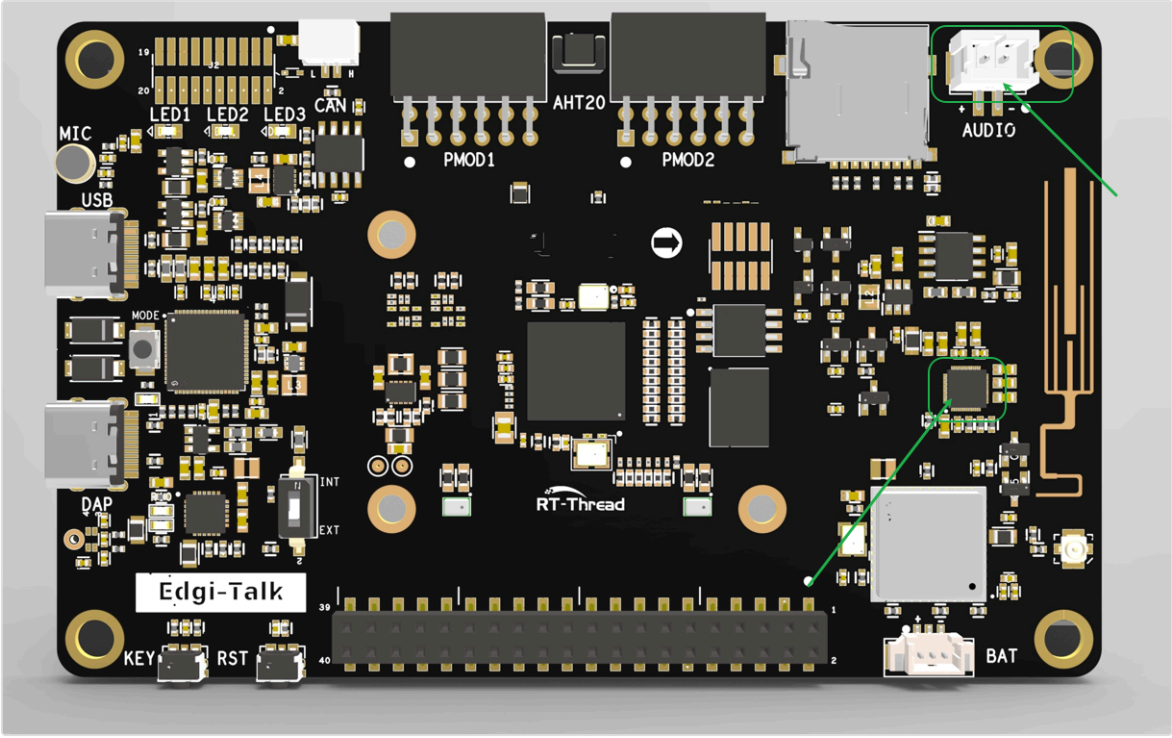
BTB Connector



MCU Pins



Board Location



Usage

Place the WAV file in a mounted filesystem. For example, after the SD card is mounted to `/sdcard`:

```
cd /sdcard
wavplay -s 16000.wav
```

You can also use the full path:

```
wavplay -s /sdcard/16000.wav
```

Common commands:

```
wavplay -h
wavplay -s /sdcard/16000.wav
wavplay -t
wavplay -p
wavplay -r
wavplay -v 80
```

Parameters:

- `-s <file>`: Start playing the specified WAV file.
- `-t`: Stop playback.
- `-p`: Pause playback.
- `-r`: Resume playback.
- `-v <0-100>`: Set playback volume.

Example output:

```
msh /sdcard>wavplay -s 16000.wav
[I/WAV_PLAYER] play start, uri=16000.wav
```

CoreMark Benchmark Demo

Function

This demo enables the RT-Thread `CoreMark` package for CPU performance benchmarking. The current configuration refers to the `Edgi_Talk_M55_CoreMark` project. The default iteration count is `36000`, and the floating-point version is not enabled.

Usage

```
core_mark
```

The command prints CoreMark parameters, elapsed time, verification result, and final score. Stop other high-load demos during testing to avoid SD card, audio, or sensor polling affecting the result.

ADC Demo

Function

This demo is ported from the `Edgi_Talk_M33_ADC` example. It demonstrates how to use the **ADC (Analog-to-Digital Converter)**. The ADC converts continuous analog voltage into a digital value for MCU processing.

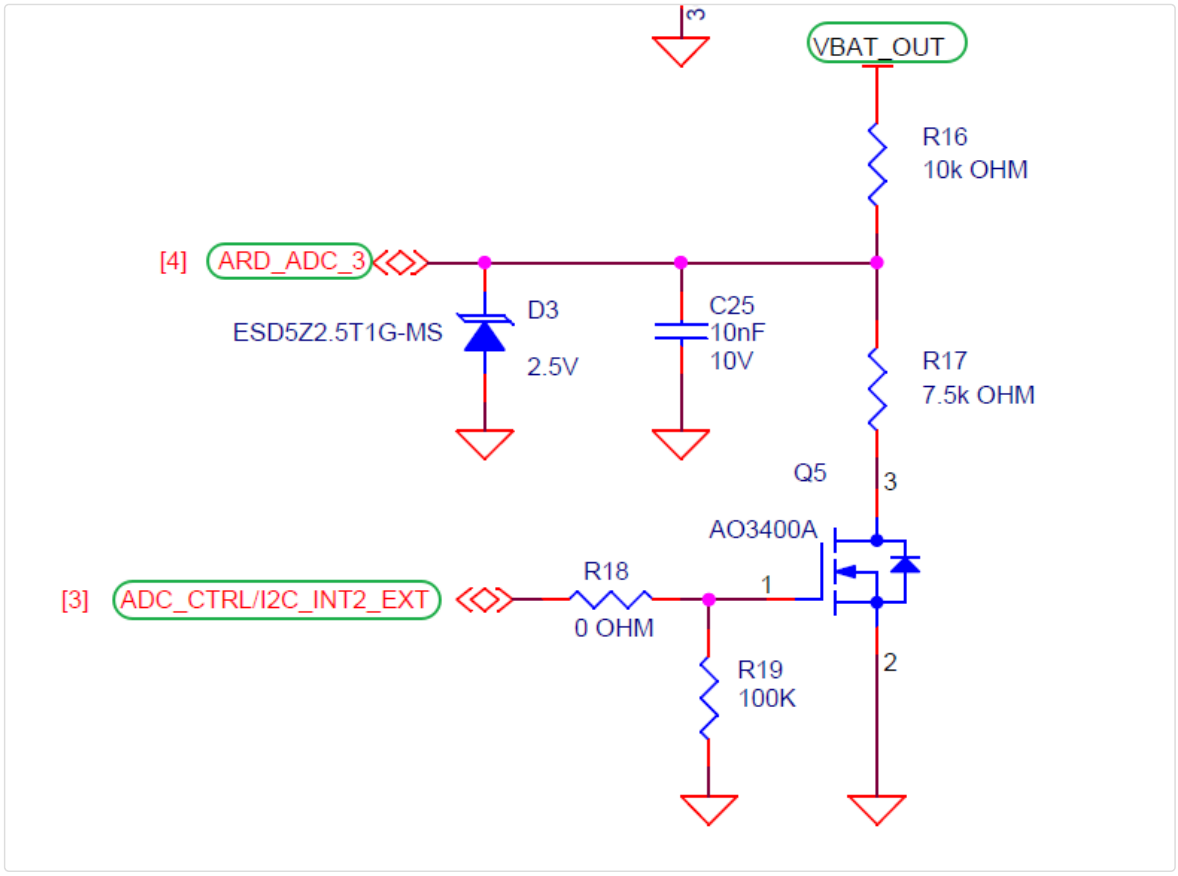
> ***Note:*** *In the current hardware connection, the ADC is only used to sample battery voltage. The Raspberry Pi connector and other external interfaces are not connected to this ADC channel, so they cannot be sampled by this command.*

The command performs the following operations:

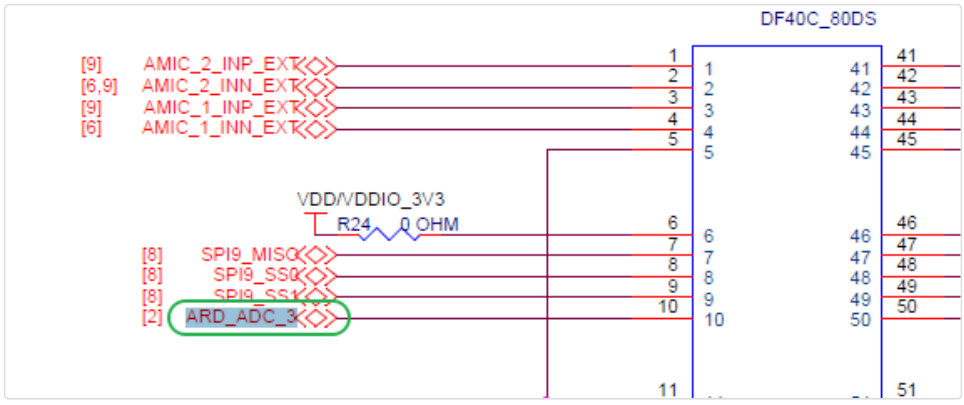
- Enables ADC1 Channel 1.
- Pulls the P8.4 ADC power-control pin high on first execution.
- Reads one ADC raw value and converts it to voltage.

Hardware

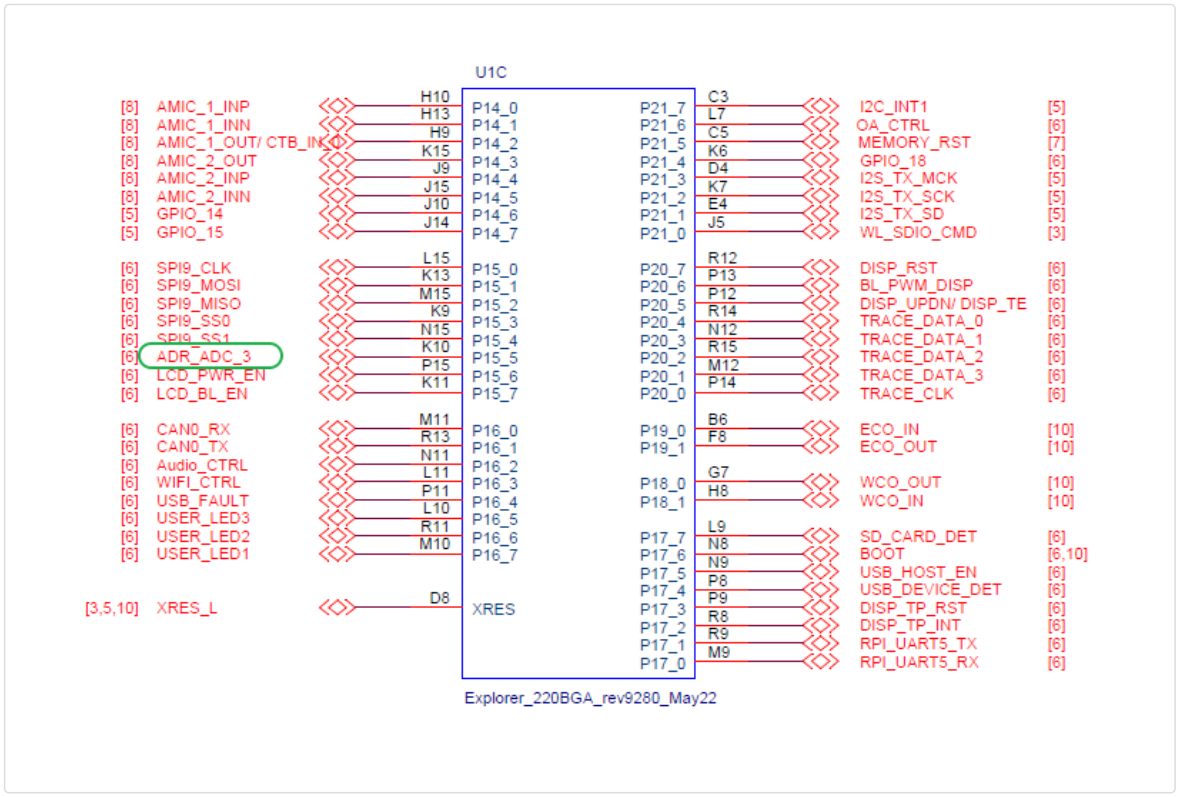
Interface



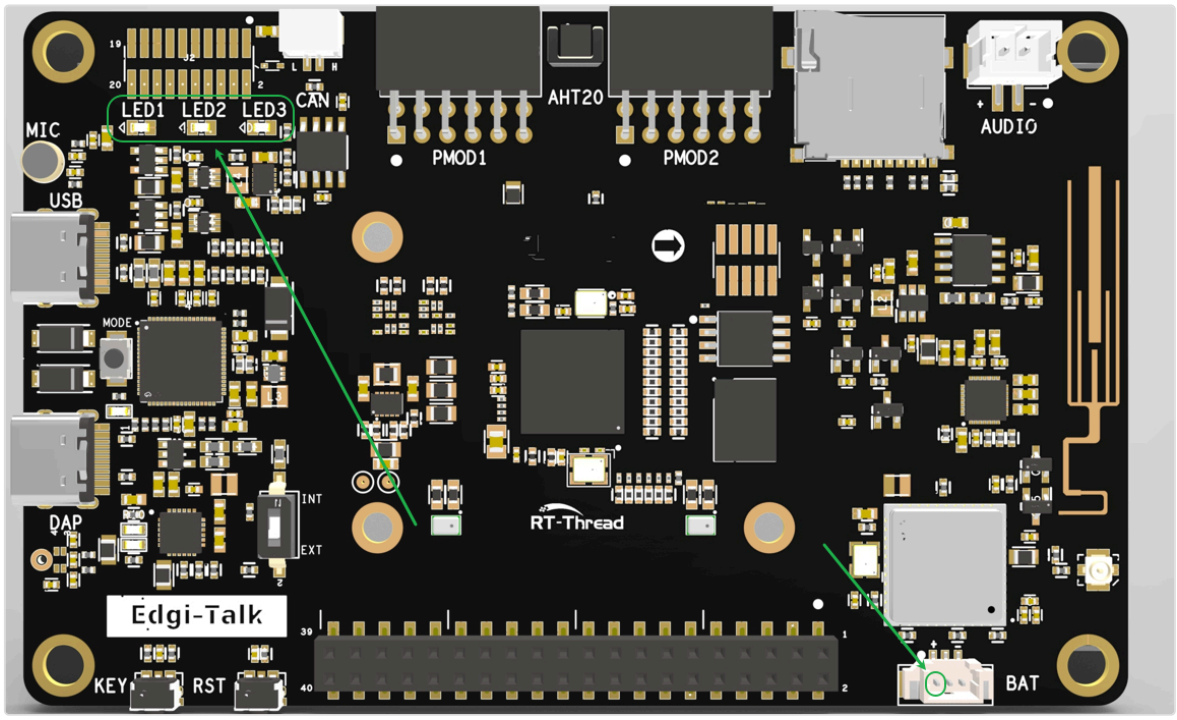
BTB Connector



MCU Pins



Board Location



Usage

```
demo_adc
```

Example output:

```
CH1: 3.123 V (raw=1340)
```

HyperRAM Demo

Function

This demo is ported from the `Edgi_Talk_M33_HyperRam` example. It validates **HyperRAM** mapping, basic read/write access, and bandwidth performance. After the HyperRAM driver is initialized, the memory region is mapped into the system address space and can be used by applications for large buffers or external heap.

Basic Read/Write Check

```
demo_hyperram
```

This command allocates a HyperRAM buffer, writes test data, reads it back, and verifies the result.

Bandwidth Test

```
demo_hyperram_speed
demo_hyperram_speed 1024 8
```

Parameters:

- `size_kb`: Data size for each test. The default value is set inside the demo.
- `loops`: Loop count. Larger values produce more stable statistics.

The test includes sequential write, sequential read, and memory-copy throughput.

Key Interrupt Demo

Function

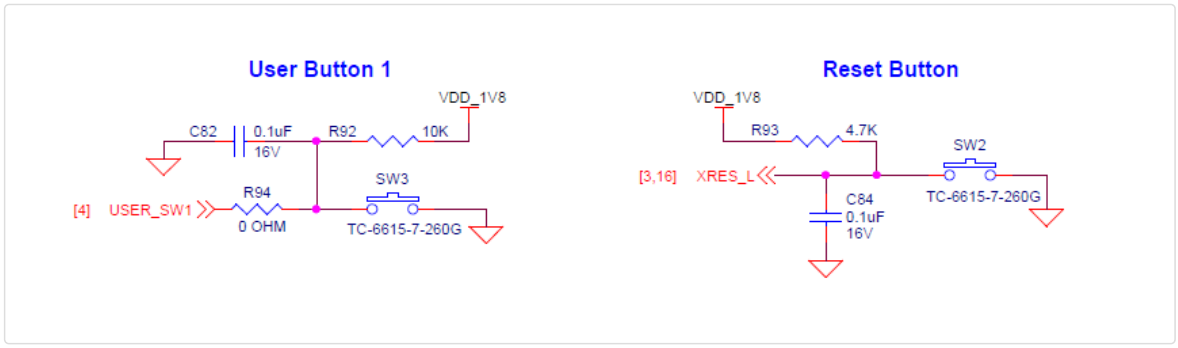
This demo is ported from the `Edgi_Talk_M33_Key_Irq` example. It demonstrates GPIO interrupt usage through the RT-Thread PIN driver.

The command performs the following operations:

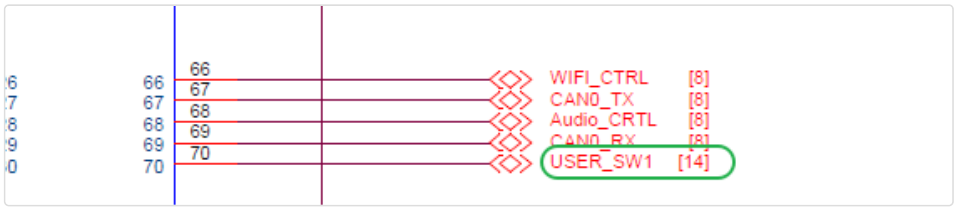
- Configures P8.3 as pull-up input.
- Registers a falling-edge interrupt callback.
- Toggles the P16.5 blue LED and prints the state after each key press.

Hardware

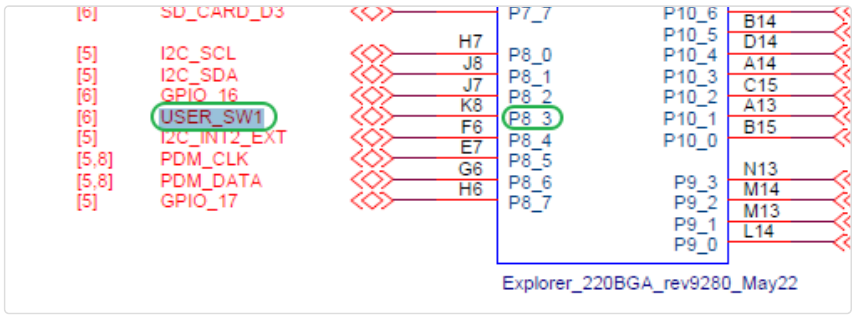
Button Interface



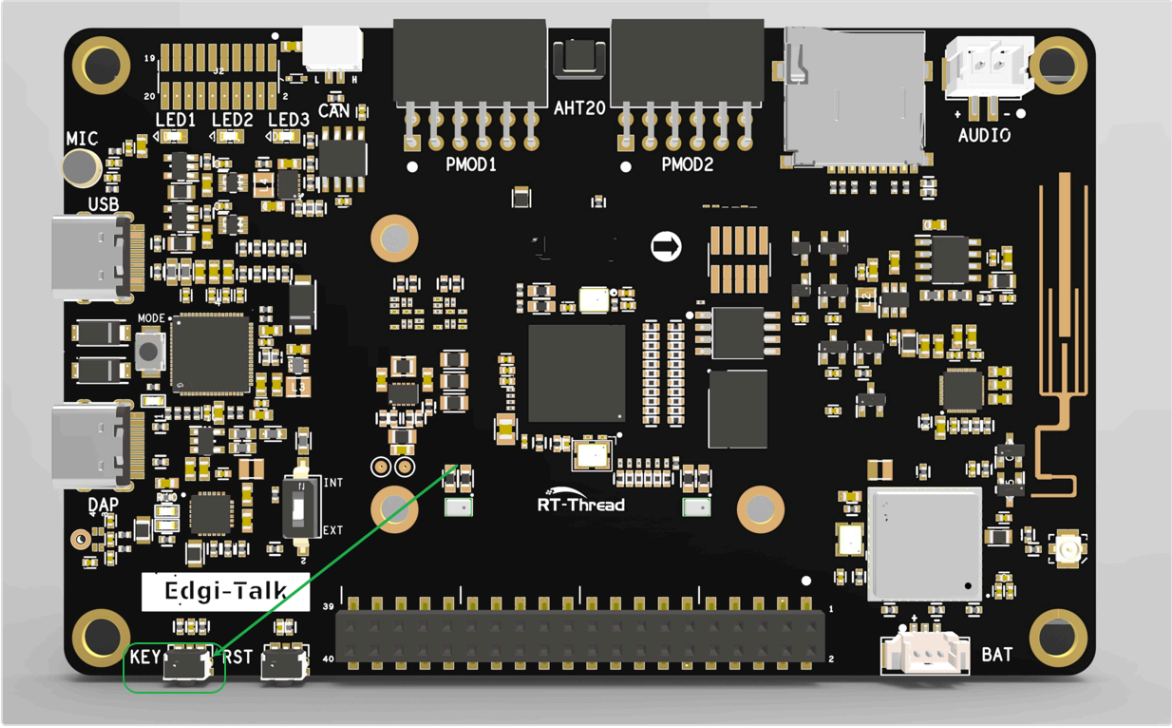
BTB Connector



MCU Interface



Board Location



Usage

```
demo_key
```

Example output:

```
Key IRQ ready. Press the button on P8.3.
button pressed (led ON)
button pressed (led OFF)
```

RTC Demo

Function

This demo is ported from the `Edgi_Talk_M33_RTC` example. It demonstrates how to read and set the **RTC (Real-Time Clock)**. RTC can be used for system time, log timestamps, scheduled tasks, and low-power wakeup scenarios.

Usage

Read the current time:

```
demo_rtc
```

Set the time and read it back:

```
demo_rtc 2026 7 17 10 30 0
```

Example output:

```
Fri Jul 17 10:30:00 2026
```

SD Card Demo

Function

This demo is ported from the `Edgi_Talk_M33_SDCARD` example. It demonstrates SD card mounting, file writing, file reading, and read/write speed testing. SD cards are non-volatile storage devices commonly used for logs, configuration files, and audio/video buffering.

The current project includes hot-plug handling:

- If an SD card is inserted during power-on, it is automatically mounted to `/sdcard`.
- If the SD card is removed during runtime, it is automatically unmounted.
- When no card is inserted, the system rescans at a low frequency to avoid repeated console output and thread stack overflow.

File Write/Read Test

```
demo_sdcard
```

This command writes text to `/sdcard/demo_test.txt`, reads it back, and prints it.

Sequential Read/Write Speed Test

```
demo_sdcard_speed
demo_sdcard_speed 8192 64
```

Parameters:

- `total_kb`: Total test file size. Default is `4096 KB`.
- `block_kb`: Block size for each read/write operation. Default is `64 KB`; maximum is `64 KB`.

Flash Filesystem Demo

Function

This project enables **FAL + MTD NOR + littlefs**. During startup, the external Flash `filesystem` partition is mounted automatically to `/flash`. If the first mount fails, the initialization logic tries to format the partition and mount it again.

Flash partition configuration:

Partition	FAL Offset	Size	Purpose
`whd_firmware`	`0x00000`	`384 KB`	Wi-Fi firmware
`whd_clm`	`0x60000`	`64 KB`	Wi-Fi CLM data
`whd_nvram`	`0x70000`	`64 KB`	Wi-Fi NVRAM
`bt_image`	`0x80000`	`512 KB`	Bluetooth image
`filesystem`	`0x100000`	`1024 KB`	`/flash` littlefs filesystem

> ***Note:*** *`demo_flash_speed` creates a temporary test file under `/flash` and deletes it after the test. This command triggers Flash erase/write operations. Avoid repeatedly running large tests on a `/flash` partition that contains important data.*

Sequential Read/Write Speed Test

```
demo_flash_speed
demo_flash_speed 512 4
```

Parameters:

- `total_kb`: Total test file size. Default is `512 KB`.
- `block_kb`: Block size for each read/write operation. Default is `4 KB`; valid range is `1 KB` to `64 KB`.

Example output:

```
Flash speed test
file=/flash/flash_speed.bin, total=512 KB, block=4 KB
write  524288 bytes, 1234 ms, 0.41 MB/s (414 KB/s)
read   524288 bytes, 78 ms, 6.41 MB/s (6564 KB/s)
```

Build and Flash

1. Open the project and complete the build.
2. Connect the board USB port to the PC using the **onboard debugger (DAP)**.
3. Flash the generated firmware to the board through the programming tool, or run the following command from the project root:

```
.\m33_program.ps1
```

To erase the whole chip before flashing, run:

```
.\m33_program.ps1 -EraseAll $true
```

4. Open the serial terminal, enter `msh />`, and run the required demo command.

Boot Sequence

The system boot sequence is as follows:

```
+------------------+
|   Secure M33     |
|   (Secure Core)  |
+------------------+
          |
          v
+------------------+
|       M33        |
| (Non-Secure Core)|
+------------------+
          |
          v
+-------------------+
|       M55         |
| (Application Core)|
+-------------------+
```

> ***Note:*** *The secure core firmware is integrated into the M33 project build and packaging flow. Prepare and flash firmware strictly according to the boot sequence above; otherwise, the system may fail to run correctly.*

To enable M55, configure the **M33 project** as follows:

```
RT-Thread Settings --> Hardware --> select SOC Multi Core Mode --> Enable CM55 (
```

2.2. Edgi-Talk_M55_Driver_All Example Project

Chinese | **English**

Introduction

This project is based on the **Edgi-Talk platform** and runs on **RT-Thread on the Cortex-M55 core**. It collects common M55 peripheral demos in one firmware, including AHT20, LSM6DS3, audio loopback, WAV playback, ADC, HyperRAM, GPIO key interrupt, RTC, SD card, Flash filesystem, and CoreMark.

The demos are exposed as MSH commands, so peripherals are tested manually instead of all running automatically after boot.

Command Overview

Command	Description
`demo_aht20`	Read AHT10/AHT20 temperature and humidity
`demo_lsm6ds3 [count] [delay_ms]`	Read LSM6DS3 accelerometer/gyroscope data
`demo_audio start/stop/status [speaker_volume] [mic_gain]`	ES8388 speaker and PDM microphone loopback
`wavplay -s <file>`	Play a WAV file
`demo_adc`	Read ADC1 channel 1 voltage
`demo_hyperram`	Run a basic HyperRAM read/write check
`demo_hyperram_speed [size_kb] [loops]`	Run a HyperRAM bandwidth test
`demo_key`	Register the P8.3 key interrupt and toggle the blue LED
`demo_rtc [YYYY MM DD HH MM SS]`	Read or set RTC time
`demo_sdcard`	Run an SD card file read/write test
`demo_sdcard_speed [total_kb] [block_kb]`	Run an SD card sequential speed test
`demo_fs_benchmark [file] [total_kb] [block_kb] [random_ops]`	Run a DFS/POSIX filesystem benchmark
`demo_flash_speed [total_kb] [block_kb]`	Run a `/flash` sequential speed test

Command	Description
`core_mark`	Run CoreMark

Software Description

- The project is developed based on the **Edgi-Talk** platform.
- This project is the Cortex-M55 RT-Thread application. The M33 firmware must enable and start the M55 core during board initialization.
- Each peripheral demo is triggered from the serial shell through an MSH command.
- Audio playback and loopback share `sound0`; stop `demo_audio` before running `wavplay`.
- SD card mounting and `/flash` filesystem mounting are handled by the common filesystem initialization logic.

Usage Instructions

Compilation and Download

1. Make sure the M33 boot project has enabled the CM55 core.
2. Open this project and complete the compilation.
3. Connect the board USB port to the PC using the **onboard debugger (DAP)**.
4. Use the programming tool or the RT-Thread Studio launch configuration to flash the generated firmware onto the development board.

Runtime Behavior

- After the M33 boot flow starts the M55 core, RT-Thread runs on M55 and enters the MSH shell.
- Run the required `demo_*` command from the serial terminal to test a peripheral.
- Only run one resource-heavy demo at a time, especially audio, SD card, Flash, and CoreMark tests.

```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Sep  5 2025 14:13:02
 2006 - 2022 Copyright by RT-Thread team
Hello RT-Thread
It's cortex-m55
msh >
```

Common Demos

AHT20 Temperature and Humidity Demo

```
demo_aht20
```

Reads AHT10/AHT20 temperature and humidity once, then prints the result through the serial console.

LSM6DS3 6-Axis Sensor Demo

```
demo_lsm6ds3
demo_lsm6ds3 10 200
```

`count` is the number of samples, and `delay_ms` is the interval between samples.

Audio Loopback Demo

```
demo_audio start
demo_audio start 60 30
demo_audio status
demo_audio stop
```

`speaker_volume` is the speaker volume, and `mic_gain` is the microphone gain. This demo occupies `mic0` and `sound0`.

WavPlayer Audio Playback Demo

```
wavplay -s /sdcard/16000.wav
wavplay -t
wavplay -p
wavplay -r
wavplay -v 80
```

`wavplay` and `demo_audio` both occupy `sound0`. Run `demo_audio stop` before playing a WAV file.

ADC Demo

```
demo_adc
```

Reads the voltage value of ADC1 Channel 1.

HyperRAM Demo

```
demo_hyperram
demo_hyperram_speed
demo_hyperram_speed 1024 8
```

Validates basic HyperRAM read/write access and sequential bandwidth.

Key Interrupt Demo

```
demo_key
```

Registers a falling-edge interrupt on the P8.3 key and toggles the P16.5 blue LED whenever the key is pressed.

RTC Demo

```
demo_rtc
demo_rtc 2026 7 17 10 30 0
```

Reads the current RTC time, or sets the time and reads it back.

SD Card Demo

```
demo_sdcard
demo_sdcard_speed
demo_sdcard_speed 8192 64
demo_fs_benchmark
demo_fs_benchmark /sdcard/demo_fs_benchmark.bin 5120 64 64
```

The SD card is mounted automatically to `/sdcard` after insertion.

Flash Filesystem Demo

```
demo_flash_speed
demo_flash_speed 512 4
```

This command creates a temporary speed-test file under `/flash` and deletes it after the test.

CoreMark Benchmark Demo

```
core_mark
```

Stop other high-load demos before running CoreMark to avoid SD card, audio, or sensor polling affecting the result.

Notes

> *Note: This project requires **RT-Thread Studio 2.2.9** or higher.*

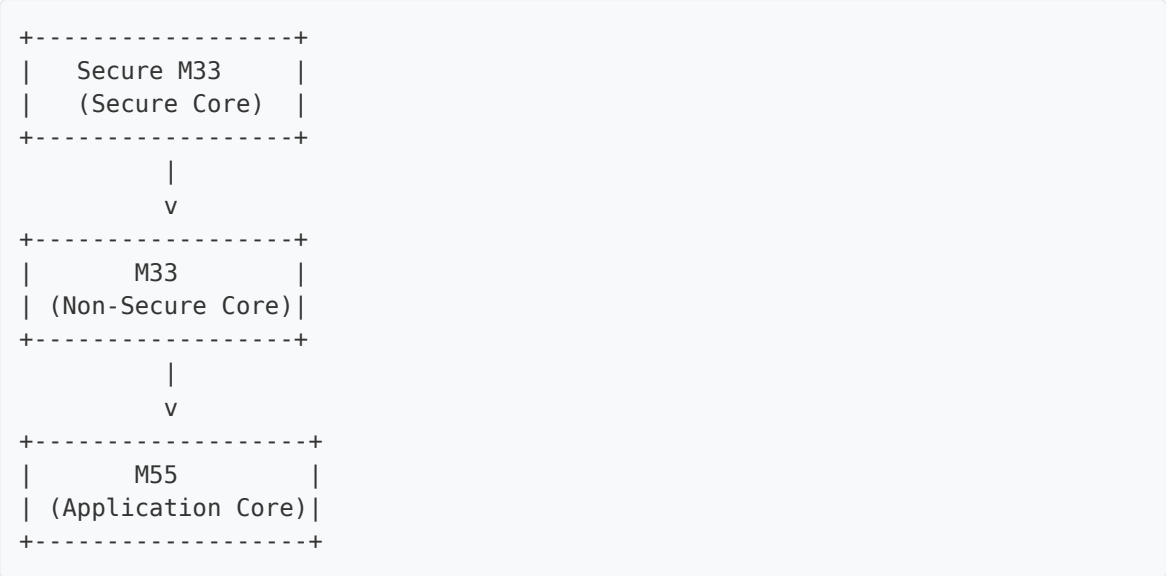
- To modify the graphical configuration of the project, open the configuration file using the following tool:

```
tools/device-configurator/device-configurator.exe
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

- After editing, save the configuration and regenerate the code.
- Format the SD card as FAT16 or FAT32.
- `demo_flash_speed` creates and removes a temporary file under `/flash`; avoid running large repeated tests on partitions that contain important data.
- If this project does not run correctly, compile and flash the **Edgi_Talk_M33_Template** project first to ensure proper initialization and core startup sequence.

Boot Sequence

The system boot sequence is as follows:



The M55 application depends on the M33 boot flow. To enable the M55 core, configure the **M33 project** as follows:

```
RT-Thread Settings --> Hardware --> select SOC Multi Core Mode --> Enable CM55 (
```

2.3. Edgi_Talk_M55_WIFI Example Project

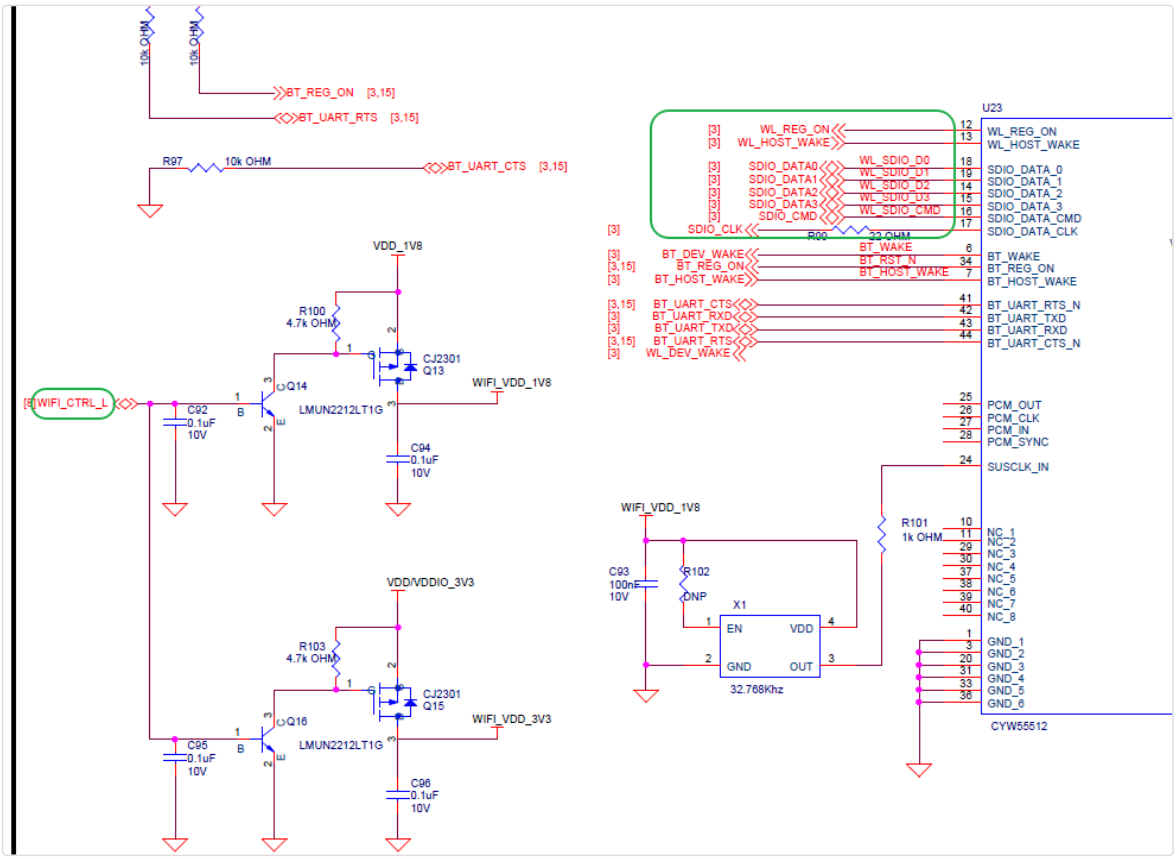
中文 | English

Introduction

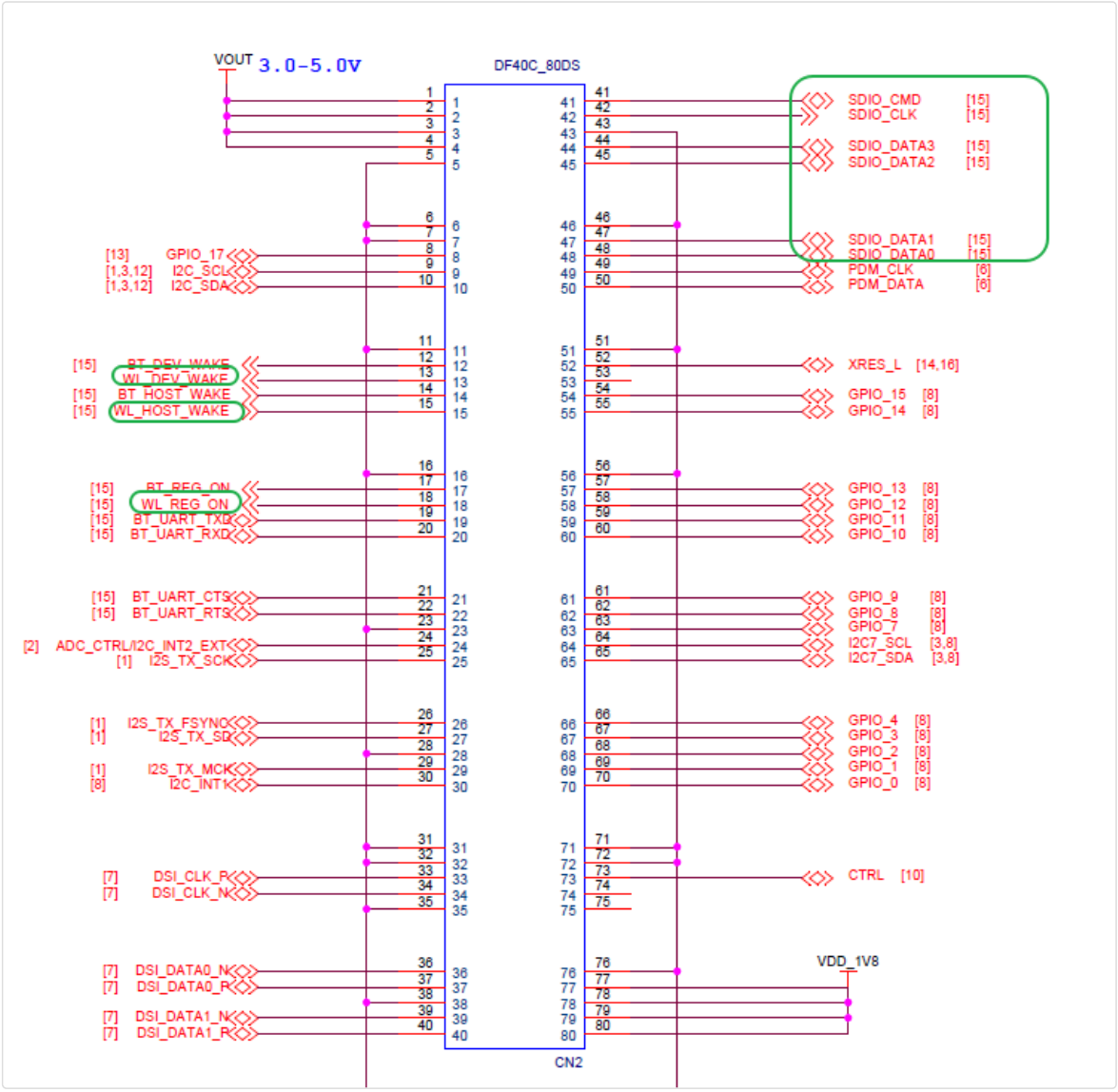
This example demonstrates **Wi-Fi functionality** on the **M55 core** using **RT-Thread RTOS**.
It allows users to quickly test Wi-Fi scanning, connection, and performance, verifying the Wi-Fi module interface.

Hardware Overview

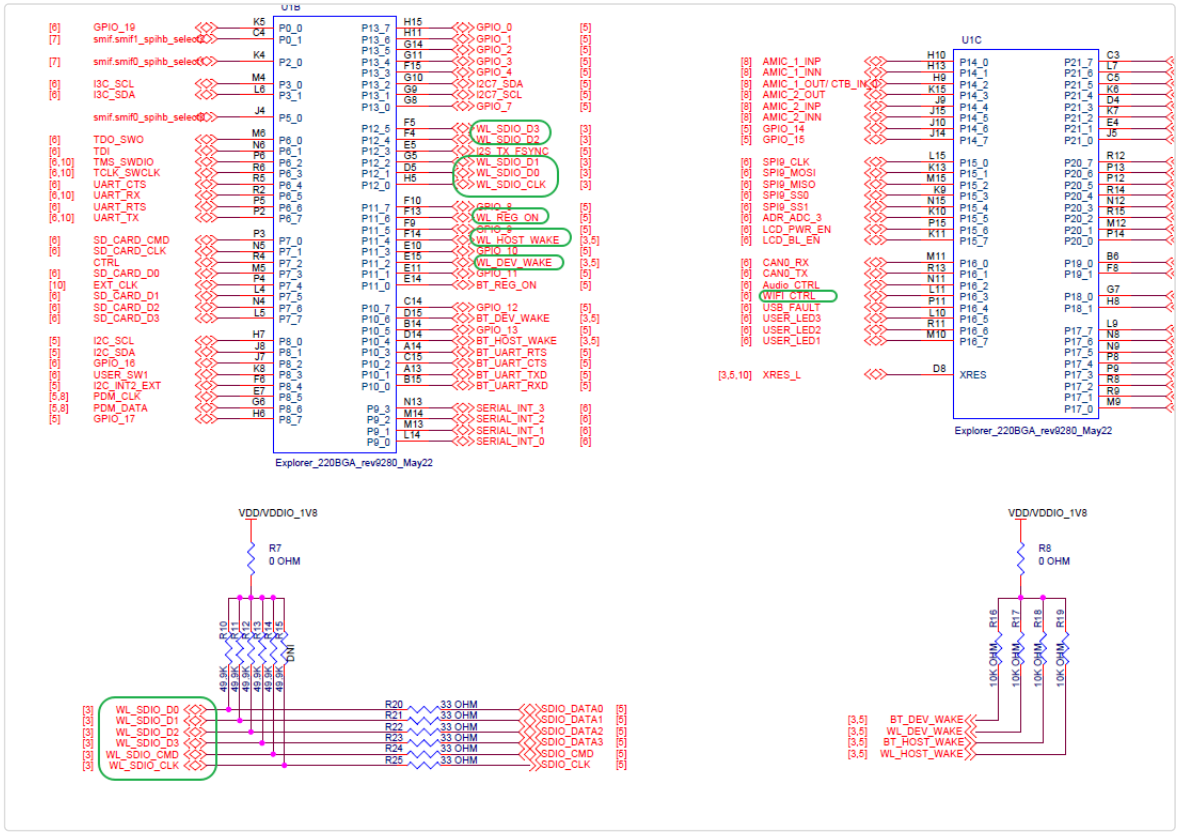
Wi-Fi Interface



BTB Socket


VOUT 3.0-5.0V
DF40C_80DS
CN2
SDIO_CMD [15]
SDIO_CLK [15]
SDIO_DATA3 [15]
SDIO_DATA2 [15]
SDIO_DATA1 [15]
SDIO_DATA0 [15]
PDM_CLK [6]
PDM_DATA [6]
[13] GPIO_17
[1,3,12] I2C_SCL
[1,3,12] I2C_SDA
[15] BT_DEV_WAKE
WL_DEV_WAKE
[15] BT_HOST_WAKE
[15] WL_HOST_WAKE
XRES_L [14,16]
GPIO_15 [8]
GPIO_14 [8]
[15] BT_REG_ON
[15] WL_REG_ON
[15] BT_UART_TXD
[15] BT_UART_RXD
GPIO_13 [8]
GPIO_12 [8]
GPIO_11 [8]
GPIO_10 [8]
[15] BT_UART_CTS
[15] BT_UART_RTS
[2] ADC_CTRL/I2C_INT2_EXT
[1] I2S_TX_SCK
GPIO_9 [8]
GPIO_8 [8]
GPIO_7 [8]
I2C7_SCL [3,8]
I2C7_SDA [3,8]
[1] I2S_TX_FSYNC
[1] I2S_TX_SD
[1] I2S_TX_MCK
[8] I2C_INT
GPIO_4 [8]
GPIO_3 [8]
GPIO_2 [8]
GPIO_1 [8]
GPIO_0 [8]
[7] DSI_CLK_P
[7] DSI_CLK_N
CTRL [10]
[7] DSI_DATA0_N
[7] DSI_DATA0_P
[7] DSI_DATA1_N
[7] DSI_DATA1_P
VDD_1V8

MCU Interface



Software Description

- Developed on **Edgi-Talk** platform.
- Example features:
- Wi-Fi scanning
- Wi-Fi connection
- Iperf performance test
- Provides a clear example of **Wi-Fi driver integration with RT-Thread**.

Usage

Build and Download

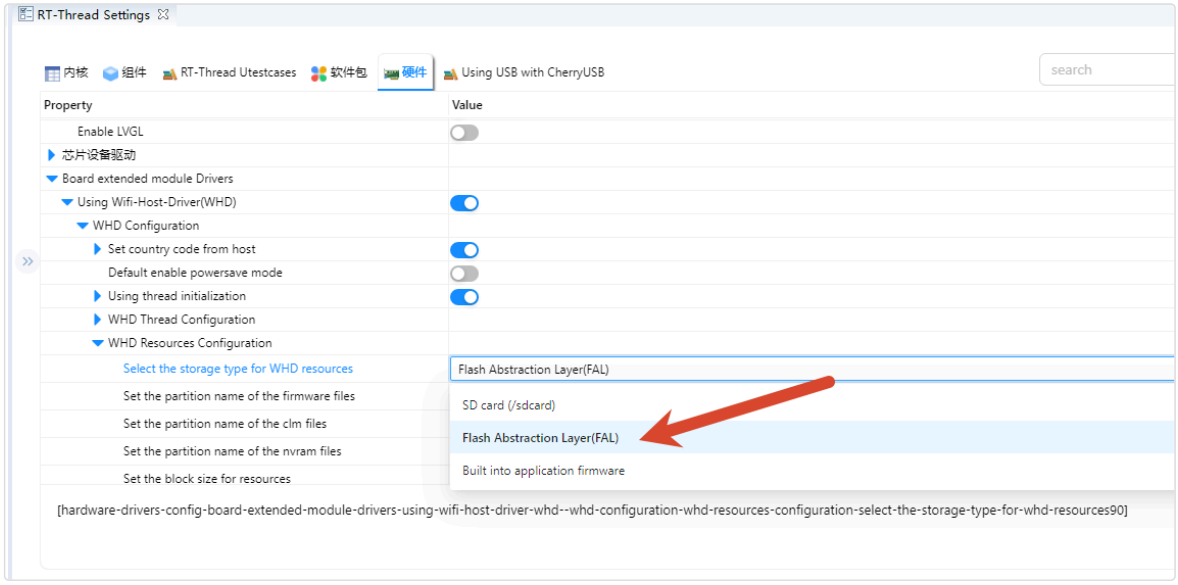
1. Open and compile the project.
2. Connect the board USB to PC via **DAP**.
3. Flash the compiled firmware from `Debug/rtthread.hex`.

Prepare Wi-Fi Resources

WHD needs three resource files before the radio can start: firmware, CLM regulatory data, and board NVRAM. This document uses the FAL partition flow as the current method. The Wi-Fi resources are stored independently in the `whd_firmware`, `whd_clm`, and `whd_nvram` partitions. In this mode, flashing `Debug/rtthread.hex` only writes the application image; the Wi-Fi resources must be written separately from the serial terminal. The default Edgi-Talk resource files are provided in the project root `resources/` directory.

Current Method: FAL Partitions

Select this mode in `settings`:

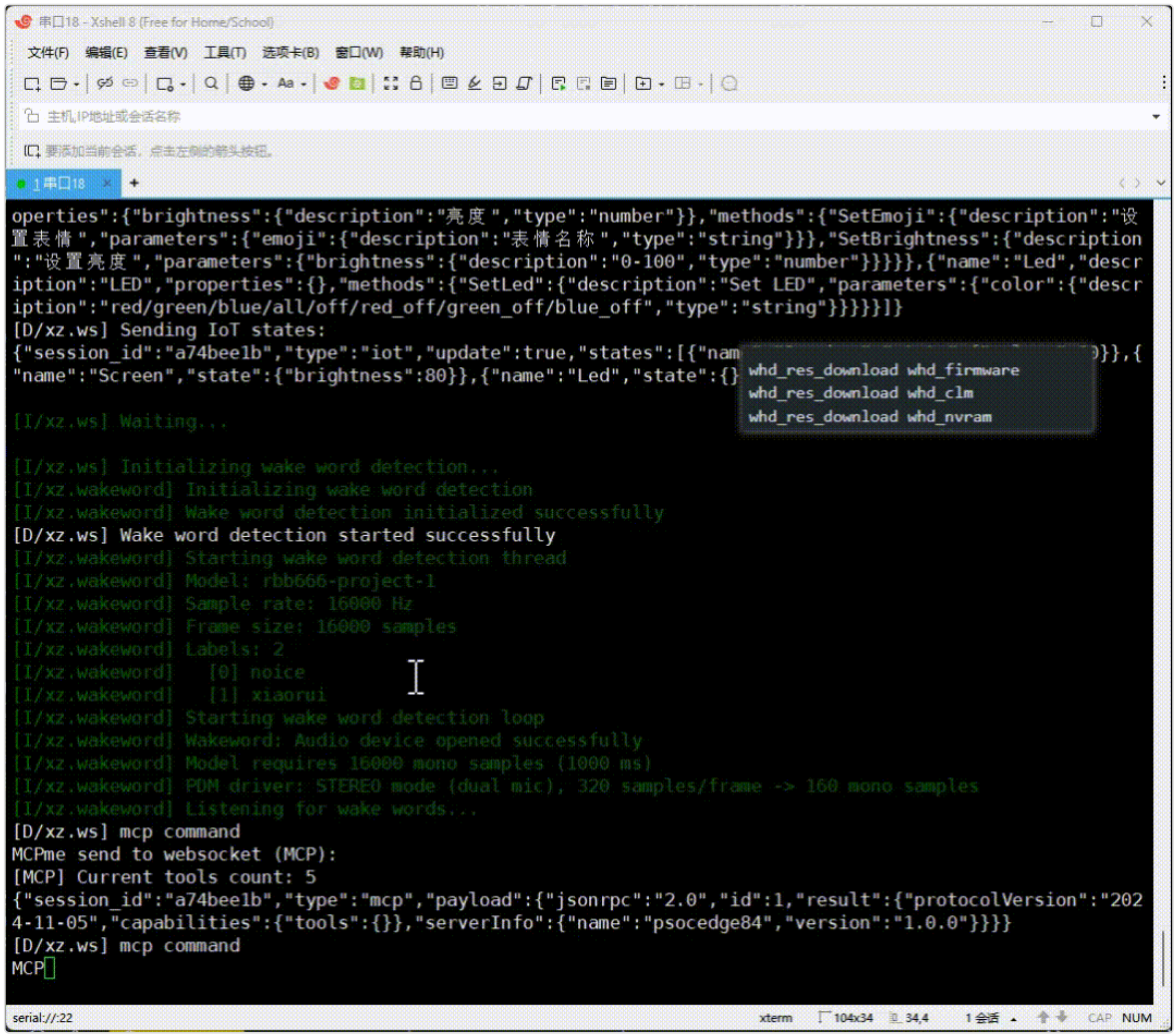


The default Edgi-Talk resource files are provided in the project root `resources/` directory.

```
whd_res_download whd_firmware
whd_res_download whd_clm
whd_res_download whd_nvram
```

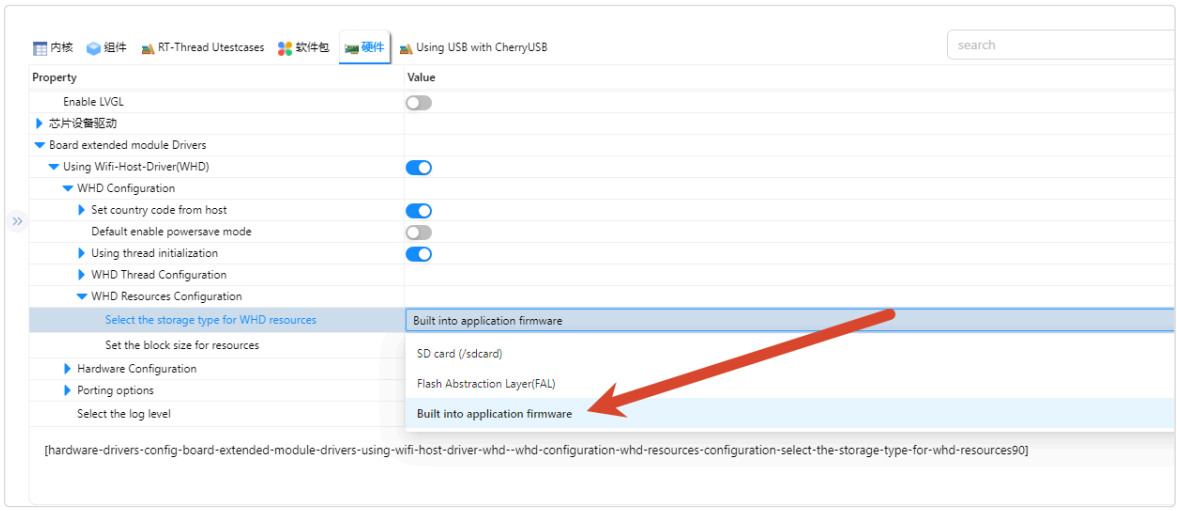
Each command switches to YMODEM mode. Use a terminal that supports YMODEM upload, such as Xshell, to send the matching file from `resources/`. Reset the board after all three transfers complete.

- Wait for the `Download ... success` message before starting the next transfer.
- After all three resources are written, reset the board so Wi-Fi can load the new resources. If the resource package is updated later, run `whd_res_download` again.



Extension: Build Wi-Fi Resources into the Application Image

If you want the Wi-Fi firmware to be programmed together with `Debug/rtthread.hex`, switch to `WHD_RESOURCES_IN_MEMORY`:



This mode builds the resource files from the project root `resources/` directory into the application image:

- `resources/55500A1.trxcse`
- `resources/55500A1.clm_blob`
- `resources/cyw55513modpse84som_rev3.txt`

After rebuilding, the Wi-Fi firmware, CLM, and NVRAM become part of the application image. After flashing `Debug/rtthread.hex`, you no longer need to run `whd_res_download whd_firmware`, `whd_res_download whd_clm`, or `whd_res_download whd_nvram`. If the Wi-Fi resource files are updated, rebuild and reflash the application image. This mode increases the application image size; the current CYW55500 firmware and CLM add about 240 KB of flash usage, plus the NVRAM text.

Note: RT-Thread Studio generates and runs makefiles from the `Debug` directory, while ENV/SCons builds from the project root. The current project handles this difference: Studio builds read resources from `../resources/`, and ENV/SCons generates resource code from the project root `resources/` directory. Place the resource files in the project root `resources/`; do not put them only in `Debug/resources/`.

Extension: SD Card Resource Loading

`WHD_RESOURCES_IN_SDCARD` loads the same three resources from `/sdcard` at runtime. The default filenames are `/sdcard/55500A1.trxcse`, `/sdcard/55500A1.clm_blob`, and `/sdcard/cyw55513modpse84som_rev3.txt`. This mode also enables the BSP SD card filesystem support used by the shared mount code.

Running Result

- After power-on, the system initializes the Wi-Fi device.
- Connect to a Wi-Fi network via serial terminal:

```
wifi scan
```

```
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w0
[I/WLAN.dev] wlan init success
[I/WLAN.lwip] eth device init ok name:w1
wifi scan
            SSID                     MAC            security    rssi chn Mbps
-------------------------------- -----------------  -------------- ---- --- ----
realthread_5G                    3c:6a:48:77:74:15  WPA2_AES_PSK   -87   36  300
realthread_5G                    3c:6a:48:c9:c2:79  WPA2_AES_PSK   -81   36  300
realthread_VIP5G                 3e:6a:48:97:74:14  WPA2_AES_PSK   -88   36  300
realthread_VIP5G                 3e:6a:48:99:c2:78  WPA2_AES_PSK   -81   36  300
realthread_5G                    3c:6a:48:77:74:15  WPA2_AES_PSK   -88   36  300
realthread_VIP5G                 3e:6a:48:97:74:14  WPA2_AES_PSK   -88   36  300
TK-ML                            9c:74:6f:b9:5d:b0  WPA_AES_PSK    -79    1  144
TK-ML-GUEST                      9c:74:6f:b9:5d:b1  WPA_AES_PSK    -79    1  144
TK-ML                            9c:74:6f:b9:5d:b0  WPA_AES_PSK    -79    1  144
TK-ML-GUEST                      9c:74:6f:b9:5d:b1  WPA_AES_PSK    -79    1  144
TK-ML                            9c:74:6f:b9:5d:b0  WPA_AES_PSK    -79    1  144
TK-ML-GUEST                      9c:74:6f:b9:5d:b1  WPA_AES_PSK    -79    1  144
TEST                             82:5f:0e:3f:19:c8  WPA2_AES_PSK   -39    6  300
TEST                             82:5f:0e:3f:19:c8  WPA2_AES_PSK   -39    6  300
realthread                       3c:6a:48:77:74:14  WPA2_AES_PSK   -70    6  300
realthread_VIP                   3e:6a:48:17:74:14  WPA2_AES_PSK   -71    6  300
MLMEETING02                      68:dd:b7:f9:d1:49  WPA2_AES_PSK   -77   11  300
realthread_5G                    3c:6a:48:77:74:15  WPA2_AES_PSK   -87   36  300
realthread_VIP5G                 3e:6a:48:97:74:14  WPA2_AES_PSK   -88   36  300
realthread_5G                    3c:6a:48:c9:c2:79  WPA2_AES_PSK   -80   36  300
realthread_VIP5G                 3e:6a:48:99:c2:78  WPA2_AES_PSK   -80   36  300
TK-ML-GUEST                      9c:74:6f:b9:5d:b1  WPA_AES_PSK    -79    1  144
ChinaNet                         00:23:89:04:be:e1  OPEN           -78    1   54
realthread_VIP                   3e:6a:48:17:74:14  WPA2_AES_PSK   -96    6  300
TEST                             82:5f:0e:3f:19:c8  WPA2_AES_PSK   -39    6  300
realthread                       3c:6a:48:c9:c2:78  WPA2_AES_PSK   -71    6  300
realthread_VIP                   3e:6a:48:19:c2:78  WPA2_AES_PSK   -71    6  300
realthread                       3c:6a:48:77:74:14  WPA2_AES_PSK   -71    6  300
realthread_VIP                   3e:6a:48:17:74:14  WPA2_AES_PSK   -71    6  300
TEST                             82:5f:0e:3f:19:c8  WPA2_AES_PSK   -39    6  300
SH-online                        00:23:89:04:bf:30  OPEN           -72   11   54
ChinaNet                         00:23:89:04:bf:31  OPEN           -72   11   54
```

```
wifi join <SSID> <PASSWORD>
```

```
msh />wifi join TEST 88888888
[I/WLAN.mgnt] wifi connect success ssid:TEST
msh />[I/WLAN.lwip] Got IP address : 10.19.22.51
```

```
ping www.rt-thread.org
```

```
msh />ping www.rt-thread.org
ping: not found specified netif, using default netdev w0.
60 bytes from 111.31.66.86 icmp_seq=0 ttl=52 time=61 ms
60 bytes from 111.31.66.86 icmp_seq=1 ttl=53 time=53 ms
60 bytes from 111.31.66.86 icmp_seq=2 ttl=53 time=37 ms
60 bytes from 111.31.66.86 icmp_seq=3 ttl=53 time=65 ms
```

- After connection, perform throughput test with iperf.
- A GUI tool (`jperf`) is provided under `packages/netutils-latest/tools` .
- Extract `jperf.rar` and run the `.bat` file to launch the tool.
- Start iperf test from the board (replace `<PC_IP>` with actual PC IP):

```
iperf -c <PC_IP>
```

- Prefer 2.4 GHz network for testing (can use PC hotspot).

Notes

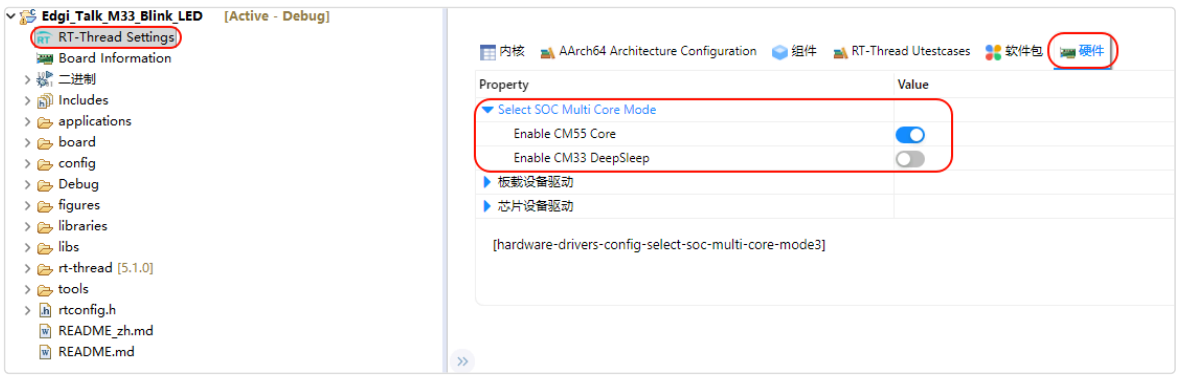
- Ensure the Wi-Fi module is correctly connected.
- Serial terminal commands allow scanning, joining, and testing Wi-Fi.

Startup Sequence

```
+------------------+
|   Secure M33     |
|  (Secure Core)   |
+------------------+
          |
          v
+------------------+
|       M33        |
| (Non-Secure Core)|
+------------------+
          |
          v
+-------------------+
|       M55         |
| (Application Core)|
+-------------------+
```

⚠️ Flash in this order strictly.

- If the example does not run, first compile and flash **Edgi_Talk_M33_Template**.
- The M55 core is started by the M33 boot chain. In current templates, **Edgi_Talk_M33_Template** already calls `Cy_SysEnableCM55(MXCM55, CY_CM55_APP_BOOT_ADDR, 10)` during board initialization, so the old `select SOC Multi Core Mode -> Enable CM55 Core` menu may no longer appear.



3. Example

3.1. Edgi-Talk_M55_LVGL Example Project

中文 | **English**

Introduction

This example is based on the **Edgi-Talk platform** and demonstrates **LVGL with multiple display demos** running on the **M55 application core** under the **RT-Thread real-time operating system**. The current default demo is **Virtual3D Animated Emoji**, which can be used to verify the LCD display, touch input, LVGL rendering flow, and M55-side graphics acceleration related configuration. The project can also be switched to LVGL official Music, Benchmark, and Stress demos as a reference for GUI application development.

LVGL Overview

LVGL (Light and Versatile Graphics Library) is an open-source embedded GUI development framework designed for resource-constrained devices. It provides modern graphical interfaces with optimized CPU and memory usage, running efficiently on both low-end MCUs and more powerful MPU platforms.

Key Features

1. **Lightweight**
   Optimized for minimal memory and CPU usage, ideal for low-power devices and resource-constrained environments.
2. **Cross-platform**
   Runs on multiple operating systems (FreeRTOS, RT-Thread, Zephyr, Linux) or bare-metal platforms. Only requires display and input drivers to be ported.
3. **Rich Widgets**
   Includes buttons, labels, sliders, charts, tables, lists, etc., and allows custom widget extensions.
4. **Advanced Rendering**
   Supports anti-aliasing, transparency, gradients, shadows, rounded corners, and animations for modern UIs.
5. **Input Device Support**
   Supports touchscreens, capacitive touch, mouse, keyboard, encoder, and multi-touch. Events are unified via LVGL's event system.
6. **Internationalization**
   UTF-8 encoding with support for bidirectional text (e.g., Arabic, Hebrew).

7. **Extensibility**
   Flexible themes, styles, and integration with file systems and image decoders.

Applications

LVGL is widely used in:

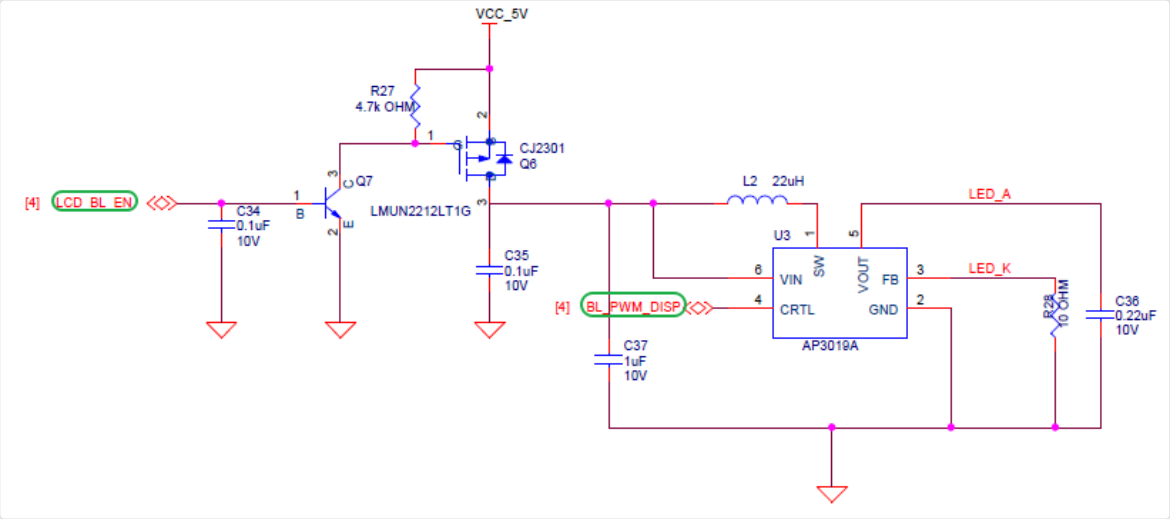
- Consumer electronics (smart home panels, smartwatches, appliances)
- Industrial HMI and instrumentation
- Automotive displays (central console, passenger screen, instrument cluster)
- Medical devices (portable monitors, handheld instruments)

Ecosystem & Community

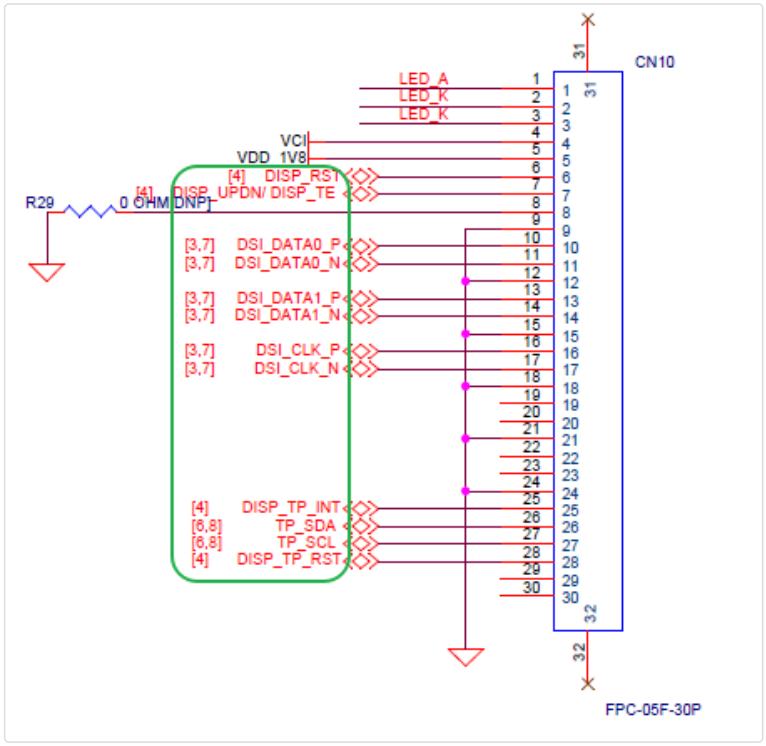
LVGL is **MIT licensed** and supported by **SquareLine Studio** for GUI design and **LVGL Simulator** for PC-based development. A large community provides open-source widgets, themes, and porting examples.

Hardware Description

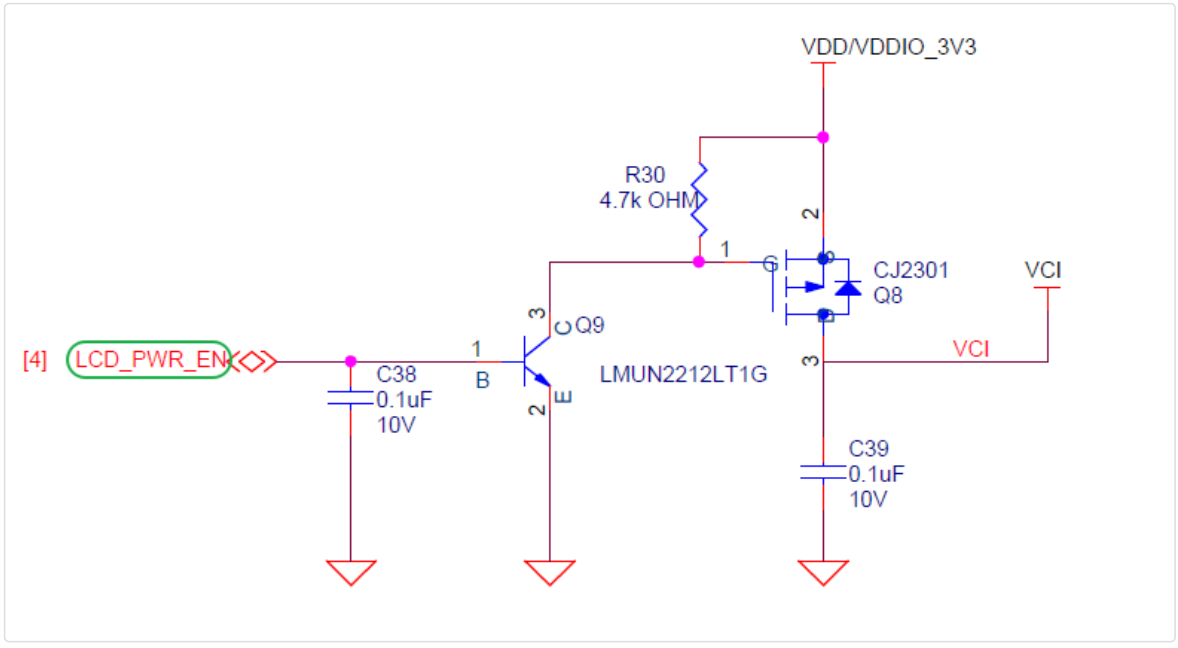
Backlight Interface



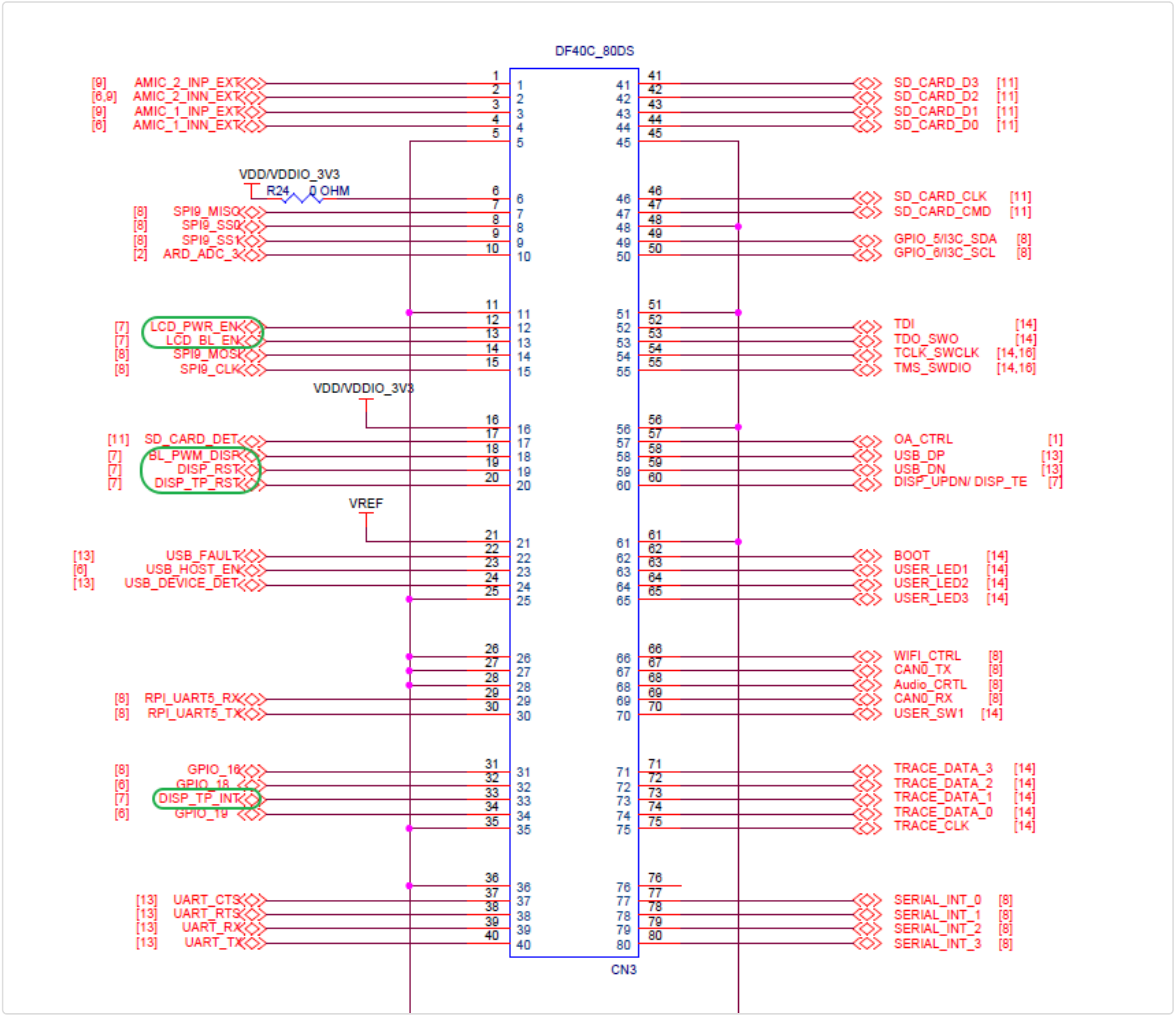
MIPI Interface

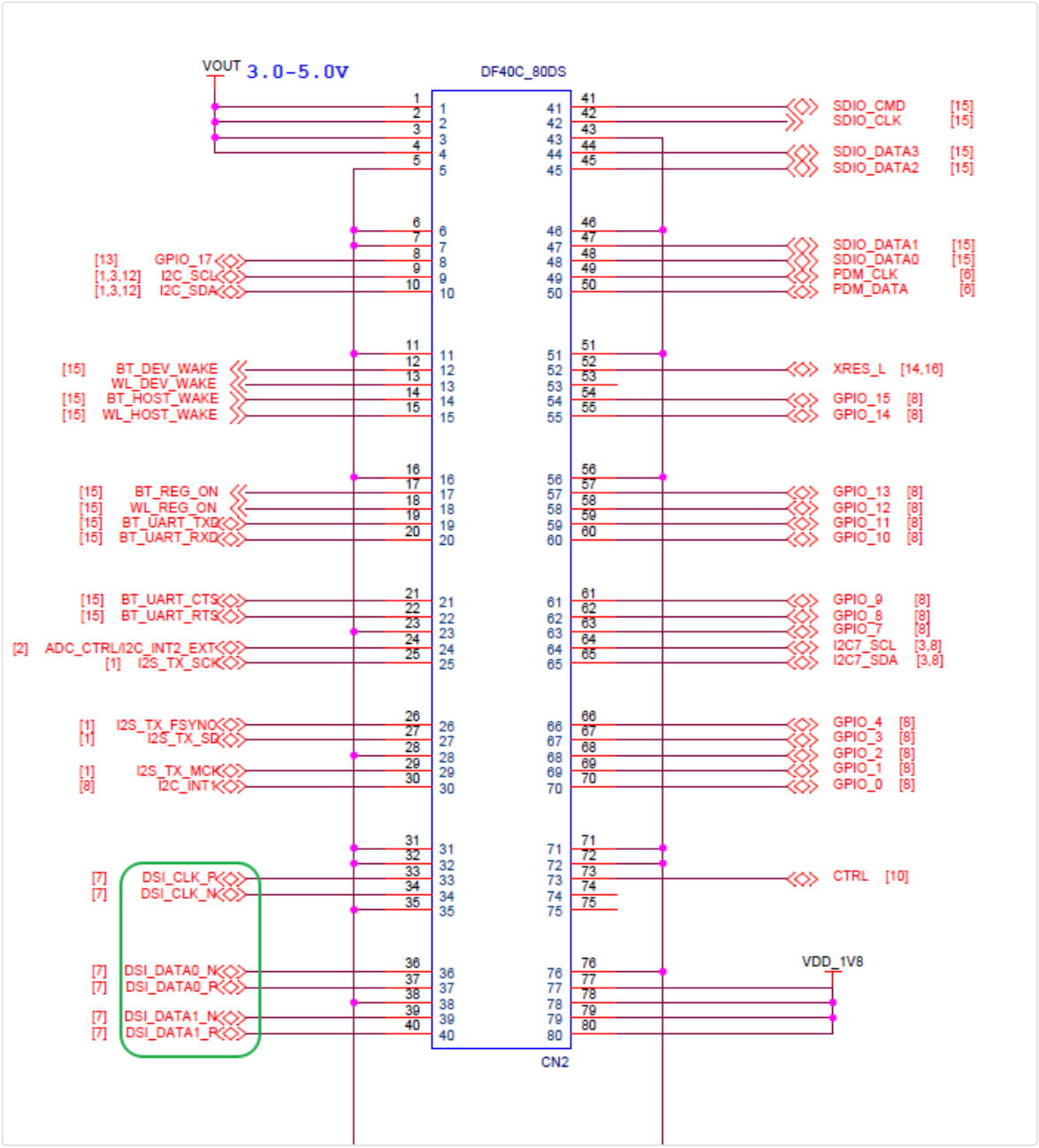


PWR Interface

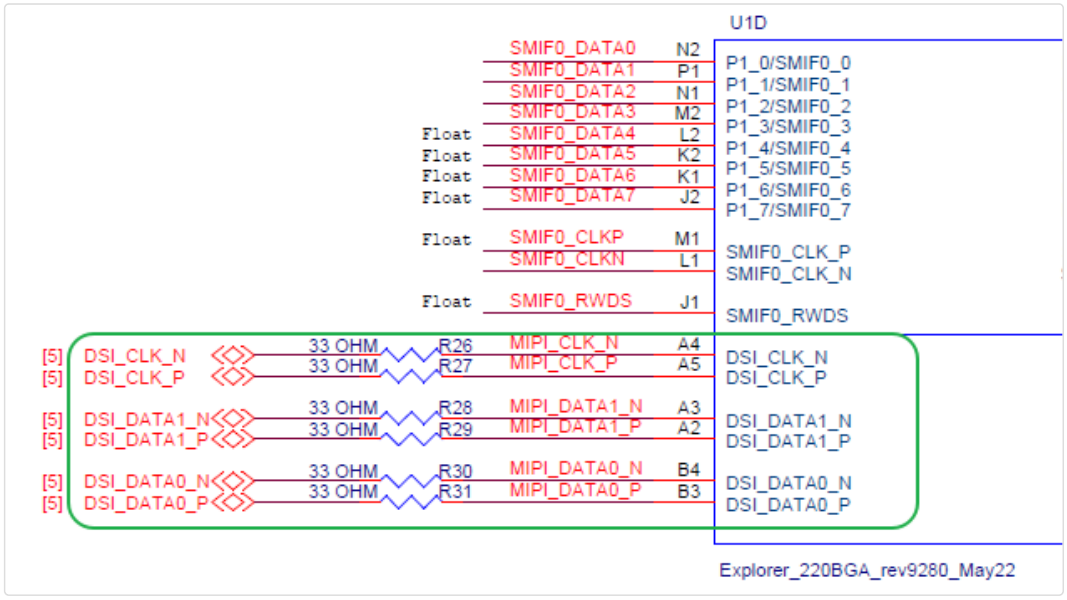


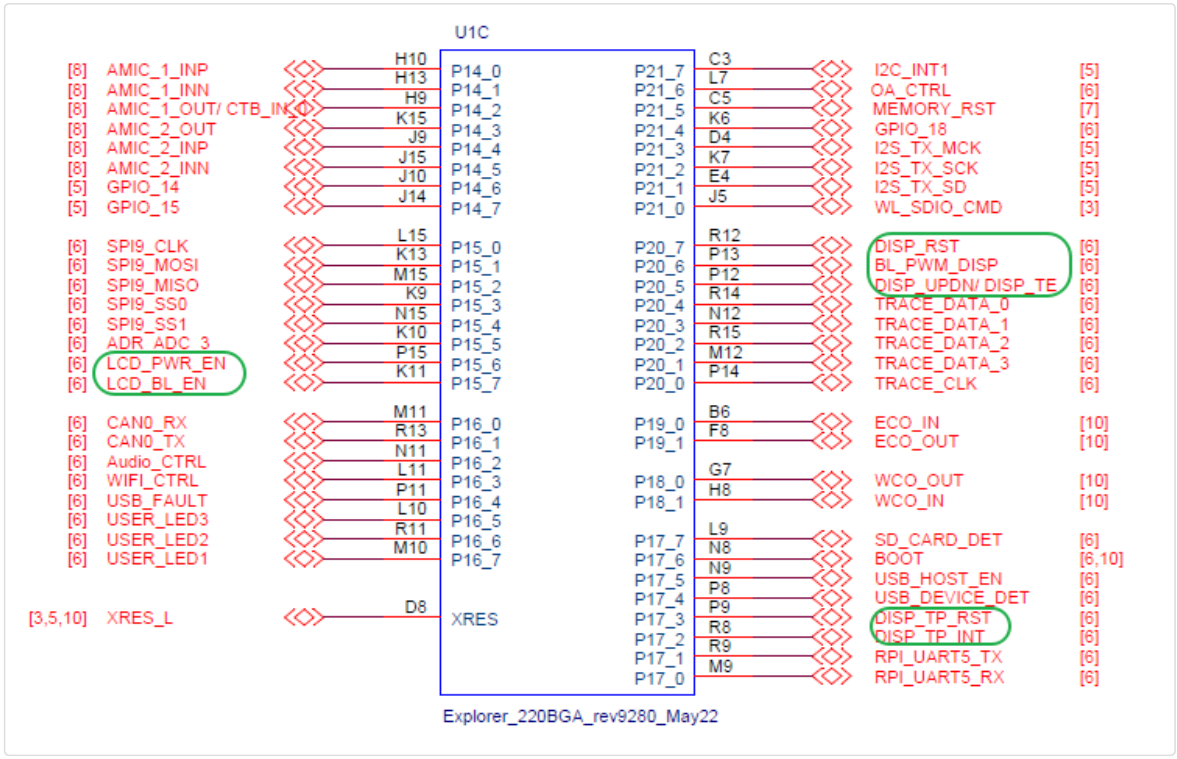
BTB Socket


DF40C_80DS
CN3
VDD/VDDIO_3V3
R24 0 OHM
VDD/VDDIO_3V3
VREF
[9] AMIC_2_INP_EXT
[6,9] AMIC_2_INN_EXT
[9] AMIC_1_INP_EXT
[6] AMIC_1_INN_EXT
[8] SPI9_MISO
[8] SPI9_SS0
[8] SPI9_SS1
[2] ARD_ADC_3
[7] LCD_PWR_EN
[7] LCD_BL_EN
[8] SPI9_MOSI
[8] SPI9_CLK
[11] SD_CARD_DET
[7] BL_PWM_DISP
[7] DISP_RST
[7] DISP_TP_RST
[13] USB_FAULT
[6] USB_HOST_EN
[13] USB_DEVICE_DET
[8] RPI_UART5_RX
[8] RPI_UART5_TX
[8] GPIO_16
[6] GPIO_18
[7] DISP_TP_INT
[6] GPIO_19
[13] UART_CTS
[13] UART_RTS
[13] UART_RX
[13] UART_TX
SD_CARD_D3 [11]
SD_CARD_D2 [11]
SD_CARD_D1 [11]
SD_CARD_D0 [11]
SD_CARD_CLK [11]
SD_CARD_CMD [11]
GPIO_5/I3C_SDA [8]
GPIO_6/I3C_SCL [8]
TDI [14]
TDO_SWO [14]
TCLK_SWCLK [14,16]
TMS_SWDIO [14,16]
OA_CTRL [1]
USB_DP [13]
USB_DN [13]
DISP_UPDN/ DISP_TE [7]
BOOT [14]
USER_LED1 [14]
USER_LED2 [14]
USER_LED3 [14]
WIFI_CTRL [8]
CAN0_TX [8]
Audio_CRTL [8]
CAN0_RX [8]
USER_SW1 [14]
TRACE_DATA_3 [14]
TRACE_DATA_2 [14]
TRACE_DATA_1 [14]
TRACE_DATA_0 [14]
TRACE_CLK [14]
SERIAL_INT_0 [8]
SERIAL_INT_1 [8]
SERIAL_INT_2 [8]
SERIAL_INT_3 [8]


VOUT 3.0-5.0V
DF40C_80DS
CN2
SDIO_CMD [15]
SDIO_CLK [15]
SDIO_DATA3 [15]
SDIO_DATA2 [15]
SDIO_DATA1 [15]
SDIO_DATA0 [15]
PDM_CLK [6]
PDM_DATA [6]
XRES_L [14,16]
GPIO_15 [8]
GPIO_14 [8]
GPIO_13 [8]
GPIO_12 [8]
GPIO_11 [8]
GPIO_10 [8]
GPIO_9 [8]
GPIO_8 [8]
GPIO_7 [8]
I2C7_SCL [3,8]
I2C7_SDA [3,8]
GPIO_4 [8]
GPIO_3 [8]
GPIO_2 [8]
GPIO_1 [8]
GPIO_0 [8]
CTRL [10]
VDD_1V8
[13] GPIO_17
[1,3,12] I2C_SCL
[1,3,12] I2C_SDA
[15] BT_DEV_WAKE
WL_DEV_WAKE
[15] BT_HOST_WAKE
[15] WL_HOST_WAKE
[15] BT_REG_ON
[15] WL_REG_ON
[15] BT_UART_TXD
[15] BT_UART_RXD
[15] BT_UART_CTS
[15] BT_UART_RTS
[2] ADC_CTRL/I2C_INT2_EXT
[1] I2S_TX_SCK
[1] I2S_TX_FSYNC
[1] I2S_TX_SD
[1] I2S_TX_MCK
[8] I2C_INT1
[7] DSI_CLK_P
[7] DSI_CLK_N
[7] DSI_DATA0_N
[7] DSI_DATA0_P
[7] DSI_DATA1_N
[7] DSI_DATA1_P

MCU Interface





Software Description

- Developed on the **Edgi-Talk platform**, running on the **M55 application core**.
- Example features:
- Initialize **LVGL 9.2**, the LCD display driver, and the touch input driver
- Start the **Virtual3D Animated Emoji** demo by default

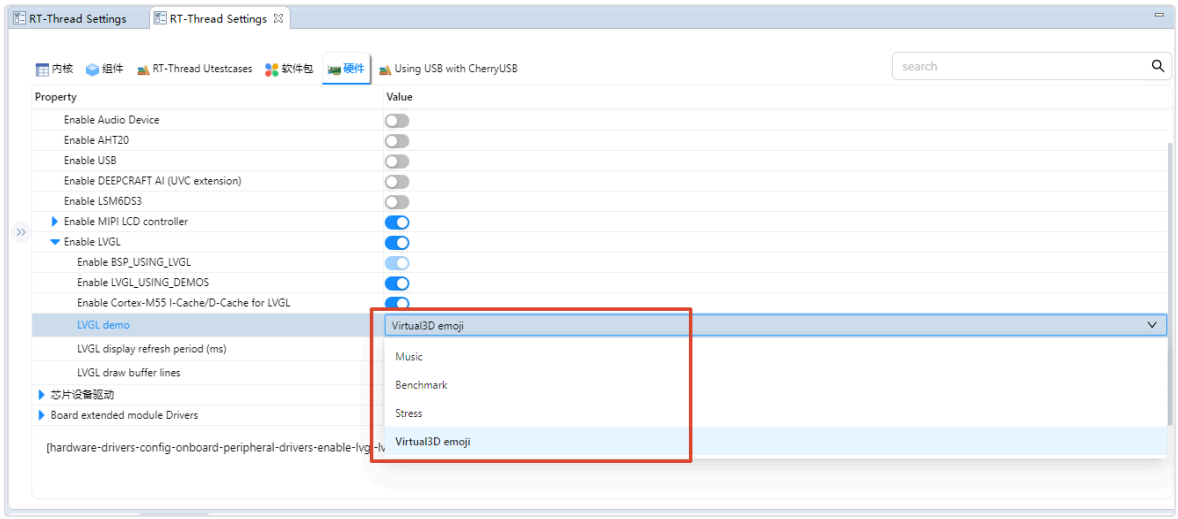
- Support switching to **lv_demo_music**, **lv_demo_benchmark**, **lv_demo_stress**, and other LVGL demos
- Enable M55 I-Cache/D-Cache by default and use AXIDMAC to optimize RGB565 area copy
- Code structure is clear for understanding display driver integration and LVGL porting.

Demo Description

This project selects the LVGL demo through the `BSP_LVGL_DEMO_*` configuration options. The default configuration is `BSP_LVGL_DEMO_VIRTUAL3D_EMOJI`.

Configuration	Demo	Description
`BSP_LVGL_DEMO_VIRTUAL3D_EMOJI`	Virtual3D Animated Emoji	Default demo. It displays a 3D Emoji animation and supports touch dragging, which helps verify the Virtual3D, LVGL display refresh, and touch input paths. This demo only supports LCD rotation at 0 or 180 degrees.
`BSP_LVGL_DEMO_MUSIC`	LVGL Music Demo	Official LVGL music UI demo, mainly used to verify complex widgets, layouts, styles, and animations.
`BSP_LVGL_DEMO_BENCHMARK`	LVGL Benchmark Demo	Official LVGL performance benchmark demo, used to observe rendering performance, frame rate, and score.
`BSP_LVGL_DEMO_STRESS`	LVGL Stress Demo	Official LVGL stress test demo. It repeatedly creates, refreshes, and destroys widgets to verify rendering stability and memory usage.

To switch demos, modify the LVGL Demo configuration in **RT-Thread Settings** or `menuconfig`. It is recommended to select only one `BSP_LVGL_DEMO_*` option at a time, then regenerate the configuration, rebuild, and download the firmware.



Usage

Build and Download

1. Open and compile the project.
2. Connect the board USB to the PC using the **onboard debugger (DAP)**.
3. Flash the generated firmware to the board.

Running Result

- After flashing and powering on, the example starts automatically.
- With the default configuration, the system starts **Virtual3D Animated Emoji**. The LCD displays the 3D Emoji animation, with the title `Virtual3D Animated Emoji` and the hint `Drag the 3D emoji`.
- The serial console prints the current demo and LCD rotation angle, for example:

```
LVGL virtual3d_emoji demo start, lcd rotation=0
```

- To check the Virtual3D demo status, run the following commands in the MSH console:

```
virtual3d_demo_stat
virtual3d_render_stat
```

- If the demo is switched to Music, Benchmark, or Stress, the LCD displays the corresponding official LVGL demo UI.

Notes

> ⚠️ ***Note:*** *This project requires* ***RT-Thread Studio 2.2.9*** *or higher.*

- To modify the **graphical configuration**, use:

```
tools/device-configurator/device-configurator.exe
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

- Save and regenerate code after modifications.
- The default `BSP_LVGL_DEMO_VIRTUAL3D_EMOJI` demo does not support LCD rotation at 90 or 270 degrees. Use 0 or 180 degrees, or switch to another LVGL demo if landscape rotation is required.
- The default configuration enables `BSP_LVGL_ENABLE_CPU_CACHE` and `BSP_LCD_USE_AXIDMAC_AREA_COPY`. If cache, framebuffer, or LCD refresh settings are changed, also check display buffer coherency.
- If the screen shows no output, check:

- LCD connections and power supply
- `lv_port_disp.c` and `lv_port_indev.c` match the actual hardware
- LCD rotation angle is compatible with the selected demo

Startup Sequence

```
+------------------+
|   Secure M33     |
|  (Secure Core)   |
+------------------+
          |
          v
+------------------+
|       M33        |
| (Non-Secure Core)|
+------------------+
          |
          v
+-------------------+
|       M55         |
| (Application Core)|
+-------------------+
```

⚠️ Strictly follow the flashing order to ensure proper system operation.

- If the example fails, first flash **Edgi_Talk_M33_Template** to ensure proper initialization.
- To enable M55, configure in **M33 project**:

```
RT-Thread Settings --> Hardware --> select SOC Multi Core Mode --> Enable CM55 (
```

3.2. XiaoZhi Sample Project

中文 | **English**

Introduction

This sample project is based on the **Edgi-Talk platform**, demonstrating the **basic functions of the XiaoZhi voice interaction device**, running on the **RT-Thread real-time operating system**. With this project, users can quickly verify the device's **WiFi connection**, **button wake-up**, and **voice interaction** capabilities, providing a fundamental reference for further application development.

Software Description

- The project is developed based on the **Edgi-Talk** platform.
- The sample includes the following functions:
- WiFi connection and status display
- Button wake-up and voice interaction
- Device state management (standby, listening, sleep, etc.)

Usage

n> ⚠️ **Note:** This project requires **RT-Thread Studio 2.2.9** or higher.

Prepare Wi-Fi resources (first-time setup)

The Wi-Fi host driver loads three blobs (firmware `.bin`, regulatory `.clm_blob`, and board-specific `nvram.txt`) from FAL before it can power up the radio. These files live outside the application image, so flashing a new binary will not refresh them automatically. The default bundles for Edgi-Talk live in the workspace root under `resources/`.

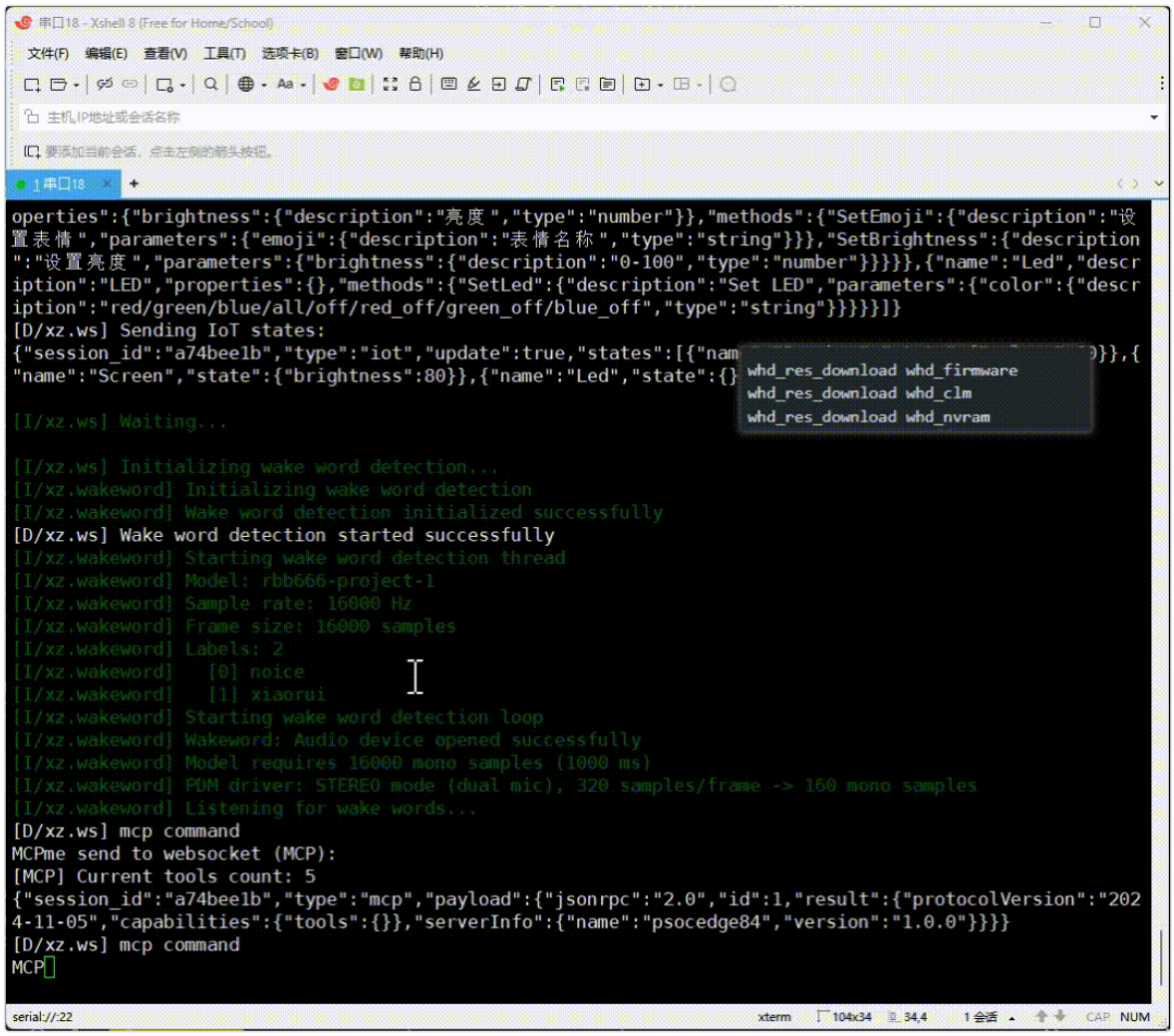
- Keep `WHD_RESOURCES_IN_EXTERNAL_STORAGE_FAL` enabled in menuconfig and make sure the FAL table provides the `whd_firmware`, `whd_clm`, and `whd_nvram` partitions (the defaults reserve 512 KB + 32 KB + 32 KB of on-chip flash).
- Attach a serial terminal, reboot into the `msh` prompt, and run the download helper for each partition:

```
whd_res_download whd_firmware
whd_res_download whd_clm
whd_res_download whd_nvram
```

Each command switches to YMODEM mode. Use a terminal that supports YMODEM upload (Xshell) to send the matching files from the top-level `resources/` directory .

- Wait for the `Download … success` message before moving to the next partition.

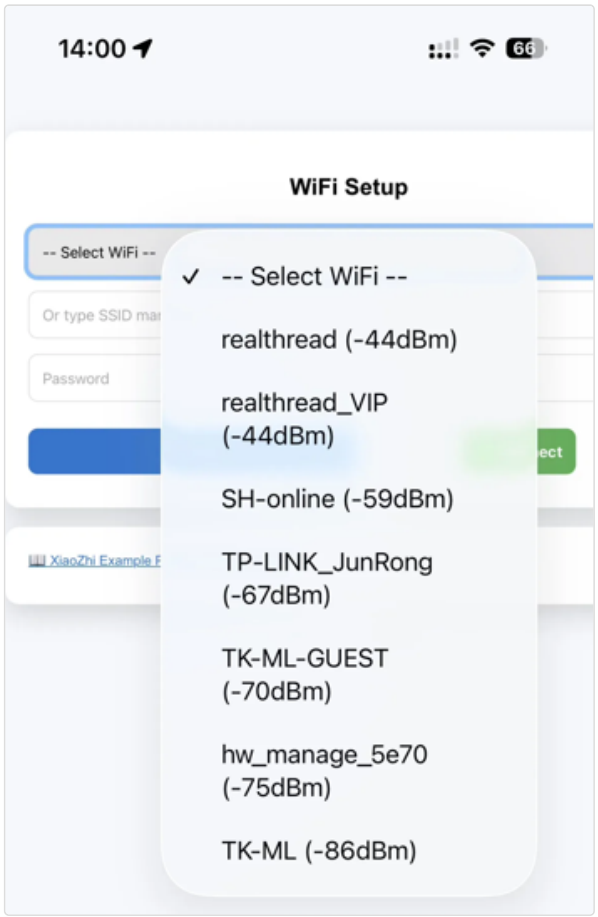
- Power-cycle or reset the board after the three transfers so Wi-Fi starts with the freshly stored blobs. Re-run the command whenever you update the firmware/CLM/NVRAM bundle.



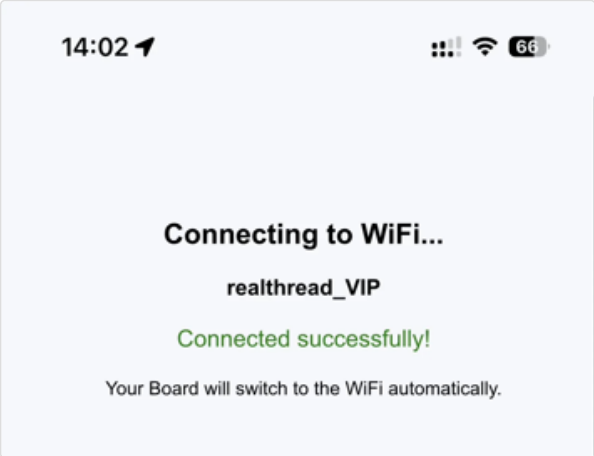
1. First-time setup (AP configuration)

1. When the development board starts, it will enter **AP mode**.
   Connect your phone or computer to the device hotspot (password shown on the screen):

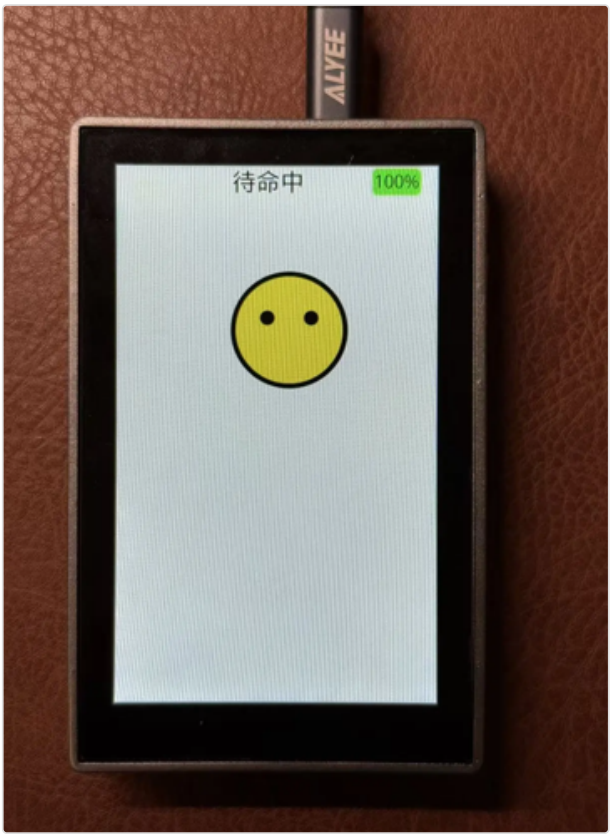
2. After a successful connection, open a browser and enter **192.168.169.1** to access the configuration interface:
3. Click **Scan** to search for nearby Wi-Fi hotspots:



4. After the WiFi connection is successful, the following page will be displayed:



5. When the device screen shows **“Standby”**, it means voice interaction is ready:

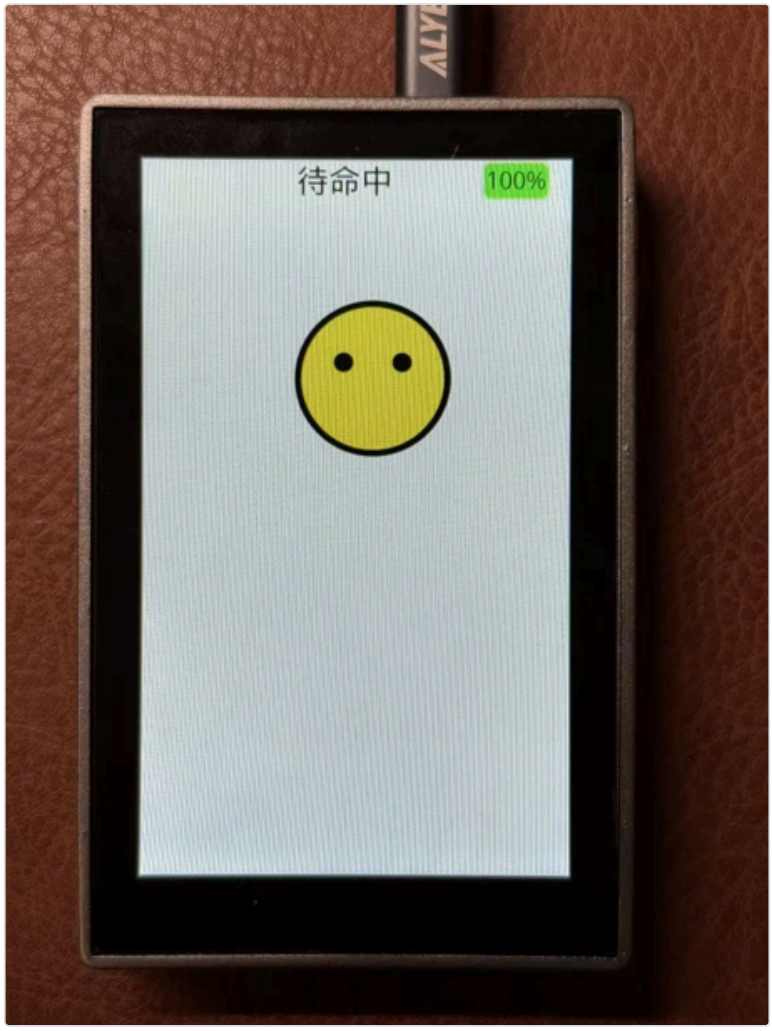


> ***Tip:*** *Press the* ***first user button*** *on the development board to enter voice input. After waiting 1–2 seconds, XiaoZhi will automatically respond.*

XiaoZhi Expression Meaning

1. Connecting (please wait)

2. Monitoring (press the button to start talking)



3. Listening (processing your speech)

4. Speaking (XiaoZhi is responding to you)

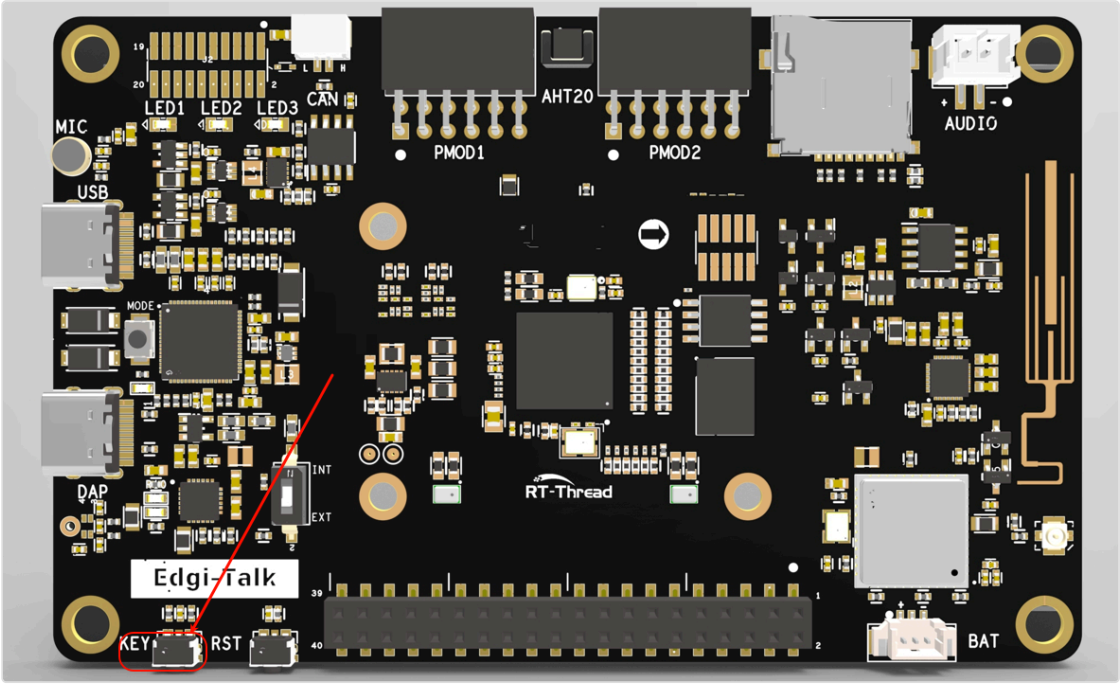
5. Sleep mode (low power)

> *To exit sleep: press the button → wait for “Standby” → interaction becomes available.*

Running Effect

- After flashing, the device will start the sample automatically on power-up.

- Press the top button once to enter the **Listening** state and interact with the device.



Notes

> ⚠️ ***Note:*** *This project requires* ***RT-Thread Studio 2.2.9*** *or higher.*

- For first-time use, visit the XiaoZhi official website to complete backend binding.

  Press the user button to display the verification code on the screen.

- Please ensure the WiFi SSID and password are correct and that you are using a **2.4GHz network**.
- The device requires an Internet connection to function properly.
- If you need to modify the **graphical configuration**, use the following tools:

```
tools/device-configurator/device-configurator.exe
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

- Save changes and regenerate code after modification.

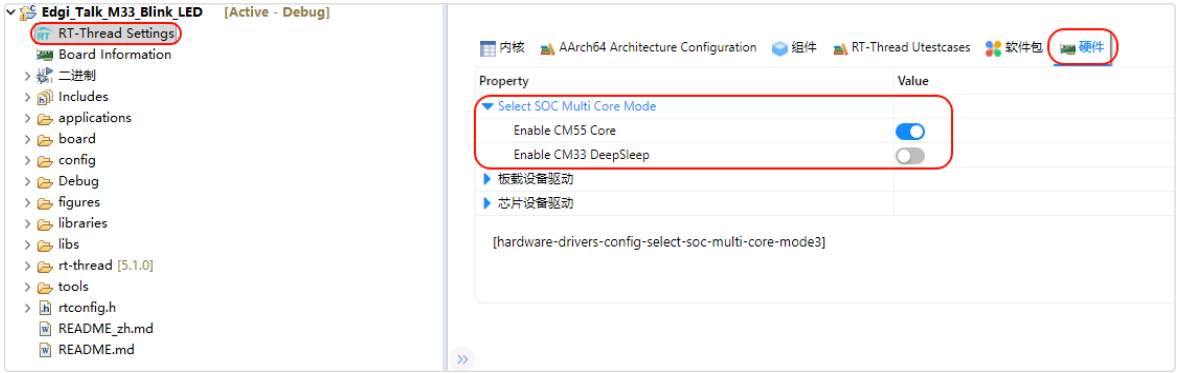
Startup Sequence

```
+------------------+
|   Secure M33     |
|  (Secure Core)   |
+------------------+
          |
          v
+------------------+
|       M33        |
| (Non-Secure Core)|
+------------------+
          |
          v
+------------------+
|       M55        |
| (Application Core)|
+------------------+
```

⚠️ Flash in this order strictly to ensure proper operation.

- If the example does not run, first compile and flash **Edgi_Talk_M33_Template**.
- To enable M55:

```
RT-Thread Settings --> Hardware --> select SOC Multi Core Mode --> Enable CM55 (
```



4. USB

4.1. Edgi_Talk_Extend_Screen CherryUSB USB Extend Screen Example

中文 | **English**

Overview

This project integrates **CherryUSB** on the **M55 core** of the Edgi-Talk board. It is prepared for **USB device mode**, uses the Infineon **DWC2** IP, and implements a **Windows USB extend screen**.

Default Configuration

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_DEVICE = y`
- `RT_CHERRYUSB_DEVICE_SPEED_HS = y`
- `RT_CHERRYUSB_DEVICE_DWC2_INFINEON = y`
- Device template: **none** (user application)
- Display transport: vendor interface with RGB565 frames

Build and Flash

1. Build the project in RT-Thread Studio or with SCons.
2. Flash the firmware via KitProg3 (DAP).
3. Connect the Type-C USB port for device enumeration.

Configuration (Switching Modes)

Open RT-Thread Studio and go to:

```
RT-Thread Settings -> USB -> CherryUSB
```

- **Device mode**: enable `RT_CHERRYUSB_DEVICE`, select device IP under **CHERRYUSB_DEVICE_IP** (default: `RT_CHERRYUSB_DEVICE_DWC2_INFINEON`).
- **Device classes**: enable class drivers and select a template under **Select usb device template**.

If an IP/class requires extra parameters, edit:

- `libraries/Common/board/ports/usb/usb_config.h`

Windows IDD Driver

Windows Indirect Display Driver (IDD) provides a user-mode driver model for monitors that are not connected to a traditional GPU output.

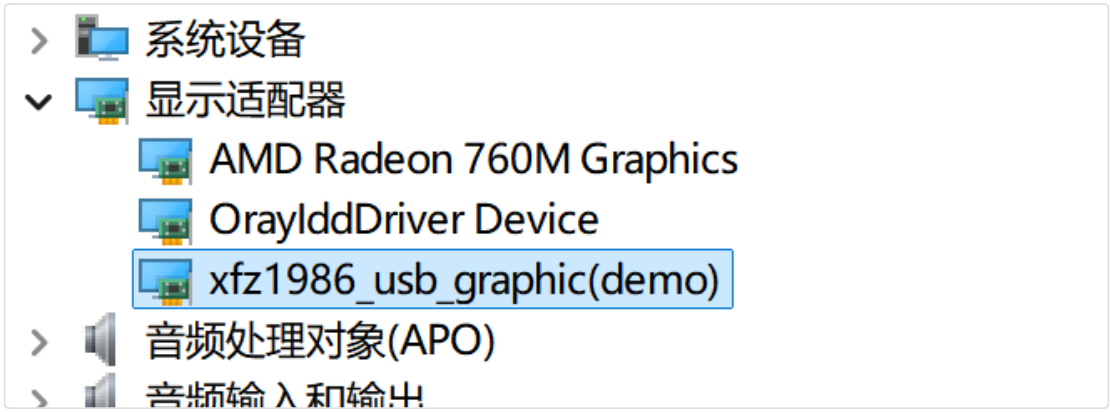
Reference: https://learn.microsoft.com/windows-hardware/drivers/display/indirect-display-driver-model-overview

This driver is based on: https://github.com/chuanjinpang/win10_idd_xfz1986_usb_graphic_driver_display

If you need to modify it, download and rebuild it yourself.

How to use

1. Install `resources\USB_Graphic\xfz1986_usb_graphic_250224_rc_sign.exe`.
2. After installation, a new display appears under Display adapters.



Notes

- The driver communicates via a vendor interface, supports multiple resolutions and formats, and is controlled by string descriptors. See: https://github.com/chuanjinpang/win10_idd_xfz1986_usb_graphic_driver_display/blob/main/README.md
- The driver supports Windows 10 and Windows 11 only.

Startup Sequence

The M55 core depends on the M33 boot flow. Flash in this order:

```
+------------------+
|   Secure M33     |
|  (Secure Core)   |
+------------------+
          |
```

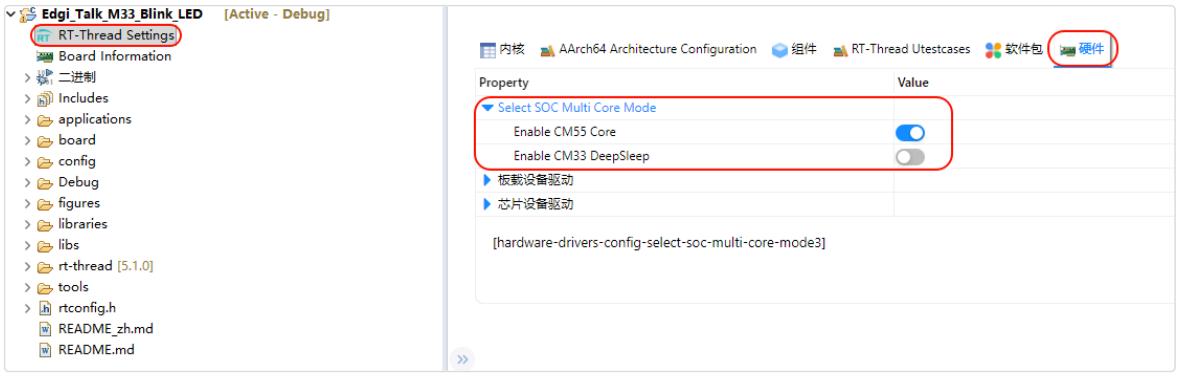
```
          v
+------------------+
|       M33        |
| (Non-Secure Core)|
+------------------+
          |
          v
+--------------------+
|       M55          |
| (Application Core)|
+--------------------+
```

Notes

> ⚠️ ***Note:*** *This project requires* ***RT-Thread Studio 2.2.9*** *or higher.*

- This project targets the M55 core in USB device mode.
- For USB role switching on M55, see projects/Edgi_Talk_CherryUSB/Edgi_Talk_M55_USB_RoleSwitch/README.md.
- For USB role switching on M33, see projects/Edgi_Talk_CherryUSB/Edgi_Talk_M33_USB_RoleSwitch/README.md.
- If the M55 example does not run, flash **Edgi_Talk_M33_Template** first.
- Enable CM55 in the M33 project:

```
RT-Thread Settings -> Hardware -> select SOC Multi Core Mode -> Enable CM55 C
```



4.2. Edgi_Talk_M33_USB_RoleSwitch CherryUSB Example Project

中文 | **English**

Overview

This project integrates **CherryUSB** on the **M33 core** of the Edgi-Talk board. It is prepared for **USB device mode** and uses the Infineon **DWC2** IP.

Default Configuration

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_DEVICE = y`
- `RT_CHERRYUSB_DEVICE_SPEED_HS = y`
- `RT_CHERRYUSB_DEVICE_DWC2_INFINEON = y`
- Device template: **CDC** (user application)

Build and Flash

1. Build the project in RT-Thread Studio or with SCons.
2. Flash the firmware via KitProg3 (DAP).
3. Connect the Type-C USB port for device enumeration.
4. Use CherryUSB test scripts to benchmark the CDC device.

Configuration (Switching Modes)

Use the board Kconfig entry to switch roles. The graphical board configurator does not need to be regenerated for USB host/device selection.

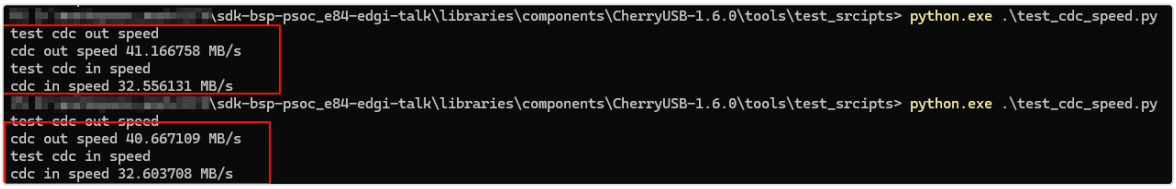
```
RT-Thread Settings ->
Hardware Drivers Config ->
Onboard Peripheral Drivers ->
Enable USB ->
USB role
```

- **Device CDC ACM**: selects `RT_CHERRYUSB_DEVICE`, high speed, Infineon DWC2, CDC ACM, and the CDC ACM template.
- **Host MSC**: selects `RT_CHERRYUSB_HOST`, Infineon DWC2, MSC, DFS, and ElmFat.

If an IP/class requires extra parameters, edit:

- `libraries/Common/board/ports/usb/usb_config.h`

USB CDC Device Result



USB U-Disk (Host) Result

```
****************** PSOC Edge MCU: CM33 Secure Mode*****************
PSRAM Cache is Enabled
PSRAM init successful
****************** PSOC Edge MCU: CM33 Secure Mode Exit*****************


 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Jan 28 2026 13:27:14
 2006 - 2022 Copyright by RT-Thread team
Hello RT-Thread
This core is cortex-m33
msh />usbh_init 0 0x44900000
msh />[I/USB] ========== dwc2 hcd params ==========
[I/USB] CID:12345678
[I/USB] GSNPSID:4f54420a
[I/USB] GHWCFG1:00000000
[I/USB] GHWCFG2:228fe052
[I/USB] GHWCFG3:1a4f80e8
[I/USB] GHWCFG4:e2103a20
[I/USB] dwc2 has 16 channels and dfifo depth(32-bit words) is 6735
[I/usbh_hub] New high-speed device on Bus 0, Hub 1, Port 1 connected
[I/usbh_core] New device found,idVendor:05e3,idProduct:0610,bcdDevice:0663
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading hub class driver on interface 0
[I/usbh_hub] Ep=81 Attr=03 Mps=1 Interval=12 Mult=00
[I/usbh_hub] Register HUB Class:/dev/hub2
[I/usbh_hub] New high-speed device on Bus 0, Hub 2, Port 2 connected
[I/usbh_core] New device found,idVendor:0930,idProduct:6545,bcdDevice:0110
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading msc class driver on interface 0
[I/usbh_msc] Get max LUN:1
[I/usbh_msc] Ep=81 Attr=02 Mps=512 Interval=00 Mult=00
[I/usbh_msc] Ep=02 Attr=02 Mps=512 Interval=00 Mult=00
[I/usbh_msc] Register MSC Class:/dev/sda
[I/usbh_msc] Capacity info:
[I/usbh_msc] Block num:121995264,block size:512
udisk: /dev/sda mount successfully
[I/usbh_hub] New high-speed device on Bus 0, Hub 2, Port 4 connected
[I/usbh_core] New device found,idVendor:0bda,idProduct:8153,bcdDevice:3100
[I/usbh_core] The device has 2 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[E/usbh_core] Do not support Class:0xff, Subclass:0xff, Protocl:0x00 on interface 0

msh />
msh />ls
Directory /:
grub                <DIR>
ventoy              <DIR>
EFI                 <DIR>
ENROLL_THIS_KEY_IN_M829
tool                <DIR>
System Volume Inform<DIR>
msh />
```

Startup Sequence

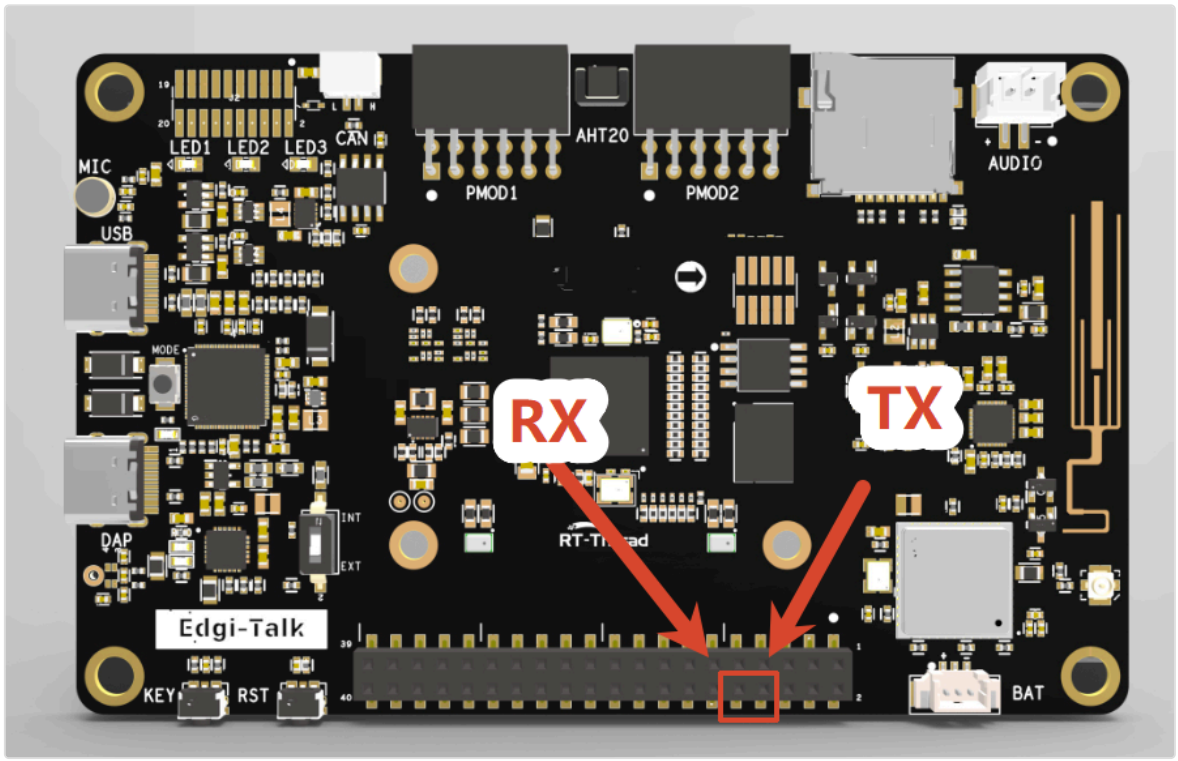
The M55 core depends on the M33 boot flow. Flash in this order:

```
+------------------+
|   Secure M33     |
|  (Secure Core)   |
+------------------+
          |
          v
+------------------+
|       M33        |
| (Non-Secure Core)|
+------------------+
          |
          v
+-------------------+
|       M55         |
| (Application Core)|
+-------------------+
```

Notes

> ⚠️ ***Note:*** *This project requires* ***RT-Thread Studio 2.2.9*** *or higher.*

- The M33 project serial log is not output through the onboard DAP virtual COM port directly. To view `msh />` and demo logs, connect an external USB-to-UART adapter, such as CH340. The connection position is shown below. Connect the board RX to the UART adapter TX, connect the board TX to the UART adapter RX, and set the host serial baud rate to 115200:



- This project targets the M33 core and supports USB device CDC ACM and USB host MSC roles.
- For the M55 USB role-switching example, see projects/Edgi_Talk_CherryUSB/Edgi_Talk_M55_USB_RoleSwitch/README.md.
- To run an M55 project, it is recommended to flash **Edgi_Talk_M33_Template** first. It is a clean M33 project and is suitable as the base firmware before starting M55.

4.3. Edgi_Talk_M55_USB_RoleSwitch CherryUSB Example Project

Chinese | **English**

Overview

This project integrates **CherryUSB** on the **M55 core** of the Edgi-Talk board. It is configured for **USB device CDC ACM mode** by default and can be switched to **USB host MSC mode** through board Kconfig without regenerating graphical board configuration files.

Default Configuration

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_DEVICE = y`
- `RT_CHERRYUSB_DEVICE_SPEED_HS = y`
- `RT_CHERRYUSB_DEVICE_DWC2_INFINEON = y`
- `RT_CHERRYUSB_DEVICE_CDC_ACM = y`
- Device template: **CDC ACM**

Build and Flash

1. Build the project in RT-Thread Studio or with SCons.
2. Flash the firmware via KitProg3 (DAP).
3. Connect the Type-C USB port for device enumeration.
4. After switching to host MSC mode, connect a USB storage device for host operation.

Configuration (Switching Modes)

Use the board Kconfig entry to switch USB roles. The graphical board configurator does not need to be regenerated for USB host/device selection.

```
RT-Thread Settings ->
Hardware Drivers Config ->
Onboard Peripheral Drivers ->
Enable USB ->
USB role
```

- **Device CDC ACM**: selects `RT_CHERRYUSB_DEVICE`, high speed, Infineon DWC2, CDC ACM, and the CDC ACM template.
- **Host MSC**: selects `RT_CHERRYUSB_HOST`, Infineon DWC2, MSC, DFS, and ElmFat.

If an IP/class requires extra parameters, edit:

- `libraries/Common/board/ports/usb/usb_config.h`

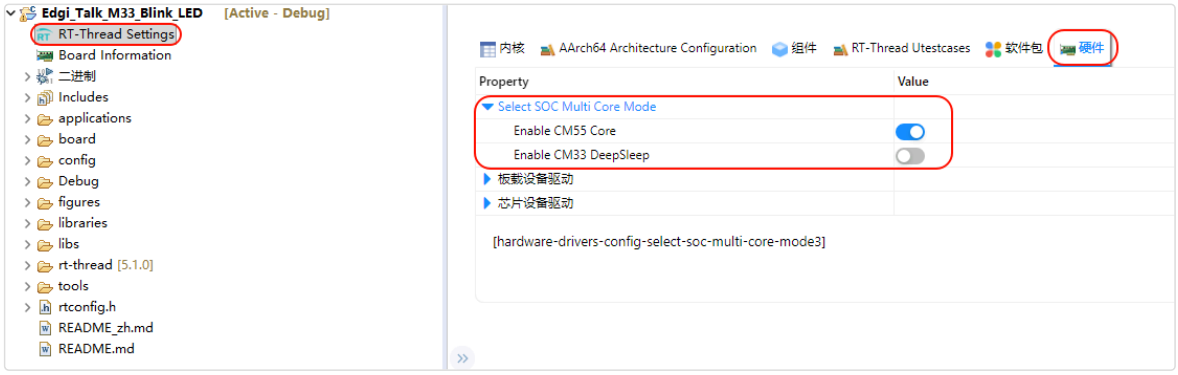
USB U-Disk (Host) Result

```
****************** PSOC Edge MCU: CM33 Secure Mode*****************
PSRAM Cache is Enabled
PSRAM init successful
****************** PSOC Edge MCU: CM33 Secure Mode Exit*****************


 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Jan 28 2026 13:27:14
 2006 - 2022 Copyright by RT-Thread team
Hello RT-Thread
This core is cortex-m33
msh />usbh_init 0 0x44900000
msh />[I/USB] ========== dwc2 hcd params ==========
[I/USB] CID:12345678
[I/USB] GSNPSID:4f54420a
[I/USB] GHWCFG1:00000000
[I/USB] GHWCFG2:228fe052
[I/USB] GHWCFG3:1a4f80e8
[I/USB] GHWCFG4:e2103a20
[I/USB] dwc2 has 16 channels and dfifo depth(32-bit words) is 6735
[I/usbh_hub] New high-speed device on Bus 0, Hub 1, Port 1 connected
[I/usbh_core] New device found,idVendor:05e3,idProduct:0610,bcdDevice:0663
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading hub class driver on interface 0
[I/usbh_hub] Ep=81 Attr=03 Mps=1 Interval=12 Mult=00
[I/usbh_hub] Register HUB Class:/dev/hub2
[I/usbh_hub] New high-speed device on Bus 0, Hub 2, Port 2 connected
[I/usbh_core] New device found,idVendor:0930,idProduct:6545,bcdDevice:0110
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading msc class driver on interface 0
[I/usbh_msc] Get max LUN:1
[I/usbh_msc] Ep=81 Attr=02 Mps=512 Interval=00 Mult=00
[I/usbh_msc] Ep=02 Attr=02 Mps=512 Interval=00 Mult=00
[I/usbh_msc] Register MSC Class:/dev/sda
[I/usbh_msc] Capacity info:
[I/usbh_msc] Block num:121995264,block size:512
udisk: /dev/sda mount successfully
[I/usbh_hub] New high-speed device on Bus 0, Hub 2, Port 4 connected
[I/usbh_core] New device found,idVendor:0bda,idProduct:8153,bcdDevice:3100
[I/usbh_core] The device has 2 bNumConfigurations
[I/usbh_core] The device has 1 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[E/usbh_core] Do not support Class:0xff, Subclass:0xff, Protocl:0x00 on interface 0

msh />
msh />ls
Directory /:
grub                <DIR>
ventoy              <DIR>
EFI                 <DIR>
ENROLL_THIS_KEY_IN_M829
tool                <DIR>
System Volume Inform<DIR>
msh />
```

CM55 Enable Configuration



Startup Sequence

The M55 core depends on the M33 boot flow. Flash in this order:

```
+------------------+
|   Secure M33     |
|  (Secure Core)   |
+------------------+
          |
          v
+------------------+
|       M33        |
| (Non-Secure Core)|
+------------------+
          |
          v
+--------------------+
|       M55          |
| (Application Core)|
+--------------------+
```

Notes

> *Note: This project requires **RT-Thread Studio 2.2.9** or higher.*

- This project targets the M55 core and supports both USB device CDC ACM and USB host MSC roles.
- For the M33 USB role-switching example, see projects/Edgi_Talk_CherryUSB/Edgi_Talk_M33_USB_RoleSwitch/README.md.
- If the M55 example does not run, flash **Edgi_Talk_M33_Template** first.
- Enable CM55 in the M33 project:

```
RT-Thread Settings -> Hardware -> select SOC Multi Core Mode -> Enable CM55 C
```

4.4. Edgi_Talk_M55_USB_UVC CherryUSB Example Project

中文 | **English**

Overview

This project runs on the **Edgi-Talk M55 core** and enables **CherryUSB host mode** on the Infineon **DWC2** controller, with **USB Video Class (UVC) host support** enabled by default.

At startup, firmware calls `usbh_initialize(0, USBHS_BASE, NULL)` to bring up USB host. The main loop toggles the green LED (P16_6) as a liveness indicator.

Default Configuration

Key macros in `rtconfig.h`:

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_HOST = y`
- `RT_CHERRYUSB_HOST_DWC2_INFINEON = y`
- `RT_CHERRYUSB_HOST_MSC = y`
- `RT_CHERRYUSB_HOST_VIDEO = y`

This gives a ready baseline for UVC device enumeration and further video application integration.

Build and Flash

1. Build in RT-Thread Studio or with SCons.
2. Flash firmware via KitProg3 (DAP).
3. Connect a UVC camera (or other USB device) to the Type-C port.

Configuration

Open RT-Thread Studio:

```
RT-Thread Settings -> USB -> CherryUSB
```

Recommended checks:

- Enable `RT_CHERRYUSB_HOST`
- Select `RT_CHERRYUSB_HOST_DWC2_INFINEON` as host IP
- Enable `RT_CHERRYUSB_HOST_VIDEO` for UVC
- Keep or disable `MSC` and other host classes as needed

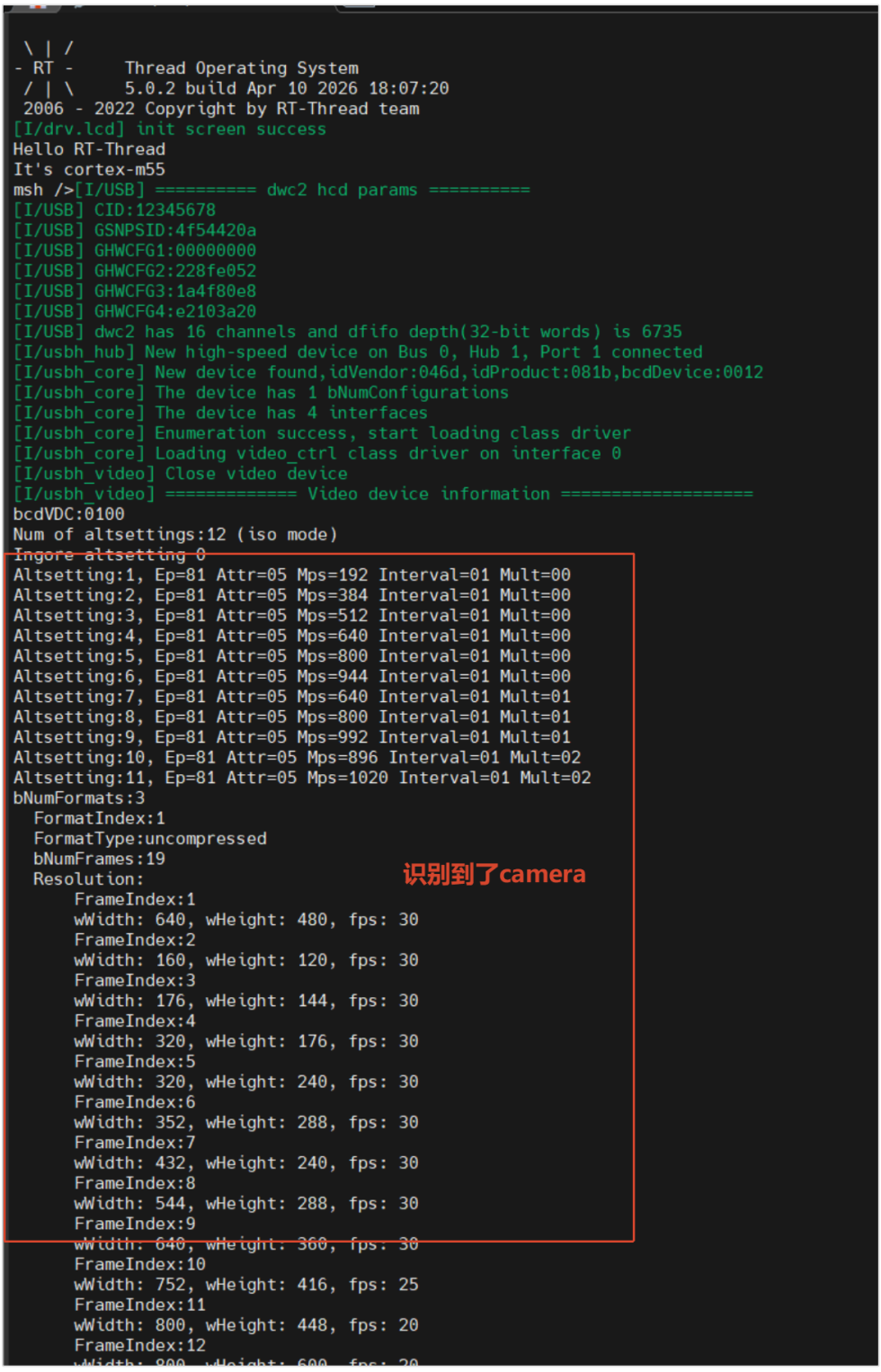
If extra USB parameters are needed, edit:

- `libraries/Common/board/ports/usb/usb_config.h`

Runtime Result

1. Plug a USB camera into the board's USB-OTG interface.

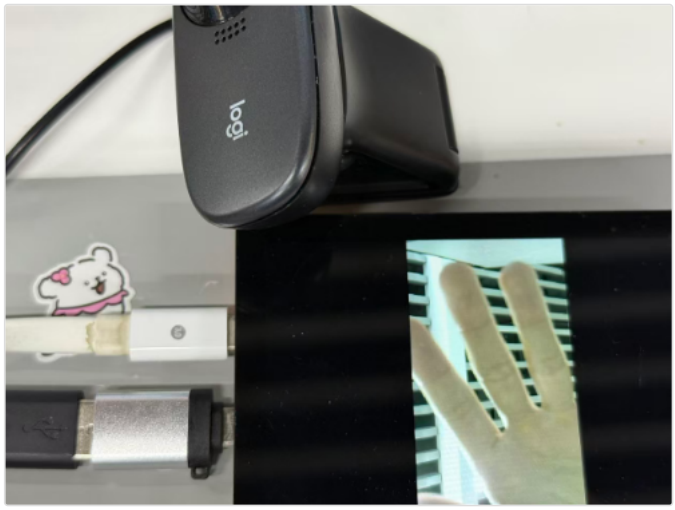
2. After boot, the system automatically detects camera capabilities (resolution, format, and related information).




```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Apr 10 2026 18:07:20
 2006 - 2022 Copyright by RT-Thread team
[I/drv.lcd] init screen success
Hello RT-Thread
It's cortex-m55
msh />[I/USB] ========== dwc2 hcd params ==========
[I/USB] CID:12345678
[I/USB] GSNPSID:4f54420a
[I/USB] GHWCFG1:00000000
[I/USB] GHWCFG2:228fe052
[I/USB] GHWCFG3:1a4f80e8
[I/USB] GHWCFG4:e2103a20
[I/USB] dwc2 has 16 channels and dfifo depth(32-bit words) is 6735
[I/usbh_hub] New high-speed device on Bus 0, Hub 1, Port 1 connected
[I/usbh_core] New device found,idVendor:046d,idProduct:081b,bcdDevice:0012
[I/usbh_core] The device has 1 bNumConfigurations
[I/usbh_core] The device has 4 interfaces
[I/usbh_core] Enumeration success, start loading class driver
[I/usbh_core] Loading video_ctrl class driver on interface 0
[I/usbh_video] Close video device
[I/usbh_video] ============= Video device information ==================
bcdVDC:0100
Num of altsettings:12 (iso mode)
Ingore altsetting 0
Altsetting:1, Ep=81 Attr=05 Mps=192 Interval=01 Mult=00
Altsetting:2, Ep=81 Attr=05 Mps=384 Interval=01 Mult=00
Altsetting:3, Ep=81 Attr=05 Mps=512 Interval=01 Mult=00
Altsetting:4, Ep=81 Attr=05 Mps=640 Interval=01 Mult=00
Altsetting:5, Ep=81 Attr=05 Mps=800 Interval=01 Mult=00
Altsetting:6, Ep=81 Attr=05 Mps=944 Interval=01 Mult=00
Altsetting:7, Ep=81 Attr=05 Mps=640 Interval=01 Mult=01
Altsetting:8, Ep=81 Attr=05 Mps=800 Interval=01 Mult=01
Altsetting:9, Ep=81 Attr=05 Mps=992 Interval=01 Mult=01
Altsetting:10, Ep=81 Attr=05 Mps=896 Interval=01 Mult=02
Altsetting:11, Ep=81 Attr=05 Mps=1020 Interval=01 Mult=02
bNumFormats:3
  FormatIndex:1
  FormatType:uncompressed
  bNumFrames:19
  Resolution:
      FrameIndex:1
      wWidth: 640, wHeight: 480, fps: 30
      FrameIndex:2
      wWidth: 160, wHeight: 120, fps: 30
      FrameIndex:3
      wWidth: 176, wHeight: 144, fps: 30
      FrameIndex:4
      wWidth: 320, wHeight: 176, fps: 30
      FrameIndex:5
      wWidth: 320, wHeight: 240, fps: 30
      FrameIndex:6
      wWidth: 352, wHeight: 288, fps: 30
      FrameIndex:7
      wWidth: 432, wHeight: 240, fps: 30
      FrameIndex:8
      wWidth: 544, wHeight: 288, fps: 30
      FrameIndex:9
      wWidth: 640, wHeight: 360, fps: 30
      FrameIndex:10
      wWidth: 752, wHeight: 416, fps: 25
      FrameIndex:11
      wWidth: 800, wHeight: 448, fps: 20
      FrameIndex:12
      wWidth: 800, wHeight: 600, fps: 20
```

3. In the MSH terminal, run `usbh_uvc_start 0 320 240` (`0` = YUYV, `1` = MJPEG, resolution `320x240`).

Note: Select parameters based on your USB camera's actual supported modes. The maximum supported resolution is only 432x240.



4. Check the serial log output. Frame-rate information is printed once per second.

```
msh />usbh_uvc_start 0 320 240
[I/uvc_app] UVC start: 320x240 format=yuyv
[I/USB] uvc rx fifo limit:2172
msh />[I/uvc_app] UVC fps:0 disp:0 drop:0 urb:0 eof:0 fid:0 err:0 empty:0 data:0 hdronly:0
 payload:0 done:0 last_off:0
[I/uvc_disp] LCD: 512x800 16bpp fb=0x262c8000 size=819200
[I/usbh_video] Probe fmt:1 frame:5 interval:333333 maxframe:153600 maxpayload:1600
[I/usbh_video] Commit fmt:1 frame:5 interval:333333 maxframe:153600 maxpayload:1600
[I/usbh_video] Ep=81 Attr=05 Mps=992 Interval=01 Mult=01
[I/usbh_video] Open video and select formatidx:1, frameidx:5, altsetting:9
[I/USB] uvc selected size:320x240
[I/USB] uvc mps :1600
[I/USB] uvc iso urb depth:32 packets (51200 bytes)
[I/USB] start uvc stream...
[I/uvc_app] frame #1  size=153600 fmt=yuyv
[I/uvc_app] UVC fps:29 disp:0 drop:17 urb:656 eof:30 fid:30 err:0 empty:0 data:6054 hdronl
y:1713 payload:4559700 done:29 last_off:153600
[I/uvc_app] frame #31  size=153600 fmt=yuyv
[I/uvc_app] UVC fps:29 disp:11 drop:72 urb:1381 eof:119 fid:119 err:0 empty:0 data:24167 h
dronly:6800 payload:18272000 done:118 last_off:153600
[I/uvc_app] frame #61  size=153600 fmt=yuyv
```

Startup Sequence

The M55 core depends on the M33 boot flow. Recommended flashing order:

```
+------------------+
|   Secure M33     |
|  (Secure Core)   |
+------------------+
          |
          v
+------------------+
|       M33        |
| (Non-Secure Core)|
+------------------+
          |
          v
+--------------------+
|       M55          |
```

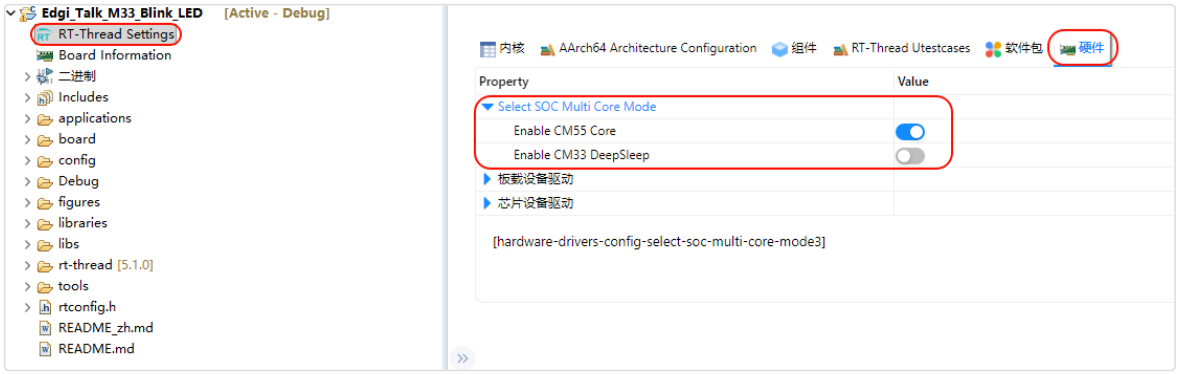
```
| (Application Core)|
+--------------------+
```

Notes

> ***Note:*** *RT-Thread Studio **2.2.9** or newer is recommended.*

- If this M55 project does not run, flash `Edgi_Talk_M33_Template` first.
- Enable CM55 in the M33 project:

```
RT-Thread Settings -> Hardware -> select SOC Multi Core Mode -> Enable CM55 C
```



4.5. Edgi_Talk_USB_Deepcraft Example Project

中文 | **English**

Overview

This project runs on the **Edgi-Talk M55 core** and bridges onboard sensors to host tools over **CherryUSB Device CDC ACM** using a **Protocol Buffers (nanopb)** based streaming protocol.

At startup, firmware:

1. Creates a protocol context (`protocol_create`) with a board UUID.
2. Registers device drivers (currently microphone and LSM6DS3 IMU).
3. Initializes USB CDC device transport.
4. Enters a request-processing loop via `protocol_process_request` .

Default Configuration

Key macros in `rtconfig.h` :

- `RT_USING_CHERRYUSB = y`
- `RT_CHERRYUSB_DEVICE = y`
- `RT_CHERRYUSB_DEVICE_DWC2_INFINEON = y`
- `RT_CHERRYUSB_DEVICE_CDC_ACM = y`
- `RT_CHERRYUSB_DEVICE_TEMPLATE_CDC_ACM = y`

USB identity from `applications/deepcraft/deepcraft_usbd.c` :

- VID: `0x058B`
- PID: `0x027D`
- Product string: `Imagimob Streaming Device`

Integrated Device Capabilities

`system_load_device_drivers()` currently registers:

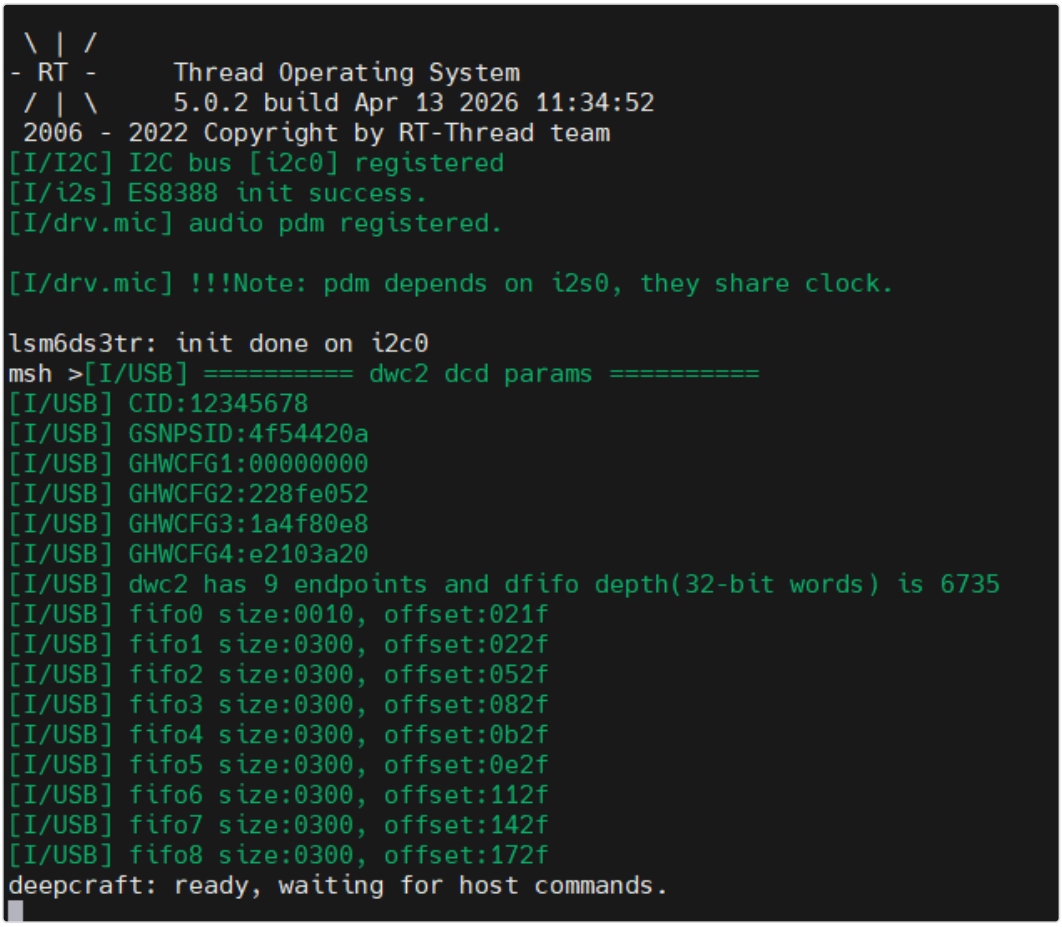
- **Microphone (PDM/PCM)**
- Options: gain, mono/stereo, sample frequency
- Stream type: `S16` output stream
- **LSM6DS3 IMU**
- Options: sample rate, accel range, gyro range, stream mode
- Modes: Combined / Split / Only Accel / Only Gyro

Build and Flash

1. Build in RT-Thread Studio or with SCons.
2. Flash firmware via KitProg3 (DAP).
3. Connect the board to host through Type-C and wait for CDC enumeration.

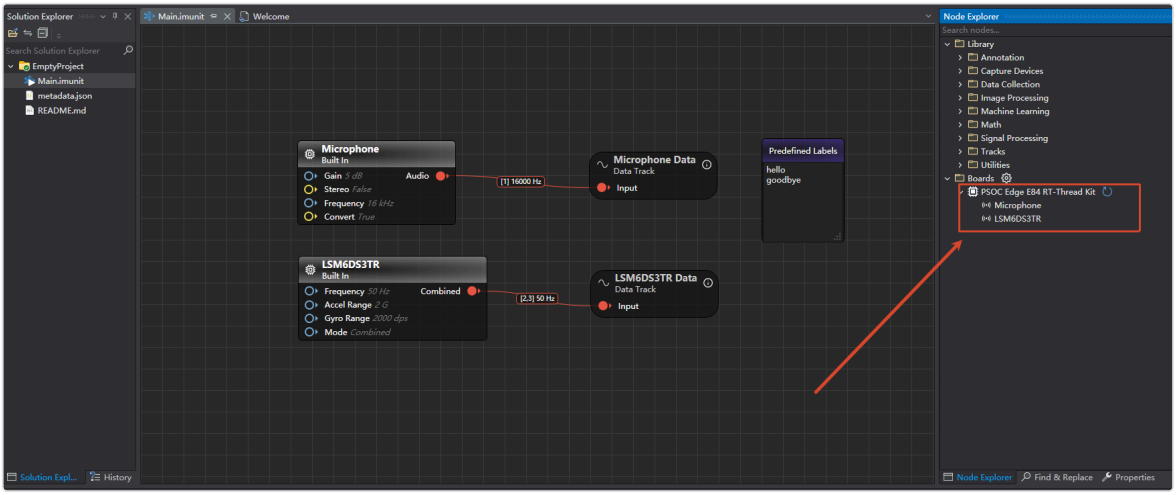
On successful startup, UART prints:

- `deepcraft: ready, waiting for host commands.`

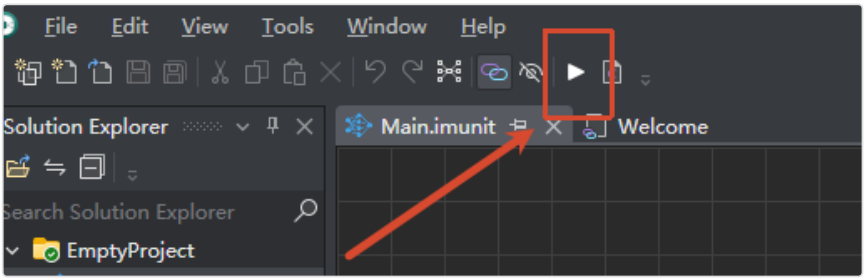


Deepcraft Studio Usage

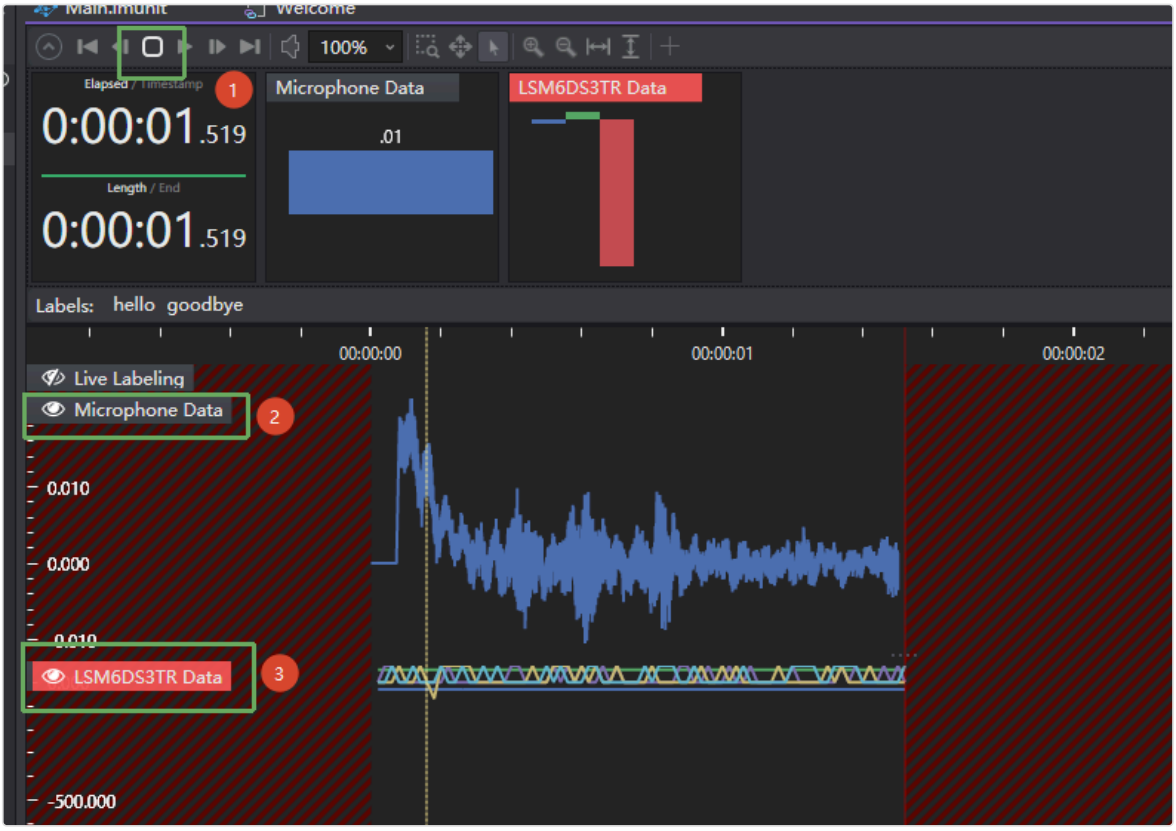
1. Download and install Deepcraft Studio:
   Install & Download Studio
2. Create a blank project:
   Real-Time Data Streaming with PSOC Edge E84 HMI
3. Connect the board to your PC with a USB data cable, then verify the device appears in Deepcraft Studio.



4. Click the build/run option to enter the data collection window.



5. Start recording and verify microphone plus 3-axis IMU data are streamed in real time.



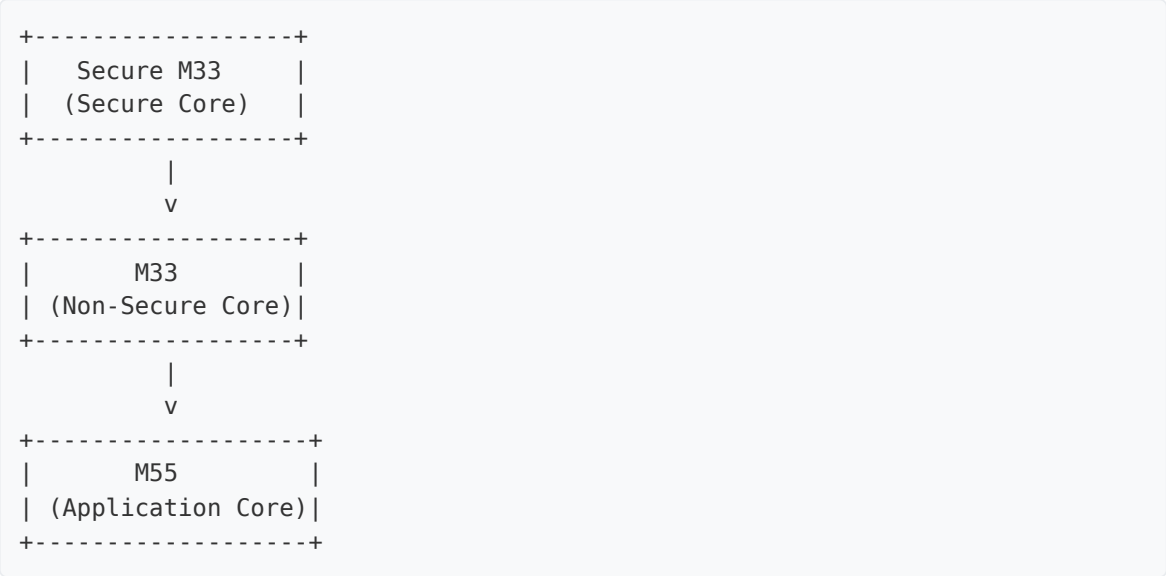
For more tutorials, see:
Data Collection Microphone

Host Communication Notes

- Transport: USB CDC ACM
- Encoding: nanopb / protobuf
- Error recovery:
- If request decode fails (`PROTOCOL_STATUS_FAILED_TO_DECODE_REQUEST`), firmware calls `deepcraft_usbd_resync_rx()` to clear and resync the RX ring buffer.

Startup Sequence

The M55 core depends on the M33 boot flow. Recommended flashing order:

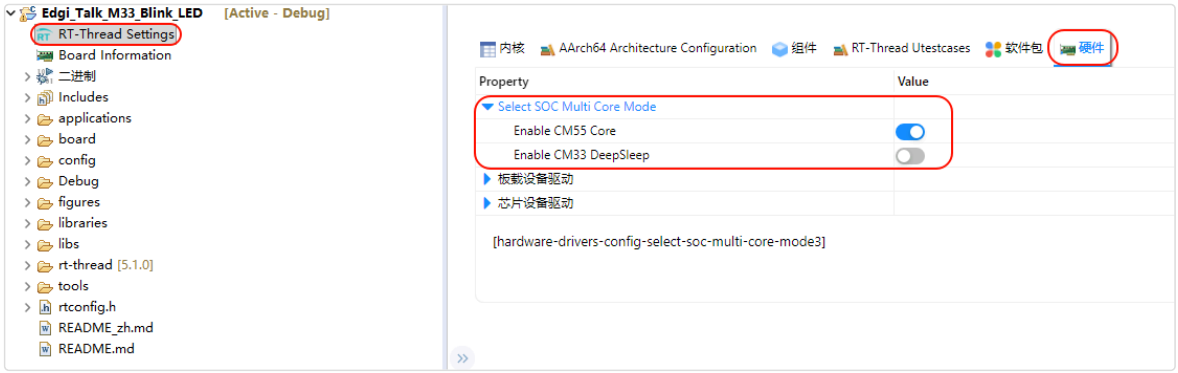


Notes

> ***Note:*** *RT-Thread Studio **2.2.9** or newer is recommended.*

- If this M55 project does not run, flash `Edgi_Talk_M33_Template` first.
- Enable CM55 in the M33 project:

```
RT-Thread Settings -> Hardware -> select SOC Multi Core Mode -> Enable CM55 C
```


Edgi_Talk_M33_Blink_LED [Active - Debug]
RT-Thread Settings
Board Information
二进制
Includes
applications
board
config
Debug
figures
libraries
libs
rt-thread [5.1.0]
tools
rtconfig.h
README_zh.md
README.md
内核
AArch64 Architecture Configuration
组件
RT-Thread Utestcases
软件包
硬件
Property
Value
Select SOC Multi Core Mode
Enable CM55 Core
Enable CM33 DeepSleep
板载设备驱动
芯片设备驱动
[hardware-drivers-config-select-soc-multi-core-mode3]

5. IPC

5.1. Edgi_Talk_M33_IPC Dual-Core Communication Example Project

中文 | **English**

Overview

This project implements **IPC (Inter-Processor Communication)** dual-core communication on the **Edgi-Talk M33 core**, demonstrating message passing between **Cortex-M33** and **Cortex-M55**.

M33 acts as the **sender**, periodically sending "Hello M33" messages to M55 and receiving "Hello M55" replies.

Default Configuration

- Communication frequency: 1000ms
- Message format: ASCII encoded text messages
- Uses Infineon PSoC E84 on-chip IPC Pipe hardware

Build and Flash

1. Build the project using RT-Thread Studio or SCons.
2. Flash firmware to M33 core via KitProg3 (DAP).
3. Both M33 and M55 projects must be built and flashed for dual-core communication.
4. Connect serial port to view communication logs as shown below:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Mar 26 2026 16:14:12
 2006 - 2022 Copyright by RT-Thread team

=======================================
   RT-Thread on Cortex-M33 Core
=======================================
Command: ipc_test_run
Status:  Running
=======================================

msh />ipc_test_run

=======================================
[M33] IPC Demo Started
---------------------------------------
Mode:     Sender
Target:   Cortex-M55
Message:  Hello M33
Interval: 1000 ms
=======================================

[M33] TX -> [M55]: "Hello M33" | Seq:     1 | Time:     5403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     1 | Time:     5413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     2 | Time:     6403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     2 | Time:     6413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     3 | Time:     7403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     3 | Time:     7413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     4 | Time:     8403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     4 | Time:     8413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     5 | Time:     9403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     5 | Time:     9413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     6 | Time:    10403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     6 | Time:    10413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     7 | Time:    11403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     7 | Time:    11413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     8 | Time:    12403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     8 | Time:    12413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     9 | Time:    13403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     9 | Time:    13413 ms
[M33] TX -> [M55]: "Hello M33" | Seq:    10 | Time:    14403 ms
[M33] RX <- [M55]: "Hello M55" | Seq:    10 | Time:    14413 ms
---------------------------------------
[M33] Statistics: TX=10, RX=10
```

Important Notes

Serial Connection

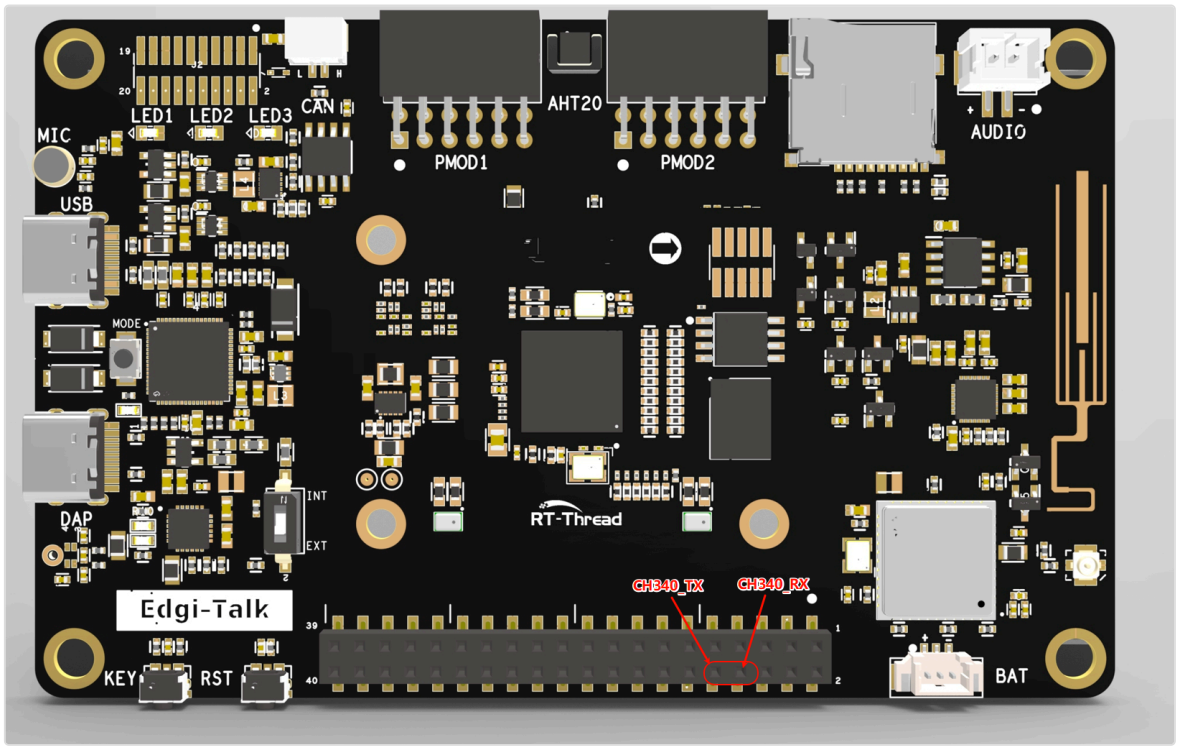
This project uses **UART5** as the debug serial port. An external **CH340 USB-to-Serial module** is required:

- **Serial Port**: UART5
- **Baud Rate**: 115200
- **Data Bits**: 8
- **Stop Bits**: 1
- **Parity**: None

CH340 Module Connection

Connect the CH340 module to the development board as follows:

Connection Diagram:



> ⚠️ ***Important Notes**:*
> *1. TX and RX must be cross-connected (CH340 TX to board RX)*
> *2. Ensure GND is common between both devices*

- To run an M55 project, it is recommended to flash **Edgi_Talk_M33_Template** first. It is a clean M33 project and is suitable as the base firmware before starting M55.

Usage

1. Configuration and Build

Open in RT-Thread Studio:

```
RT-Thread Settings -> Hardware Drivers Config -> On-chip Peripheral Drivers
```

Check `Enable IPC` option.

2. Run Example

Step 1: Start M55 Receiver

In M55 serial terminal, type:

```
msh> ipc_test_run
```

M55 will enter listening mode, waiting for messages from M33.

Step 2: Start M33 Sender

In M33 serial terminal, type:

```
msh> ipc_test_run
```

M33 will start sending "Hello M33" messages every second and receive "Hello M55" replies.

3. Observe Output

M33 Output Example:

```
========================================
[M33] IPC Demo Started
----------------------------------------
Mode:     Sender
Target:   Cortex-M55
Message:  Hello M33
Interval: 1000 ms
========================================

[M33] TX -> [M55]: "Hello M33" | Seq:     1 | Time:     1234 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     1 | Time:     1256 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     2 | Time:     2234 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     2 | Time:     2256 ms
----------------------------------------
[M33] Statistics: TX=10, RX=10
----------------------------------------
```

M55 Output Example:

```
========================================
[M55] IPC Listener Running
----------------------------------------
Mode:     Receiver & Responder
Peer:     Cortex-M33
Response: Hello M55
========================================

[M55] RX <- [M33]: "Hello M33" | Seq:     1 | Time:     1250 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     1 | Time:     1252 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     2 | Time:     2250 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     2 | Time:     2252 ms
----------------------------------------
[M55] Statistics: RX=10, TX=10
----------------------------------------
```

Data Protocol

IPC communication uses the `edge_rc_frame_t` structure:

```
typedef struct {
    uint8_t client_id;          // Client ID
```

```
    uint16_t intr_mask;         // Interrupt mask
    uint8_t role;               // Role identifier (M33/M55_ECHO)
    uint32_t magic;             // Magic word (0x5243444DU)
    uint32_t seq;               // Sequence number
    uint16_t channel[8];        // 8-channel data (stores ASCII messages)
    uint32_t checksum;          // Checksum
} edge_rc_frame_t;
```

Message "Hello M33" encoding:

```
channel[0] = 0x4865  /* 'He' */
channel[1] = 0x6C6C  /* 'll' */
channel[2] = 0x6F20  /* 'o '  (space) */
channel[3] = 0x4D33  /* 'M3' */
channel[4] = 0x0033  /* '3' */
```

Startup Sequence

M55 depends on M33 boot flow. Flash in this order:

```
+------------------+
|   Secure M33     |
|  (Secure Core)   |
+------------------+
          |
          v
+------------------+
|       M33        |
|   (IPC Sender)   |
+------------------+
          |
          v
+-------------------+
|       M55         |
|  (IPC Receiver)   |
+-------------------+
```

Hardware Connection

This example uses Infineon PSoC E84 on-chip IPC hardware:

- Uses IPC Pipe hardware in CM0P
- CM33 uses EP1 (Endpoint 1)
- CM55 uses EP2 (Endpoint 2)
- Uses semaphores for mutual exclusion
- No external wiring required

Configuration Parameters

The following parameters can be modified in `libraries/HAL_Drivers/ipc_common.h`:

- `EDGE_IPC_FRAME_POOL_SIZE`: Transmit buffer pool size (default 64)
- `EDGE_IPC_RX_QUEUE_SIZE`: Receive queue size (default 128)
- `EDGE_IPC_SEMA_RETRY_MAX`: Semaphore retry count (default 2)

Notes

> ⚠️ ***Note:*** *This project requires* ***RT-Thread Studio 2.2.9*** *or higher.*

- This project targets M33 core as IPC sender.
- For M55 receiver, see projects/Edgi_Talk_IPC/Edgi_Talk_M55_IPC/README.md.
- IPC driver is located at `libraries/HAL_Drivers/drv_ipc.c`.
- Ensure both cores are configured with `BSP_USING_IPC`.

References

- IPC driver referenced from `D:\Desktop\Fmt\FMT-Firmware\target\infineon\edge-e83`
- Infineon PSoC E84 Technical Reference Manual

5.2. Edgi_Talk_M55_IPC Dual-Core Communication Example Project

中文 | **English**

Overview

This project implements **IPC (Inter-Processor Communication)** dual-core communication on the **Edgi-Talk M55 core**, demonstrating message passing between **Cortex-M55** and **Cortex-M33**.

M55 acts as the **receiver and responder**, receiving "Hello M33" messages from M33 and replying with "Hello M55".

Default Configuration

- Response mode: Automatic reply
- Message format: ASCII encoded text messages
- Uses Infineon PSoC E84 on-chip IPC Pipe hardware

Build and Flash

1. Build the project using RT-Thread Studio or SCons.
2. Flash firmware to M55 core via KitProg3 (DAP).
3. Both M33 and M55 projects must be built and flashed for dual-core communication.
4. Connect serial port to view communication logs as shown below:

```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.2 build Mar 26 2026 16:12:28
 2006 - 2022 Copyright by RT-Thread team

=======================================
   RT-Thread on Cortex-M55 Core
=======================================
Command: ipc_test_run
Status:  Running
=======================================

msh />ipc_test_run
msh />
=======================================
[M55] IPC Listener Running
---------------------------------------
Mode:     Receiver & Responder
Peer:     Cortex-M33
Response: Hello M55
=======================================

[M55] RX <- [M33]: "Hello M33" | Seq:     1 | Time:     5429 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     1 | Time:     5429 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     2 | Time:     6429 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     2 | Time:     6429 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     3 | Time:     7439 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     3 | Time:     7439 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     4 | Time:     8439 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     4 | Time:     8439 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     5 | Time:     9439 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     5 | Time:     9439 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     6 | Time:    10439 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     6 | Time:    10439 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     7 | Time:    11449 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     7 | Time:    11449 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     8 | Time:    12449 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     8 | Time:    12449 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     9 | Time:    13449 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     9 | Time:    13449 ms
[M55] RX <- [M33]: "Hello M33" | Seq:    10 | Time:    14449 ms
[M55] TX -> [M33]: "Hello M55" | Seq:    10 | Time:    14449 ms
---------------------------------------
[M55] Statistics: RX=10, TX=10
```

Usage

1. Configuration and Build

Open in RT-Thread Studio:

```
RT-Thread Settings -> Hardware Drivers Config -> On-chip Peripheral Drivers
```

Check `Enable IPC` option.

2. Run Example

In M55 serial terminal, type:

```
msh> ipc_test_run
```

M55 will start a listener thread, waiting for messages from M33 and automatically replying.

3. Observe Output

M55 Output Example:

```
========================================
[M55] IPC Listener Running
----------------------------------------
Mode:     Receiver & Responder
Peer:     Cortex-M33
Response: Hello M55
========================================

[M55] RX <- [M33]: "Hello M33" | Seq:     1 | Time:     1250 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     1 | Time:     1252 ms
[M55] RX <- [M33]: "Hello M33" | Seq:     2 | Time:     2250 ms
[M55] TX -> [M33]: "Hello M55" | Seq:     2 | Time:     2252 ms
----------------------------------------
[M55] Statistics: RX=10, TX=10
----------------------------------------
```

Corresponding M33 Output:

```
========================================
[M33] IPC Demo Started
----------------------------------------
Mode:     Sender
Target:   Cortex-M55
Message:  Hello M33
Interval: 1000 ms
========================================

[M33] TX -> [M55]: "Hello M33" | Seq:     1 | Time:     1234 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     1 | Time:     1256 ms
[M33] TX -> [M55]: "Hello M33" | Seq:     2 | Time:     2234 ms
[M33] RX <- [M55]: "Hello M55" | Seq:     2 | Time:     2256 ms
----------------------------------------
[M33] Statistics: TX=10, RX=10
----------------------------------------
```

Data Protocol

IPC communication uses the `edge_rc_frame_t` structure:

```
typedef struct {
    uint8_t client_id;          // Client ID
    uint16_t intr_mask;         // Interrupt mask
    uint8_t role;               // Role identifier (M33/M55_ECHO)
    uint32_t magic;             // Magic word (0x5243444DU)
    uint32_t seq;               // Sequence number
    uint16_t channel[8];        // 8-channel data (stores ASCII messages)
    uint32_t checksum;          // Checksum
} edge_rc_frame_t;
```

Message "Hello M55" encoding:

```
channel[0] = 0x4865  /* 'He' */
channel[1] = 0x6C6C  /* 'll' */
channel[2] = 0x6F20  /* 'o '  (space) */
channel[3] = 0x4D35  /* 'M5' */
channel[4] = 0x0035  /* '5' */
```

Startup Sequence

M55 depends on M33 boot flow. Flash in this order:

Hardware Connection

This example uses Infineon PSoC E84 on-chip IPC hardware:

- Uses IPC Pipe hardware in CM0P
- CM33 uses EP1 (Endpoint 1)
- CM55 uses EP2 (Endpoint 2)
- Uses semaphores for mutual exclusion
- No external wiring required

Working Principle

1. **Receive messages**: M55 continuously reads from IPC device in a separate thread
2. **Verify messages**: Check magic word, role identifier, and checksum
3. **Parse messages**: Verify if it's a "Hello M33" message
4. **Send reply**: Construct "Hello M55" message and reply to M33
5. **Statistics**: Print statistics every 10 messages processed

Configuration Parameters

The following parameters can be modified in `libraries/HAL_Drivers/ipc_common.h`:

- `EDGE_IPC_FRAME_POOL_SIZE`: Transmit buffer pool size (default 64)
- `EDGE_IPC_RX_QUEUE_SIZE`: Receive queue size (default 128)
- `EDGE_IPC_SEMA_RETRY_MAX`: Semaphore retry count (default 2)

6. AI

6.1. Edgi_Talk_M55_DEEPCRAFT_Deploy_Vision

English (current) | 中文

1. Overview

This project is an **RT-Thread implementation** of `PSOC Edge MCU: Machine learning - DEEPCRAFT deploy vision`, running on the Edgi-Talk **CM55** core, using:

- CherryUSB Host (DWC2, Infineon)
- UVC USB camera capture
- DEEPCRAFT/TFLM model inference
- Real-time LCD rendering of bounding boxes and labels

It keeps the same functional goal as the ModusToolbox example (real-time vision detection + UART logs), but the software framework is RT-Thread (MSH, SCons, RT-Thread Studio).

---

2. Main Differences vs. ModusToolbox Example

- OS/project framework: ModusToolbox -> RT-Thread project structure and configuration flow
- Runtime control: start/stop UVC stream through MSH commands (`usbh_uvc_start` / `usbh_uvc_stop`)
- USB app layer location: `libraries/Common/board/ports/usb`
- AI app layer location: `libraries/Common/deepcraft_ai` and `applications/uvc_ai_app.c`
- Model files are split out into:
- `libraries/Common/deepcraft_ai/model/model.c`
- `libraries/Common/deepcraft_ai/model/model.h`

---

3. Key Directory Paths

- App entry: `applications/main.c`
- AI app logic: `applications/uvc_ai_app.c`
- UVC app layer: `libraries/Common/board/ports/usb/usbh_uvc_app.c`

- UVC stream processing: `libraries/Common/board/ports/usb/usbh_uvc_stream.c`
- Model files: `libraries/Common/deepcraft_ai/model/model.c`, `libraries/Common/deepcraft_ai/model/model.h`
- AI core processing: `libraries/Common/deepcraft_ai/src/uvc_ai.c`

---

4. Environment and Dependencies

- RT-Thread Studio (2.2.9 or newer recommended)
- GNU Arm toolchain (with Cortex-M55 + Helium/MVE support)
- Board: PSOC Edge E84 family (Edgi-Talk target hardware)
- Camera: UVC USB camera (320x240 YUYV recommended)
- Display: project default LCD configuration

> *Note: This project depends on the M33 boot chain to enable CM55. If CM55 does not start, verify M33 project configuration and flashing first.*

---

5. Important Configuration Items

Check these options in `rtconfig.h` / `.config`:

- `RT_USING_CHERRYUSB`
- `RT_CHERRYUSB_HOST`
- `RT_CHERRYUSB_HOST_DWC2_INFINEON`
- `RT_CHERRYUSB_HOST_VIDEO`
- `BSP_USING_DEEPCRAFT_AI`
- `RT_USING_MSH`, `RT_USING_FINSH`

If you want local changes in `usbh_uvc_stream.c` / `usbh_uvc_port.c` to take effect, enable:

- `RT_CHERRYUSB_HOST_BUILD_FROM_SOURCE`

---

6. Build and Flash

6.1 RT-Thread Studio

1. Import `projects/Edgi_Talk_M55_DEEPCRAFT_Deploy_Vision`
2. Check configuration (menuconfig / Settings)

3. Build
4. Flash via KitProg3

6.2 Command line (SCons)

Run in project directory:

```
scons -j8
```

Generated artifacts include:

- `rt-thread.elf`
- `rtthread.hex`

---

7. Run Steps

1. Connect UART terminal at 115200 8N1
2. Connect LCD and UVC camera
3. After boot, run in MSH:

```
usbh_uvc_start 0 320 240
```

Parameters:

- `0` = YUYV
- `1` = MJPEG

> *Note: In AI mode, the app forces `YUYV + 320x240` (other input parameters are overridden).*

Stop command:

```
usbh_uvc_stop
```

7.1 Runtime Screenshots

- Figure 1: System running and detection display

- Figure 2: Camera frame with detection boxes

- Figure 3: UART logs and detection output

---

8. Logs and Behavior

- UVC status logs:
- `UVC fps:...`
- `start uvc stream...`
- AI logs:
- `AI inference ...`
- `AI #... bbox=[...]

---

9. Error Handling

If you see:

- `uvc abort1` / `uvc abort2`

The current implementation triggers a simplified auto-recovery flow (restart UVC stream) and prints:

- `UVC stream restart (...)`

If this happens frequently, prioritize checking:

- camera power/cable quality
- UVC bandwidth and resolution settings
- application load (display refresh, AI inference, log volume)

---

10. Model Replacement

This project now manages model files in a dedicated folder (aligned with the reference structure):

- `libraries/Common/deepcraft_ai/model/model.c`
- `libraries/Common/deepcraft_ai/model/model.h`

When replacing model files, keep exported interfaces compatible with current code (for example `IMAI_init / IMAI_compute / IMAI_finalize`) and rebuild.

---

11. Multi-core Boot Chain

Recommended flashing order:

1. CM33 Secure
2. CM33 Non-Secure (enables CM55)
3. CM55 this project

If CM55 does not run, make sure CM55 is enabled in the M33 project configuration.

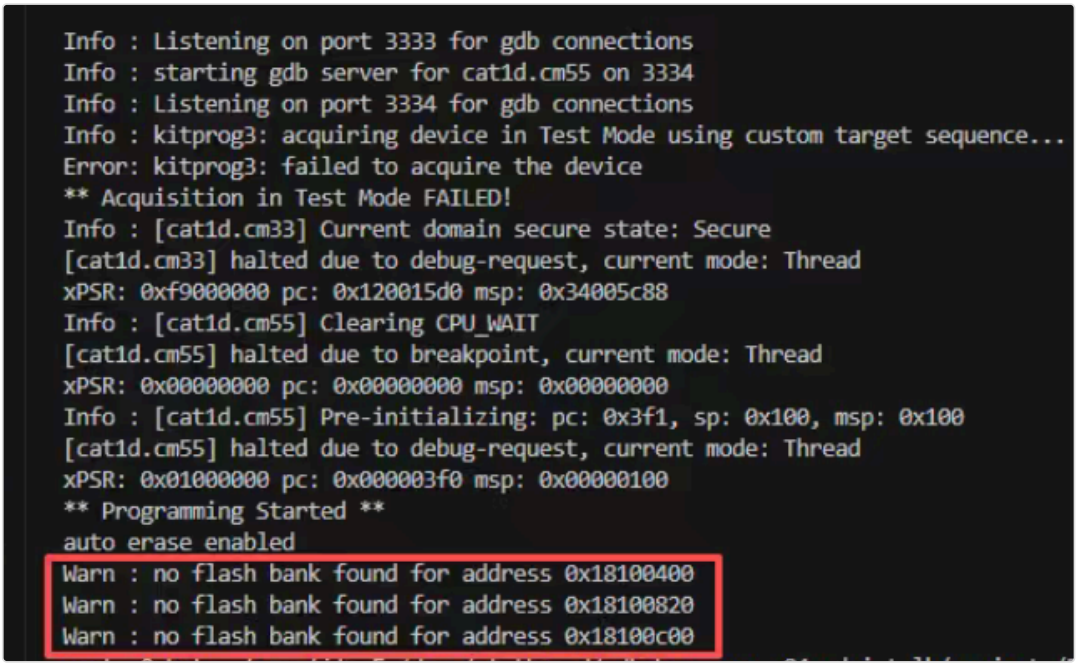
7. FAQ

7.1. 常见问题解答（FAQ）

本文档收集了用户在使用过程中遇到的常见问题及解决方案。

一、烧录

Q1. 烧录固件出错



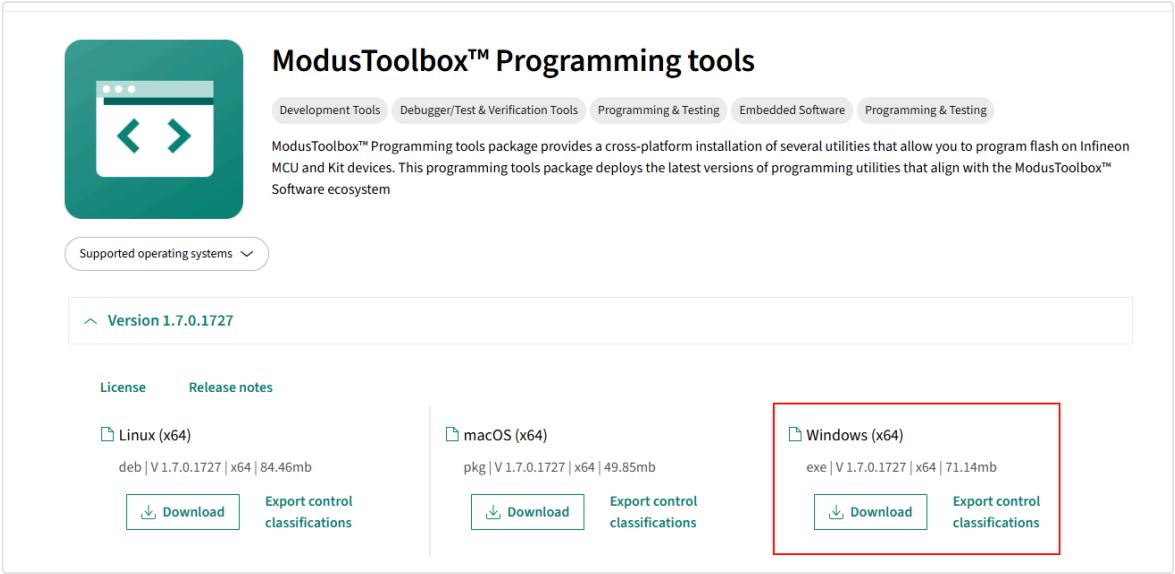
Log Console

```
** RRAM Boot version: 2.0.0.7127
** SE RT Services Base version: 1.0.0.2361
** SE RT Services version: 0.0.0.0
** Extended Boot version: 1.1.0.1700
** Boot Status : CYBOOT_SUCCESS
** Life Cycle  : DEVELOPMENT
***********************************
Info : [cat1d.sys] Examination succeed
Info : [cat1d.sys33] Examination succeed
Info : [cat1d.cm33] Examination succeed
Info : [cat1d.cm55] Clearing CPU_WAIT
Warn : [cat1d.cm55] Failed to clear CPU_WAIT:
Info : [cat1d.cm55] Examination succeed
Info : gdb port disabled
Info : gdb port disabled
Info : starting gdb server for cat1d.cm33 on 3333
Info : Listening on port 3333 for gdb connections
Info : starting gdb server for cat1d.cm55 on 3334
Info : Listening on port 3334 for gdb connections
Info : SWD DPIDR 0x4c013477
Warn : Connecting DP: stalled AP operation, issuing ABORT
Info : SWD DPIDR 0x4c013477
Error executing event examine-end on target cat1d.cm55:
../scripts/target/infineon/cat1d/func_cat1d_pse84.cfg:18: Error:
in procedure 'program'
in procedure 'ocd_process_reset'
in procedure 'ocd_process_reset_inner' called at file "embedded:startup.tcl", line 1225
at file "../scripts/target/infineon/cat1d/cat1d.cfg", line 81
at file "../scripts/target/infineon/cat1d/func_cat1d_pse84.cfg", line 18
** Acquisition in Test Mode FAILED!
Info : [cat1d.cm33] Current domain secure state: Secure
[cat1d.cm33] halted due to debug-request, current mode: Thread
xPSR: 0xf9000000 pc: 0x120015d0 msp: 0x34005c88
Info : [cat1d.cm55] Clearing CPU_WAIT
Warn : [cat1d.cm55] Failed to clear CPU_WAIT:
** Unable to reset target **
shutdown command invoked
```

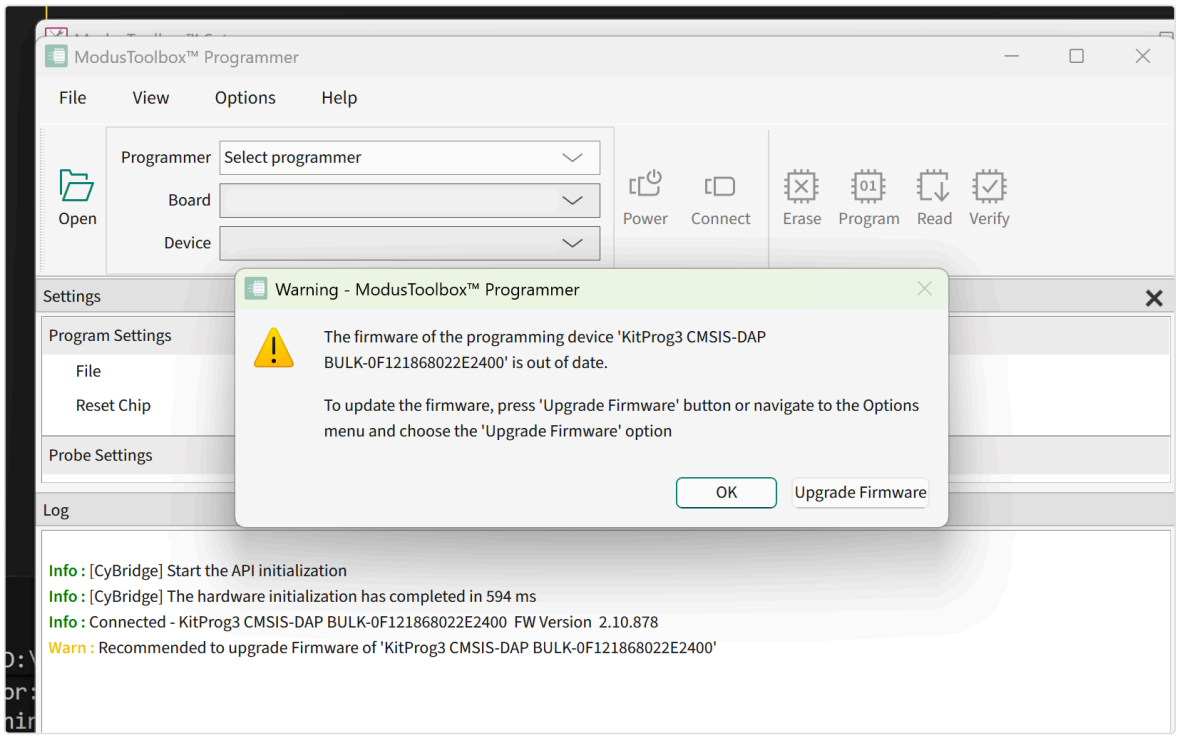
解决方法：

首先下载如下链接的上位机，我们需要进行固件更新：

ModusToolbox™ Programming tools - Infineon Developer Center



如果提示如下的警告，说明你开发板上面的DAP-Link固件需要更新，才可以烧录固件，这时需要点击 **Upgrade Fimeware** 按钮



更新完成后，重新上电开发板，然后即可下载程序了

```
[cat1d.cm55] halted due to breakpoint, current mode: Thread
xPSR: 0x00000000 pc: 0x41e9ac50 msp: 0xb71c5c44
Info : [cat1d.cm55] Pre-initializing: pc: 0x3f1, sp: 0x100, msp: 0x100
[cat1d.cm55] halted due to debug-request, current mode: Thread
xPSR: 0x01000000 pc: 0x000003f0 msp: 0x00000100
** Programming Started **
[ 50%] [################                ] [ Erasing     ]
[100%] [################################] [ Erasing     ]
[ 12%] [###                             ] [ Programming ]
[ 14%] [####                            ] [ Programming ]
[ 18%] [#####                           ] [ Programming ]
[ 25%] [########                        ] [ Programming ]
[ 30%] [#########                       ] [ Programming ]
[ 37%] [###########                     ] [ Programming ]
[ 38%] [############                    ] [ Programming ]
[ 44%] [##############                  ] [ Programming ]
[ 50%] [################                ] [ Programming ]
[ 58%] [##################              ] [ Programming ]
[ 63%] [####################            ] [ Programming ]
[ 71%] [######################          ] [ Programming ]
[ 76%] [########################        ] [ Programming ]
[ 85%] [##########################      ] [ Programming ]
[ 89%] [############################    ] [ Programming ]
[ 99%] [##############################  ] [ Programming ]
[100%] [################################] [ Programming ]
** Programming Finished **
shutdown command invoked
```

Q2. 为什么烧录了Edgi_Talk_M55_XXX工程后，没有现象？

解决方法：

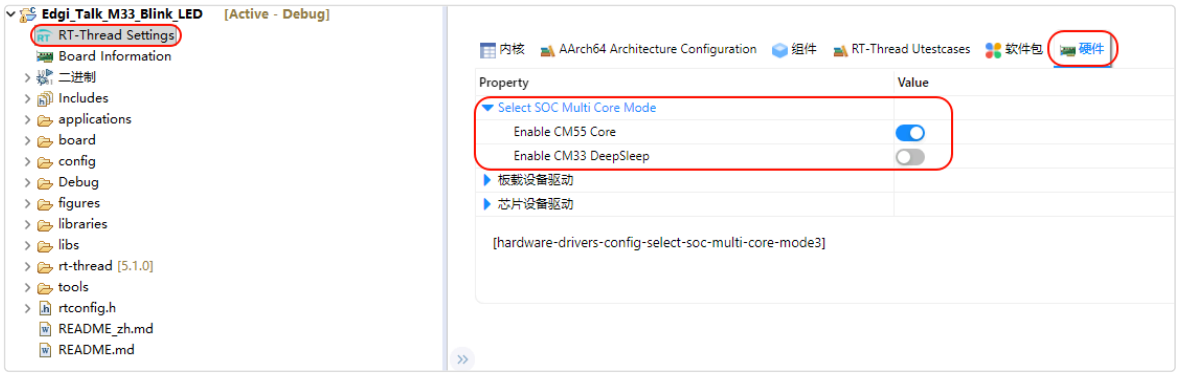
芯片启动顺序如下：

```
+------------------+
|   Secure M33     |
|  （安全内核启动）  |
+------------------+
          |
          v
+------------------+
|       M33        |
|  （非安全核启动）  |
+------------------+
          |
          v
+-------------------+
|       M55         |
|  （应用处理器启动）  |
+-------------------+
```

⚠️ 请严格按照以上顺序烧写固件，否则系统可能无法正常运行。

- 若示例工程无法正常运行，建议先编译并烧录 **Edgi_Talk_M33_Template** 工程，确保初始化与核心启动流程正常，再运行本示例。
- 若要开启 M55，需要在 **M33 工程** 中打开配置：

```
RT-Thread Settings --> 硬件 --> select SOC Multi Core Mode --> Enable CM55 Core
```

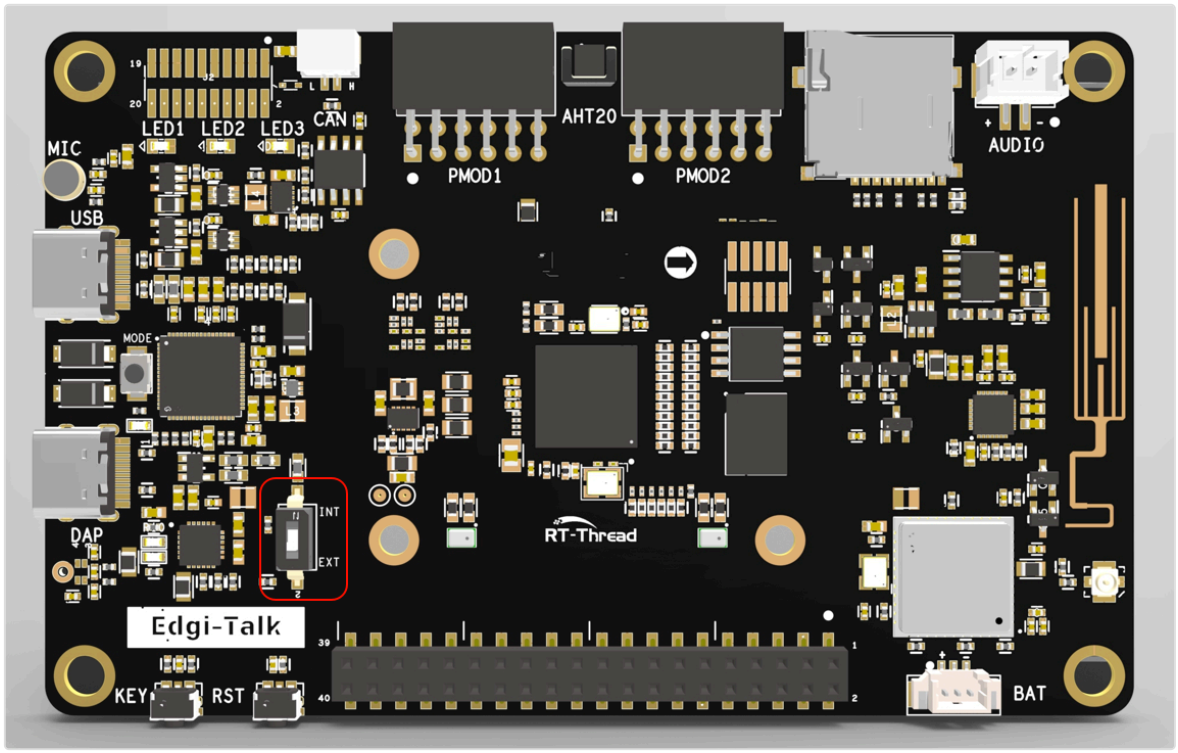


Q3.烧录固件出错

```
Info : kitprog3: acquiring device in Test Mode using custom target sequence...
Error: kitprog3: failed to acquire the device
Info : clock speed 4000 kHz
Info : SWD DPIDR 0x4c013477
** Target is not in Test Mode
**************************************
** Silicon: 0xED94, Family: 0x115, Rev.: 0x21 (B0)
** Detected Device: PSE846GPS2DBZC4A
** SROM Boot version: 2.0.0.6022
** RRAM Boot version: 2.0.0.7127
** SE RT Services Base version: 1.0.0.2361
** SE RT Services version: 0.0.0.0
** Extended Boot version: 1.1.0.1700
** Boot Status : CYBOOT_SUCCESS
** Life Cycle  : DEVELOPMENT
**************************************
Info : [cat1d.sys] Examination succeed
Info : [cat1d.sys33] Examination succeed
Info : SWD DPIDR 0x4c013477
Error: Failed to read memory at 0xe000ed00
Error: [cat1d.cm33] Examination failed
Warn : target cat1d.cm33 examination failed
Info : SWD DPIDR 0x4c013477
Error: Failed to read memory at 0xe000ed00
Error: [cat1d.cm55] Examination failed
Warn : target cat1d.cm55 examination failed
Info : gdb port disabled
Info : gdb port disabled
Info : starting gdb server for cat1d.cm33 on 3333
Info : Listening on port 3333 for gdb connections
Info : starting gdb server for cat1d.cm55 on 3334
Info : Listening on port 3334 for gdb connections
Info : SWD DPIDR 0x4c013477
Error: Failed to read memory at 0xe000ed00
Info : SWD DPIDR 0x4c013477
Error: Failed to read memory at 0xe000ed00
Info : kitprog3: acquiring device in Test Mode using custom target sequence...
Error: kitprog3: failed to acquire the device
** Acquisition in Test Mode FAILED!
Info : SWD DPIDR 0x4c013477
Info : SWD DPIDR 0x4c013477
Info : [cat1d.cm33] AP write error, reset will not halt
Info : SWD DPIDR 0x4c013477
Info : SWD DPIDR 0x4c013477

Info : cat1d.dap: powering down debug domain...
Info : SWD DPIDR 0x4c013477
```

解决方法：



1. 将 BOOT 配置切换至 **INT**
2. 重启开发板
3. 下载程序
4. 将 BOOT 配置切换至 **EXT**
5. 再次重启开发板

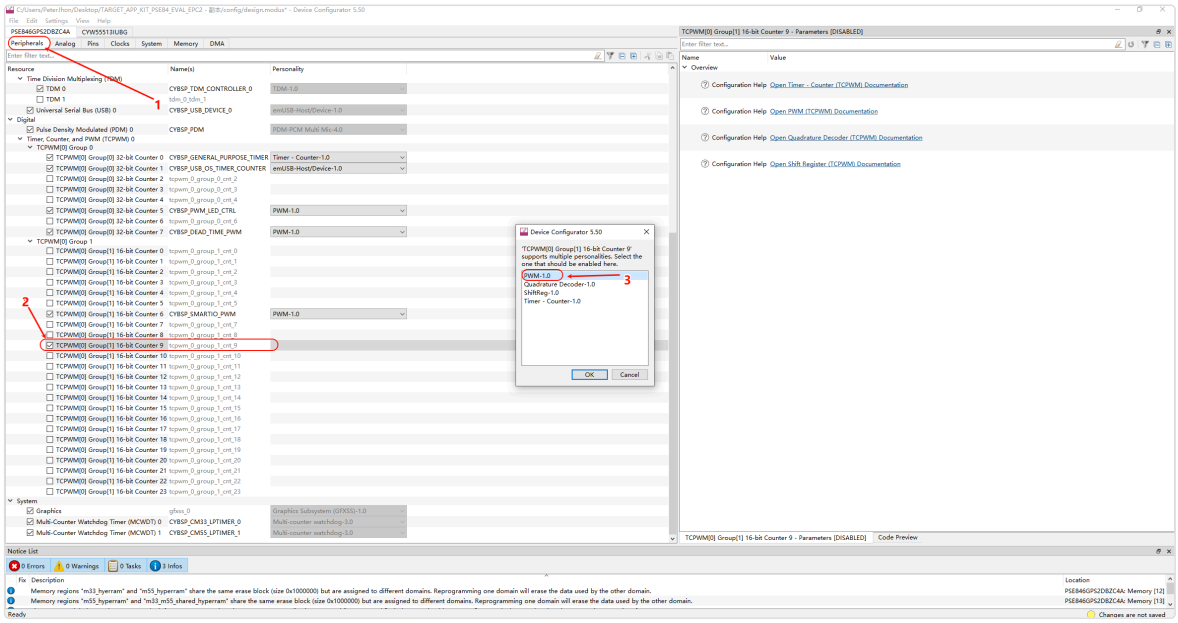
7.2. PWM Device Configurator 配置指南

打开配置文件

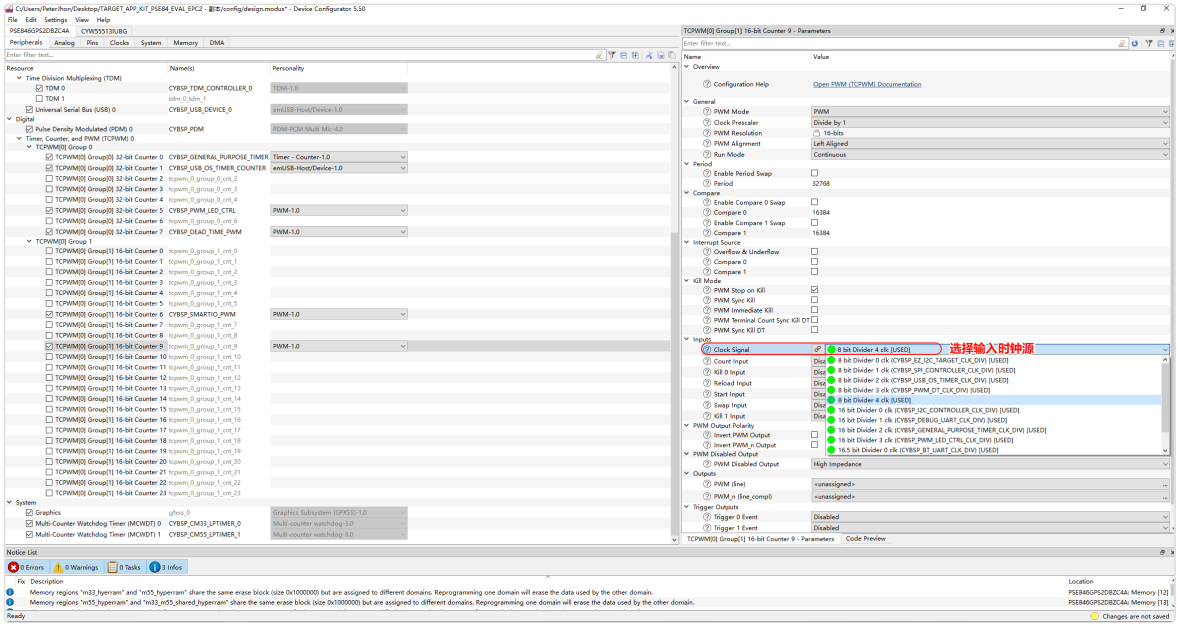
请使用以下工具打开配置文件：

```
tools/device-configurator/device-configurator.exe
libs/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
```

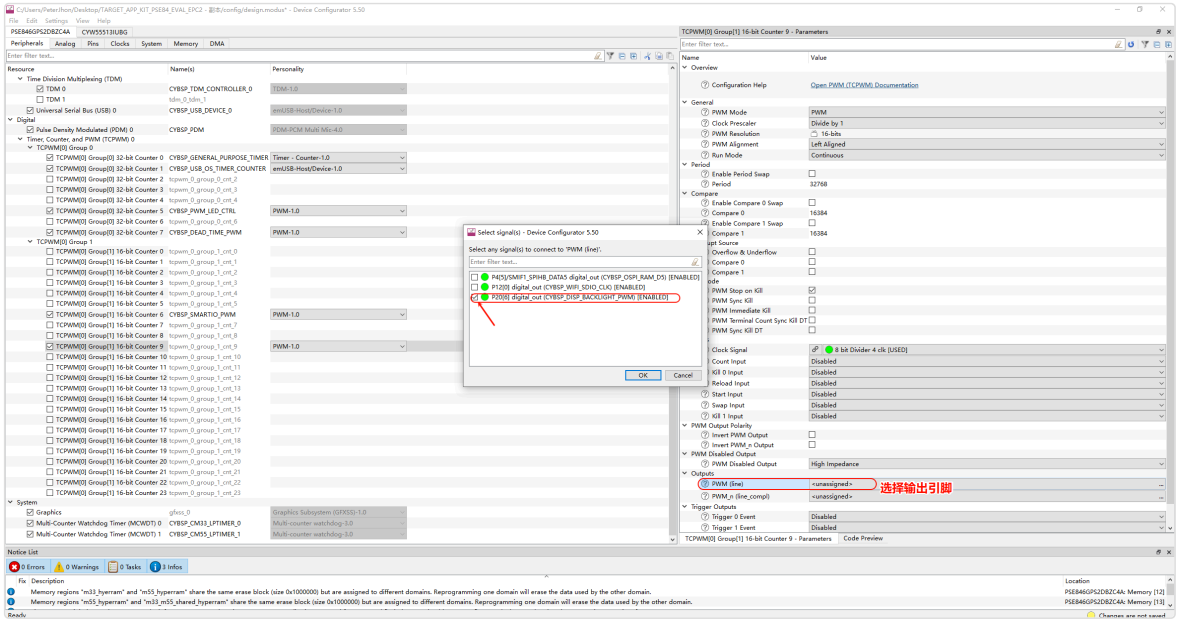
1. 配置 TCPWM 为 PWM 模式



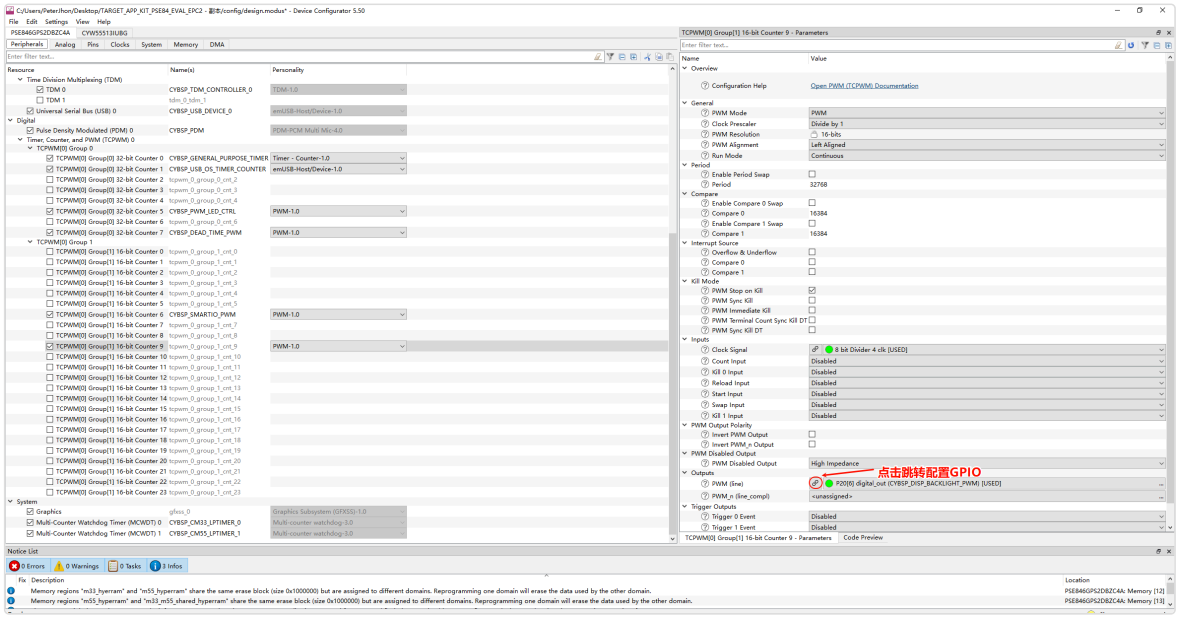
2. 选择输入时钟源

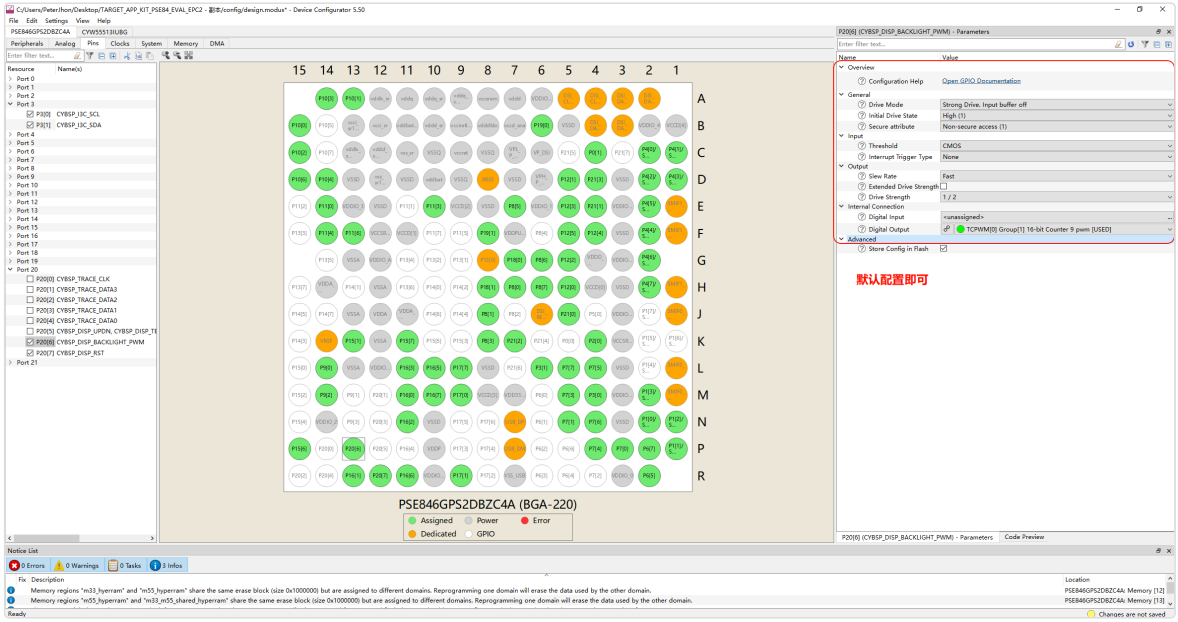


3. 设置 PWM 输出引脚



4. 设置输入引脚模式





保存配置

完成全部配置后，按 `Ctrl + S` 保存即可。